Cmod A7-35T RISC-V MCU

Datasheet & Reference Manual

MicroBlaze-V soft MCU on the Digilent Cmod A7-35T — peripherals, pins, power, and boot guides

Platform	Xilinx Artix-7 (xc7a35tcpg236-1) — Cmod A7-35T
Processor	MicroBlaze RISC-V
Toolchain	Vivado & Vitis 2025.2

Document DS-CMOD7-0001 · Revision A · 2027

© 2027 NCKU MPPA Course

Table of Contents

1 Device Overview . . . 4
1.1 Introduction . . . 4
1.2 Features . . . 4
1.3 On-Board Components . . . 4
1.4 System Block Diagram . . . 5
2 Pinout . . . 6
2.1 DIP Connector Overview . . . 6
2.2 Pin Map — Left Side (Pin 1–24) . . . 7
2.3 Pin Map — Right Side (Pin 25–48) . . . 7
2.4 On-Board I/O (No DIP Pin Exposure) . . . 8
3 Electrical Characteristics and Power . . . 9
3.1 Electrical Characteristics . . . 9
3.2 Power Architecture . . . 9
3.3 Power Input Options . . . 9
3.4 Output Power Rails . . . 10
3.4.1 Powering External Circuits . . . 10
3.5 VU Pin Behavior . . . 10
3.6 Dual Power Source (USB + External) . . . 10
4 System Architecture . . . 11
4.1 MicroBlaze RISC-V Core . . . 11
4.2 Clock . . . 11
4.3 Reset . . . 11
4.4 Interrupt Controller . . . 11
4.5 Debug Module . . . 12
4.6 Complete Address Map . . . 13
5 Memory Hierarchy . . . 14
5.1 Memory Hierarchy Overview . . . 14
5.2 Memory Map . . . 15
5.3 Tightly-Coupled Memory (ITCM / DTCM / Bootloader) . . . 15
5.4 External SRAM . . . 16
5.5 QSPI Flash . . . 16
5.6 Caches . . . 16
5.7 Measured Access Latencies . . . 17
5.8 Memory Placement Guidance . . . 17
5.8.1 Interrupt Latency . . . 18
6 Peripherals . . . 19
6.1 Register Access Conventions . . . 19
6.2 GPIO . . . 20
6.2.1 DIP Connector GPIO (3 Groups × 8-bit) . . . 20
6.2.2 GPIO Register Map . . . 20
6.2.3 On-Board GPIO . . . 21
6.3 External Interrupt Inputs . . . 23
6.3.1 Interrupt Register Map . . . 23
6.4 Timers and PWM . . . 25
6.4.1 System Timers . . . 25
6.4.2 PWM Outputs . . . 25
6.4.3 Timer/PWM Register Map . . . 25
6.5 UART . . . 27
6.5.1 USB UART (`uart_USB`) . . . 27
6.5.2 External UART (`uart_1`) . . . 27
6.5.3 UART Register Map . . . 27
6.6 I2C Controller . . . 30
6.6.1 I2C Register Map . . . 30
6.7 SPI Master . . . 32
6.7.1 SPI Register Map . . . 32
6.7.2 Serial Clock Control Unit . . . 33
6.8 Analog-to-Digital Converter (XADC) . . . 34
6.8.1 XADC Register Map . . . 34
6.8.2 Analog Input Circuit . . . 35
6.9 Interrupt Controller . . . 36
6.9.1 Interrupt Controller Register Map . . . 36
7 JTAG Debug Mode . . . 38

7.1 Prerequisites . . . 38
7.2 Open the Workspace . . . 38
7.3 Create a Platform . . . 40
7.3.1 New Platform Component . . . 40
7.3.2 Platform Name and Location . . . 40
7.3.3 Select the Hardware Design (XSA) . . . 41
7.3.4 Select Operating System and Processor . . . 42
7.3.5 Platform Summary . . . 42
7.4 Create an Application . . . 44
7.4.1 Create from Template . . . 44
7.4.2 Application Name and Location . . . 44
7.4.3 Select the Platform . . . 45
7.4.4 Select the Domain . . . 45
7.4.5 Application Summary . . . 46
7.5 Replace the Sources with the Course Template . . . 47
7.5.1 Delete the Generated Files . . . 47
7.5.2 Import the Template File . . . 48
7.5.3 Point the Component at the MCU Folder . . . 49
7.5.4 About the Linker Script . . . 49
7.6 Set the Console UART (BSP) . . . 49
7.7 Build . . . 51
7.7.1 Build the Platform . . . 51
7.7.2 Build the Application . . . 51
7.8 Debug over JTAG . . . 53
7.8.1 Start a Debug Session . . . 53
7.8.2 Open a Serial Terminal, Then Continue . . . 54
7.8.3 Breakpoints and Stepping . . . 55
7.9 Inspection Tools . . . 55
7.9.1 Memory Inspector . . . 55
7.9.2 Register Inspector . . . 56
7.9.3 Disassembly View . . . 56
7.10 Troubleshooting . . . 57
8 Standalone Boot . . . 60
8.1 How Standalone Boot Works . . . 60
8.2 Prerequisites . . . 61
8.3 Create the Bootloader Component . . . 61
8.3.1 New Application from Template . . . 61
8.3.2 Name and Location . . . 62
8.3.3 Select the Platform . . . 63
8.3.4 Summary . . . 63
8.4 Configure the Bootloader . . . 64
8.4.1 Set the Application Slot Address . . . 64
8.4.2 Disable Verbose Output . . . 65
8.5 Build the Bootloader . . . 66
8.6 Merge the Bootloader into the Hardware Image . . . 67
8.7 Prepare the Application Image . . . 68
8.8 Program the Flash . . . 69
8.8.1 Program the Hardware Image . . . 70
8.8.2 Program the Application . . . 71
8.9 Boot and Verify . . . 72
8.10 Updating the Application . . . 72
8.11 Working with JTAG Debug Mode . . . 72
8.12 Troubleshooting . . . 72

1 Device Overview

1.1 Introduction

The Cmod A7-35T RISC-V MCU is a soft microcontroller implemented on the Digilent Cmod A7-35T module. It combines a MicroBlaze-V (RISC-V) processor with tightly-coupled memory, cached external SRAM, and QSPI flash, and provides memory-mapped peripherals: GPIO, timers and PWM, UART, I2C, SPI, and an ADC.

1.2 Features

Feature	Description
Core	32-bit RISC-V (RV32IMB), 100 MHz
Cache	16 KB instruction, 16 KB data
Tightly-coupled memory	128 KB: 32 KB bootloader, 32 KB ITCM, 64 KB DTCM
External RAM	512 KB SRAM, cached
Flash	4 MB QSPI
GPIO	24 pins (3 × 8-bit ports), 4 external interrupts
Timers / PWM	3 × 32-bit timers with interrupts, 3 PWM channels
UART	2 × 16550 UART
I2C	Master, 100 kHz
SPI	Master, ≈1.6 – 25 MHz, 2 slave selects
ADC	12-bit, 2 external inputs
I/O	48-pin DIP, 3.3 V
Boot	Standalone from flash, or JTAG from Vitis
Debug	JTAG (IEEE 1149.1)

1.3 On-Board Components

The MCU is implemented on the module's FPGA; the remaining devices provide storage, memory, communication, and power.

Component	Part	Role
FPGA	Xilinx Artix-7 (`xc7a35tcpg236-1`)	Implements the MCU: processor, memories, and peripherals
QSPI flash	Macronix MX25L3273F (Micron N25Q032A on older boards)	4 MB non-volatile storage for the hardware image and the application
SRAM	ISSI IS61WV5128BLL-10BLI	512 KB application memory
USB bridge	FTDI FT2232HQ	JTAG and UART over the Micro-USB connector
Regulator	Linear Technology LTC3569	Generates the on-board supply rails

© 2027 NCKU MPPA Course

1.4 System Block Diagram

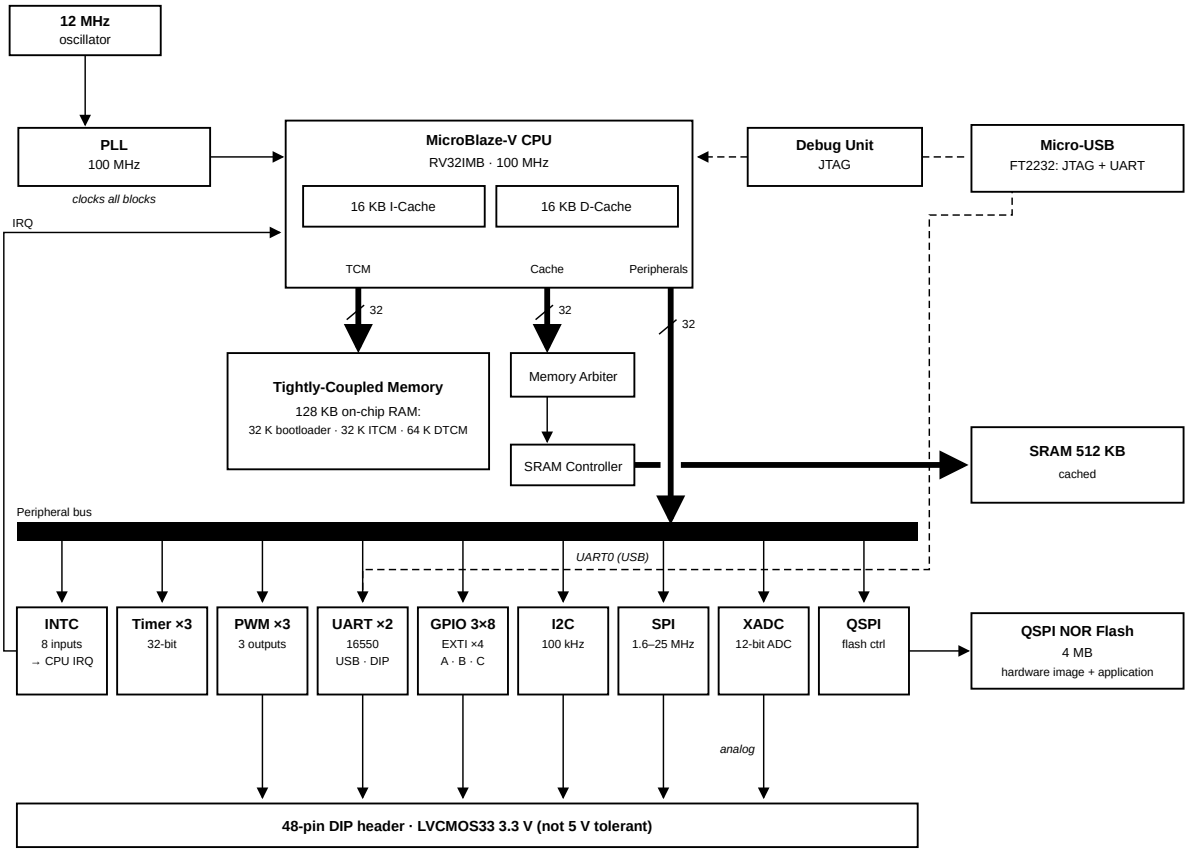


Figure 1: System Architecture

2 Pinout

2.1 DIP Connector Overview

The Cmod A7-35T has a 48-pin DIP connector. All digital I/O pins operate at LVCMOS33 (3.3 V logic level) with no series resistors.

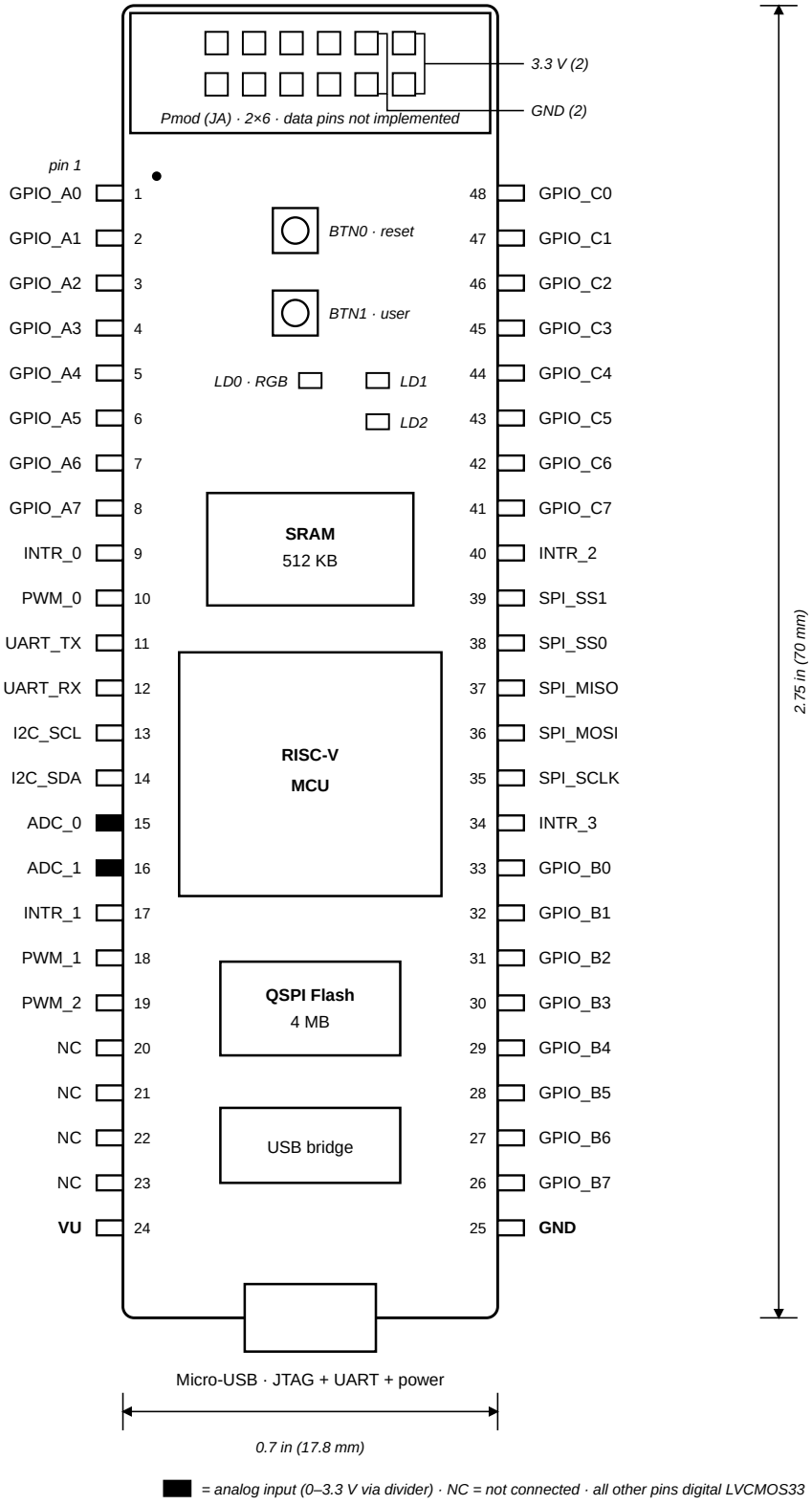


Figure 2: Cmod A7-35T DIP pinout (top view)

Peripheral registers and base addresses are listed in Section 4.6 and detailed in Section 6.

2.2 Pin Map — Left Side (Pin 1–24)

DIP Pin	Signal Name	FPGA Pin	Direction	Function
1	GPIO_A0	M3	I/O	General purpose GPIO, Group A bit 0
2	GPIO_A1	L3	I/O	General purpose GPIO, Group A bit 1
3	GPIO_A2	A16	I/O	General purpose GPIO, Group A bit 2
4	GPIO_A3	K3	I/O	General purpose GPIO, Group A bit 3
5	GPIO_A4	C15	I/O	General purpose GPIO, Group A bit 4
6	GPIO_A5	H1	I/O	General purpose GPIO, Group A bit 5
7	GPIO_A6	A15	I/O	General purpose GPIO, Group A bit 6
8	GPIO_A7	B15	I/O	General purpose GPIO, Group A bit 7
9	INTR_0	A14	Input	External interrupt input 0
10	PWM_0	J3	Output	PWM output channel 0
11	UART_TX	J1	Output	UART 1 transmit
12	UART_RX	K2	Input	UART 1 receive
13	I2C_SCL	L1	I/O (open-drain)	I2C clock (external 4.7 kΩ pull-up recommended)
14	I2C_SDA	L2	I/O (open-drain)	I2C data (external 4.7 kΩ pull-up recommended)
15	ADC_0	G3 / G2	Analog	Analog input 0 (XADC VAUX4)
16	ADC_1	H2 / J2	Analog	Analog input 1 (XADC VAUX12)
17	INTR_1	M1	Input	External interrupt input 1
18	PWM_1	N3	Output	PWM output channel 1
19	PWM_2	P3	Output	PWM output channel 2
20	NC	M2	—	Not connected (reserved for future use)
21	NC	N1	—	Not connected (reserved for future use)
22	NC	N2	—	Not connected (reserved for future use)
23	NC	P1	—	Not connected (reserved for future use)
24	**VU**	—	Power	Power input (ext) / Power output (USB)

2.3 Pin Map — Right Side (Pin 25–48)

DIP Pin	Signal Name	FPGA Pin	Direction	Function
25	**GND**	—	Power	Ground reference
26	GPIO_B7	R3	I/O	General purpose GPIO, Group B bit 7
27	GPIO_B6	T3	I/O	General purpose GPIO, Group B bit 6
28	GPIO_B5	R2	I/O	General purpose GPIO, Group B bit 5
29	GPIO_B4	T1	I/O	General purpose GPIO, Group B bit 4
30	GPIO_B3	T2	I/O	General purpose GPIO, Group B bit 3
31	GPIO_B2	U1	I/O	General purpose GPIO, Group B bit 2
32	GPIO_B1	W2	I/O	General purpose GPIO, Group B bit 1
33	GPIO_B0	V2	I/O	General purpose GPIO, Group B bit 0
34	INTR_3	W3	Input	External interrupt input 3
35	SPI_SCLK	V3	Output	SPI clock (software-settable, up to 25 MHz)
36	SPI_MOSI	W5	Output	SPI master-out slave-in
37	SPI_MISO	V4	Input	SPI master-in slave-out
38	SPI_SS0	U4	Output	SPI slave select 0 (active low)
39	SPI_SS1	V5	Output	SPI slave select 1 (active low)
40	INTR_2	W4	Input	External interrupt input 2
41	GPIO_C7	U5	I/O	General purpose GPIO, Group C bit 7
42	GPIO_C6	U2	I/O	General purpose GPIO, Group C bit 6
43	GPIO_C5	W6	I/O	General purpose GPIO, Group C bit 5
44	GPIO_C4	U3	I/O	General purpose GPIO, Group C bit 4

© 2027 NCKU MPPA Course

DIP Pin	Signal Name	FPGA Pin	Direction	Function
45	GPIO_C3	U7	I/O	General purpose GPIO, Group C bit 3
46	GPIO_C2	W7	I/O	General purpose GPIO, Group C bit 2
47	GPIO_C1	U8	I/O	General purpose GPIO, Group C bit 1
48	GPIO_C0	V8	I/O	General purpose GPIO, Group C bit 0

2.4 On-Board I/O (No DIP Pin Exposure)

These devices are mounted on the module and use no DIP pins; the names match the board silkscreen. The LEDs and the user button are GPIO ports; the reset button is wired to the system reset.

Device	Function
LD0	RGB LED (three GPIO outputs)
LD1, LD2	Status LEDs (GPIO outputs)
BTN1	User button (GPIO input)
BTN0	Reset button (not readable by software)

3 Electrical Characteristics and Power

3.1 Electrical Characteristics

Recommended Operating Conditions

Parameter	Value
I/O logic level	LVCMOS33 (3.3 V)
Digital input voltage	0 V to 3.3 V
Analog input voltage (pins 15, 16)	0 V to 3.3 V

Absolute Maximum Ratings

Stresses beyond these ratings may cause permanent damage to the device.

Parameter	Min	Max
Digital I/O pin voltage	–0.4 V	3.85 V

> **Not 5 V tolerant:** The DIP pins connect to the device with no series resistors. Drive them only with 3.3 V logic; use a level shifter or resistive divider for any 5 V signal.

> **Warning:** Disconnect any external supply from the VU pin before connecting USB. When a USB host is attached, VU is driven to the USB voltage (4.5–5.5 V), which can damage a source feeding VU — a battery especially, since it cannot tolerate reverse current.

3.2 Power Architecture

The Cmod A7-35T uses a Linear Technology LTC3569 triple-output buck regulator to generate all onboard supply voltages from a single input rail called VU.

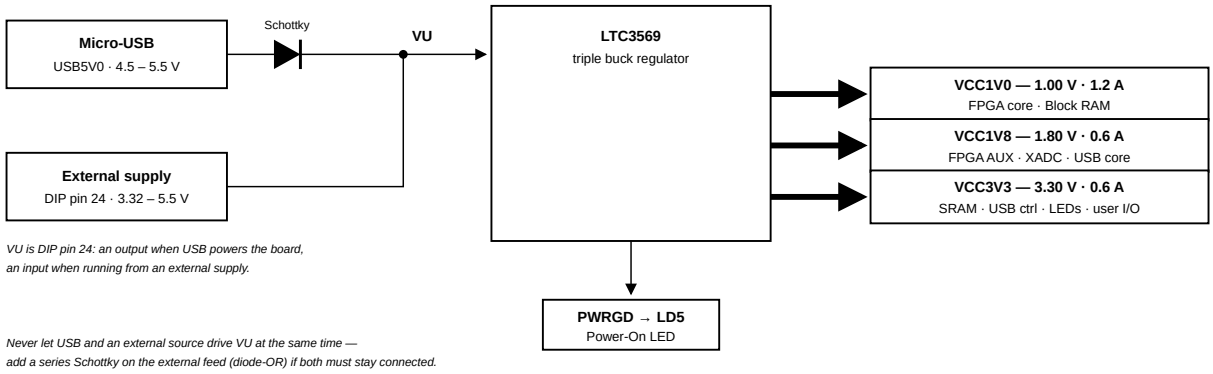


Figure 3: Cmod A7 power supply tree

3.3 Power Input Options

The board can be powered from either of the following sources:

Source	Connector	Schematic Net	Min Voltage	Recommended	Max Voltage
USB	Micro-USB	USB5V0	4.5 V	5.0 V	5.5 V
External Supply	DIP pin 24	VU	3.32 V*	5.0 V	5.5 V

> **Note:** * The minimum external voltage on VU depends on the current drawn from the VCC3V3 rail via the Pmod header:
>
> - 0 mA drawn: minimum 3.32 V
> - 100 mA drawn: minimum 3.38 V
> - 250 mA drawn: minimum 3.48 V

The GND reference is DIP pin 25.

3.4 Output Power Rails

All three rails are generated by the LTC3569 from the VU input.

Rail	Typical Voltage	Min / Max Voltage	Max Current	Powered Devices
VCC1V0	1.00 V	0.95 V / 1.05 V	1.2 A	FPGA Core, Block RAM
VCC1V8	1.80 V	1.71 V / 1.89 V	0.6 A	FPGA AUX, ADC, USB Core
VCC3V3	3.30 V	3.135 V / 3.465 V	0.6 A	SRAM, USB Controller, Pmod, LEDs, Buttons, FPGA User I/O

3.4.1 Powering External Circuits

The Pmod connector is the board's 3.3 V supply point for user circuits: its two VCC pins provide regulated 3.3 V and its two GND pins the return. The DIP header exposes only VU and GND, so an external 3.3 V module is powered from the Pmod and its signals are routed through the DIP GPIO pins. The Pmod data pins are not implemented in this device.

These pins share the 0.6 A VCC3V3 rail with the on-board devices; reserve them for low-current loads such as sensor modules, and give motors their own supply. When the board runs from an external supply, the minimum VU voltage rises with the current drawn here (see the note in Section 3.3).

3.5 VU Pin Behavior

Condition	VU Pin Function
USB connected	Output. Driven from USB 5 V through a Schottky diode; approximately 5 V after the diode drop
USB not connected	Input. Accepts an external supply of 3.32–5.5 V

3.6 Dual Power Source (USB + External)

An on-board Schottky diode isolates the USB port from the VU node; the VU pin itself has no diode. The behavior of each configuration is as follows:

Configuration	Behavior
USB only	Normal operation
External supply only	Normal operation
Both connected, one source active	Normal operation. The inactive source supplies no current.
Both sources active	Not permitted. The two supplies drive the VU node against each other.
Both sources active, with a series Schottky on the external feed	Permitted. The higher voltage supplies the board; the other is blocked (diode-OR).

> **Note:** When both sources must operate at the same time (for example, USB for the console while powering from an external supply), add a Schottky diode in series with the external supply at the VU pin.

4 System Architecture

4.1 MicroBlaze RISC-V Core

Item	Value
Architecture	32-bit RISC-V, RV32IMB (hardware multiply/divide, bit-manipulation)
Clock	100 MHz
Cache	16 KB instruction + 16 KB data, write-through, 32-byte lines
Cached region	External SRAM only
Debug	JTAG: breakpoints, register and memory access

Description: The processor executes firmware from SRAM. All memory and peripherals are memory-mapped into one address space. The external SRAM is cached; the tightly-coupled memory and peripheral registers are not, so peripheral reads and writes always take effect immediately.

4.2 Clock

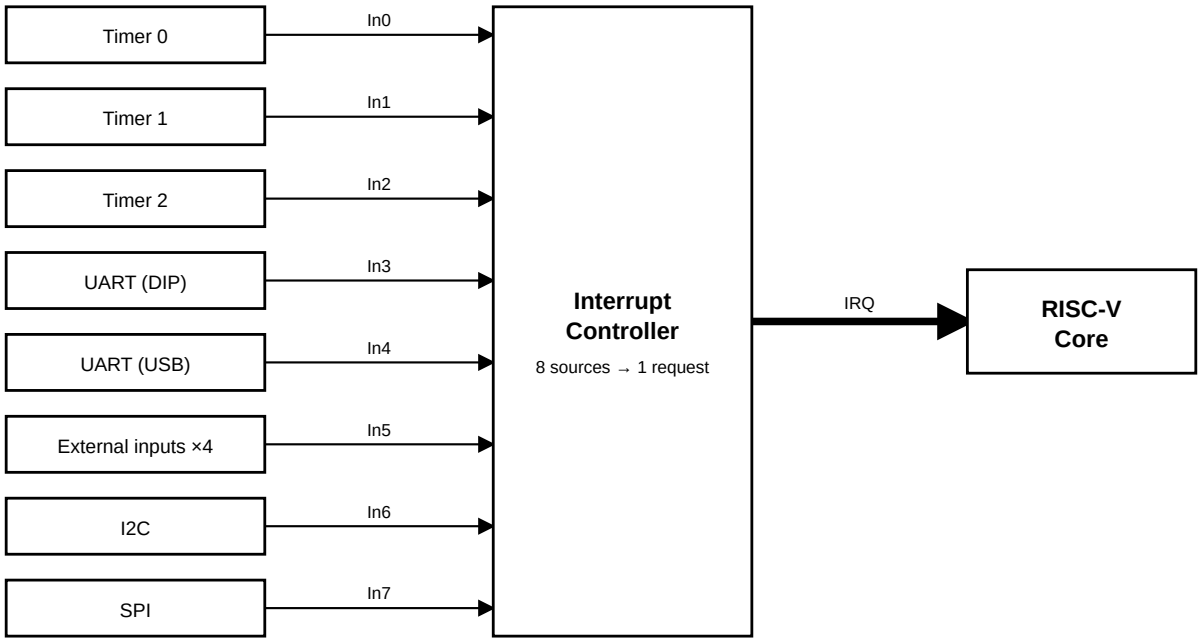
Description: The system clock is 100 MHz, generated from the 12 MHz on-board oscillator by an on-chip PLL. The processor, the buses, and all peripheral registers run at this frequency; only the SPI master's serial clock is generated separately (see the SPI Master section).

4.3 Reset

Description: A reset restarts the processor from the bootloader, which reloads the application from flash (see Section 8), and returns peripheral registers to their reset values. There are three reset sources: power-on, the on-board reset button BTN0, and a system reset issued by the debugger.

4.4 Interrupt Controller

The interrupt controller combines the peripheral interrupt sources into a single request line to the processor. When an interrupt is taken, the processor reads the controller to determine which source is active.



A source reaches the core only when the source, the controller, and the core all enable interrupts.

Figure 4: The interrupt controller collects eight sources into one request to the core

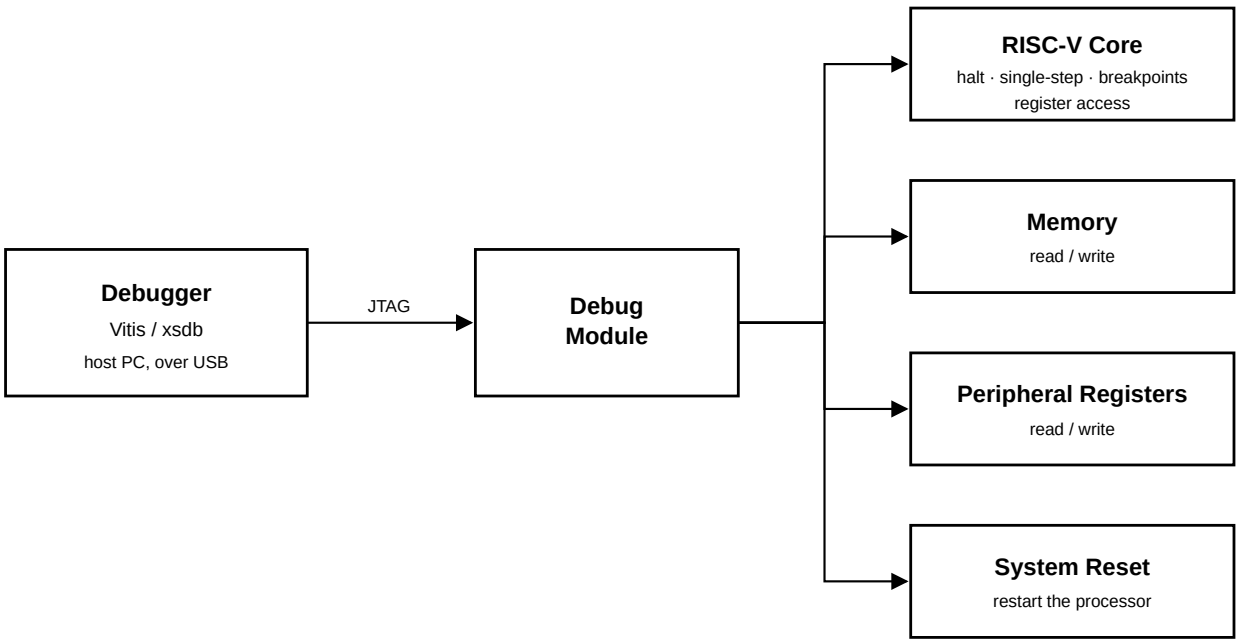
Interrupt Mapping:

Channel	Source	Description
In0	`timer_0`	System timer 0 interrupt
In1	`timer_1`	System timer 1 interrupt
In2	`timer_2`	System timer 2 interrupt
In3	`uart_1`	External UART interrupt
In4	`uart_USB`	USB UART interrupt
In5	`INT_0_3`	External GPIO interrupt (4-bit)
In6	`i2c_0`	I2C controller interrupt
In7	`spi_0`	External SPI master interrupt

> **Note:** The controller's registers and programming interface are described with the other peripheral registers in the Peripherals chapter.

4.5 Debug Module

Description: The debug module connects a host debugger to the processor over the JTAG port. Vitis and `xsdb` sessions use it to halt and single-step the core, set breakpoints, read and write the processor registers, memory, and peripheral registers, and reset the system. Because every peripheral is memory-mapped, the debugger can operate a device directly — for example writing a GPIO register to drive an output — with no firmware running.



Debug access is implemented in hardware; the core can be halted even when the application is unresponsive.

Figure 5: The debug module gives the host debugger access to the core, memory, peripheral registers, and system reset over JTAG

4.6 Complete Address Map

Peripheral addresses follow a class-based convention: `0x40[C]x_xxxx`, where `C` is the peripheral class (1 MB per class, 64 KB per instance). The device type is identifiable directly from the address: class 0 = GPIO, 1 = Timer, 2 = PWM, 3 = UART, 4 = INTC, 5 = QSPI control, 6 = XADC, 7 = I2C, 8 = SPI, 9 = SPI clock control.

Base Address	Range	Peripheral	Category
`0x0000_0000`	128K	Tightly-coupled memory (bootloader, ITCM, DTCM)	Memory
`0x4000_0000`	64K	On-board LEDs	GPIO
`0x4001_0000`	64K	User button (BTN1)	GPIO
`0x4002_0000`	64K	RGB LED	GPIO
`0x4003_0000`	64K	GPIO group A	GPIO
`0x4004_0000`	64K	GPIO group B	GPIO
`0x4005_0000`	64K	GPIO group C	GPIO
`0x4007_0000`	64K	External interrupt inputs	Interrupt
`0x4010_0000`	64K	Timer 0	Timer
`0x4011_0000`	64K	Timer 1	Timer
`0x4012_0000`	64K	Timer 2	Timer
`0x4020_0000`	64K	PWM 0	PWM
`0x4021_0000`	64K	PWM 1	PWM
`0x4022_0000`	64K	PWM 2	PWM
`0x4030_0000`	64K	USB UART	Communication
`0x4031_0000`	64K	External UART (DIP 11/12)	Communication
`0x4040_0000`	64K	Interrupt controller	System
`0x4050_0000`	64K	QSPI flash controller	Memory
`0x4060_0000`	64K	ADC (XADC)	ADC
`0x4070_0000`	64K	I2C controller (DIP 13/14)	Communication
`0x4080_0000`	64K	SPI master (DIP 35–39)	Communication
`0x4090_0000`	64K	SPI clock control	Communication
`0x6000_0000`	512K	External SRAM (cached)	Memory

© 2027 NCKU MPPA Course

5 Memory Hierarchy

5.1 Memory Hierarchy Overview

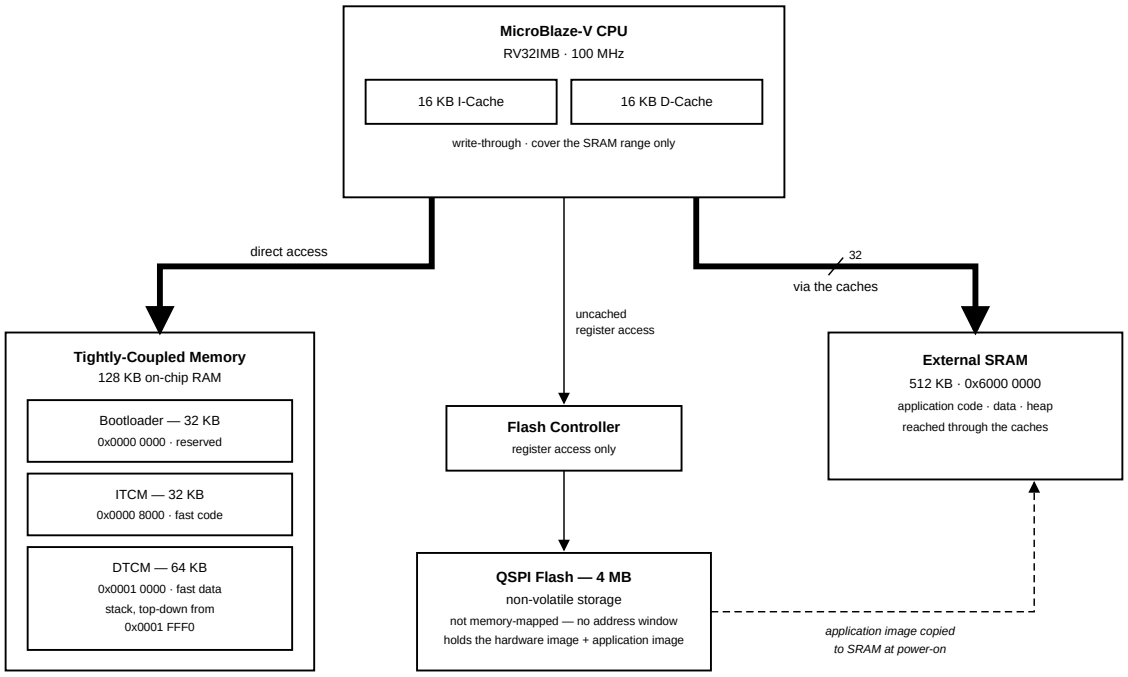


Figure 6: Memory Hierarchy

Memory	Size	Role
Tightly-coupled memory	128 KB	Interrupt handlers, stack, frequently accessed data
I-Cache + D-Cache	16 KB + 16 KB	Transparent acceleration of SRAM
External SRAM	512 KB	Application code, data, heap
QSPI flash	4 MB	Non-volatile: application + hardware image

5.2 Memory Map

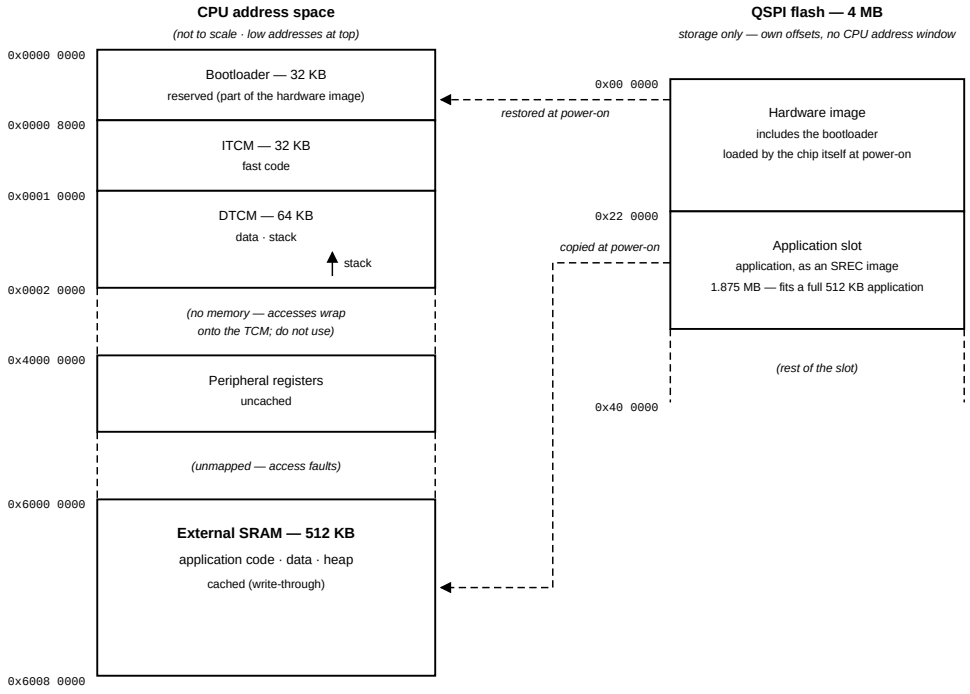


Figure 7: Memory Map

Address range	Size	Region	Access
`0x0000_0000` – `0x0000_7FFF`	32 KB	Bootloader (reserved)	uncached
`0x0000_8000` – `0x0000_FFFF`	32 KB	ITCM — application code	uncached
`0x0001_0000` – `0x0001_FFFF`	64 KB	DTCM — data; stack grows down from `0x0001_FFF0`	uncached
`0x4000_0000` – `0x40FF_FFFF`	—	Peripheral registers	uncached
`0x6000_0000` – `0x6007_FFFF`	512 KB	External SRAM — application code / data / heap	cached

Note: Unused addresses in the `0x4000_0000` – `0x7FFF_FFFF` range return a bus error. Elsewhere the tightly-coupled memory is not decoded fully: an access to an address outside the listed regions, such as `0x0002_0000`, wraps around onto the 128 KB tightly-coupled memory. A stray pointer there therefore fails silently and can overwrite the bootloader, ITCM code, or the stack.

5.3 Tightly-Coupled Memory (ITCM / DTCM / Bootloader)

Item	Value
Address	`0x0000_0000` – `0x0001_FFFF` (128 KB)
Ports	Dual-port: simultaneous instruction and data access
Layout	32 KB bootloader + 32 KB ITCM + 64 KB DTCM

Description: The tightly-coupled memory (TCM) is the on-chip RAM, accessed outside the cache path. It is a dual-port memory, so an instruction fetch and a data access can proceed at the same time. It serves the same role as ITCM/DTCM on other MCU cores (e.g. Cortex-M7), and is divided into the bootloader region, the ITCM for interrupt handlers and timing-critical code, and the DTCM for the stack and fast data.

Note: The bootloader region has no software load path. Its contents are carried inside the hardware image and restored from flash at every power-on, before the processor starts, so application updates never touch it (see Section 8). The region is ordinary RAM, however: software must never write below `0x0000_8000`. A stray write there persists until the next power cycle, and a reset then restarts the damaged bootloader.

5.4 External SRAM

Item	Value
Base Address	`0x6000_0000`
Size	512 KB — `0x6000_0000` – `0x6007_FFFF`
Memory	On-board asynchronous SRAM
Cache	Covered by the I-Cache and D-Cache

Description: This memory controller interfaces to the on-board 512 KB SRAM, the main application memory: standalone boot copies the program here and executes it.

5.5 QSPI Flash

Item	Value
Base Address	`0x4050_0000`
Flash Device	4 MB QSPI NOR flash: Macronix MX25L3273F, or Micron N25Q032A on older boards
Mode	CPU-programmable register mode — flash contents are not memory-mapped
FIFO	256 B TX/RX — one full flash page (256 B) per transfer

Description: This controller manages the on-board Quad-SPI NOR Flash. The CPU drives the flash through the controller, so firmware can read, erase, and program it at runtime.

Flash contents are not memory-mapped: there is no XIP window, and code cannot execute from flash directly. The standalone-boot design instead copies the application from flash into SRAM at power-on (see Section 8).

> **Design note:** A read-only memory-mapped XIP window and CPU-programmable register mode are mutually exclusive in this controller. This MCU uses register mode: keeping the flash CPU-programmable at runtime is preferred over execute-in-place, and execution is served by the 512 KB SRAM and the I-cache instead.

5.6 Caches

Item	Value
Instruction cache	16 KB, 32-byte lines
Data cache	16 KB, 32-byte lines
Write policy	write-through — every store is forwarded to SRAM
Cached range	exactly the SRAM: `0x6000_0000` – `0x6007_FFFF`

Only the SRAM range is cached. The tightly-coupled memories are already fast enough, and peripheral registers must observe every access, so neither is ever cached. The caches are always on for the cached range and require no software management.

The write-through policy has two practical consequences:

- Reads benefit fully from the cache: a hit has the same latency as a TCM access.
- Writes always incur the SRAM latency, even on a cache hit. Frequently written data (counters, buffers filled in interrupt handlers) should therefore be placed in the DTCM.

5.7 Measured Access Latencies

The figures below were measured on the board at 100 MHz by the `mem_bench` example: optimized (`-O2`) read and write loops timed with a hardware cycle counter, with the loop's own overhead (≈7 cycles per word) subtracted; the cache-miss figure was taken immediately after invalidating the data cache. Each value is the latency of a single access.

Access	Cycles	Notes
TCM read	1	single-cycle; identical every time
SRAM read, cache **hit**	1	a hit matches the TCM
SRAM read, cache **miss**	≈151	fetches one 32-byte line (≈19 cycles per word)
SRAM write (write-through)	≈16	per store; the store waits for SRAM

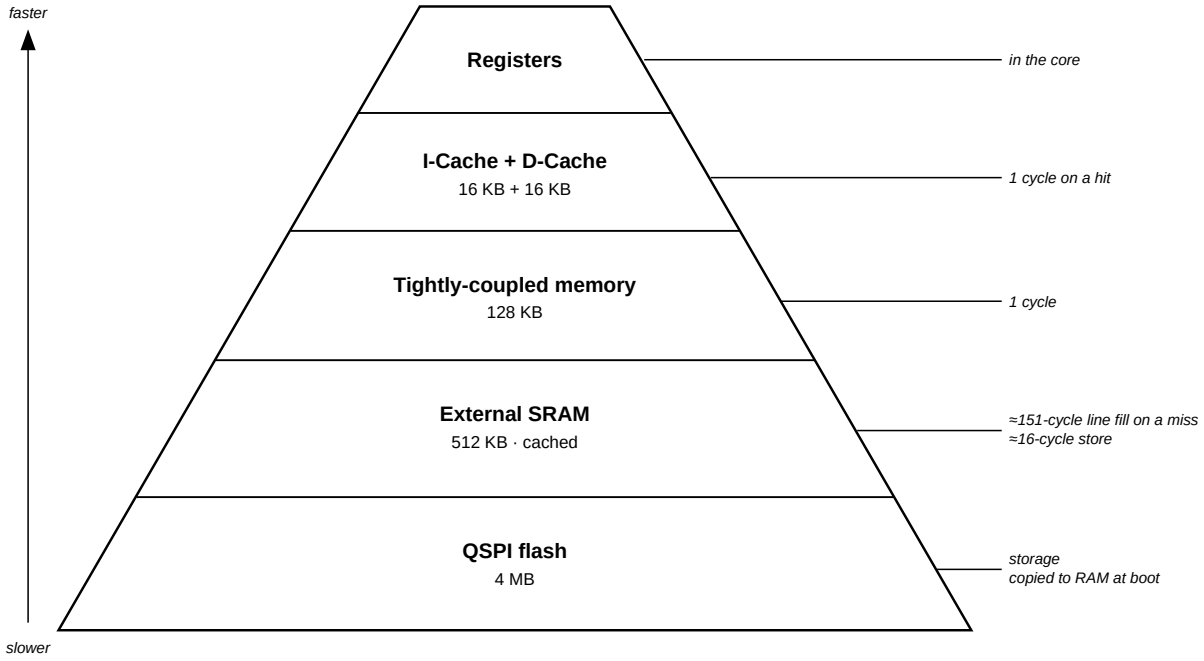


Figure 8: Memory hierarchy: faster and smaller toward the top, larger and slower toward flash

5.8 Memory Placement Guidance

The linker script `mcu/cmod_mcu.ld` assigns every section to a memory, and the device header `mcu/cmod_mcu.h` provides the `ITCM_FUNC` and `DTCM_DATA` attributes that move individual functions and variables into the tightly-coupled memories.

What	Put it in	How
Interrupt handlers, timing-critical loops	ITCM	`ITCM_FUNC`
Frequently-written data, buffers, the `XIntc` driver instance	DTCM	`DTCM_DATA`
Stack	DTCM	automatic: all DTCM after the `DTCM_DATA` variables
Everything else — code, constants, data	SRAM	default
Heap	SRAM	automatic: all SRAM after the program
Firmware image	QSPI flash	see the Standalone Boot Mode guide

As a general rule, place code and data in SRAM by default and move them to the TCMs when predictable timing is required. In the tightly-coupled memories the worst-case latency equals the best-case latency.

Code in the ITCM keeps that property only while everything it calls is in the ITCM as well. At `-O0` the board-support package's `Xil_In32()` and `Xil_Out32()` are real function calls into SRAM, so ITCM code accesses registers with `REG32()` from `cmod_mcu.h`.

5.8.1 Interrupt Latency

The trap entry and the interrupt controller's dispatch code always run from the ITCM, and the tables they read live in the DTCM. What remains to place is the handler itself and the `XIntc` driver instance:

Handler placement	Warm cache	Cold cache
`ITCM_FUNC` handler, `XIntc` instance in DTCM	187	187
`ITCM_FUNC` handler, `XIntc` instance in SRAM	187	227
Handler in SRAM	187	591
`ITCM_FUNC` handler calling `Xil_In32()` at `-O0`	199	348

Cycles at 100 MHz from a timer event to the handler's first register read, measured on the board: `timer_2` raises the interrupt as it reloads, and the handler's first action reads the counter. "Cold" means both caches were invalidated before every interrupt, the worst case. Each value is the median of 256 interrupts; no run varied by more than 2 cycles. With the handler in the ITCM and the instance in the DTCM, an interrupt reaches the handler in 1.87 µs whatever the cache state.

6 Peripherals

All peripheral registers are memory-mapped; base addresses are listed in Section 4.6. Pin locations for every signal are listed in the pin maps of Section 2. The QSPI flash controller is described with the memory system in Section 5.5.

6.1 Register Access Conventions

Every peripheral register in this device is a 32-bit word at a word-aligned offset from its instance base address (the Complete Address Map lists all base addresses). Registers must be accessed with full-word reads and writes: in C through `Xil_In32`/`Xil_Out32` or a `volatile` pointer, in assembly with `lw`/`sw`. Byte and halfword accesses are not supported by the peripheral bus. The device header `mcu/cmod_mcu.h` names the base addresses, register offsets, and bits of this chapter for C and assembly, and provides `REG32()`, a `volatile` word access.

The register tables in this chapter use the following access codes:

Code	Meaning
RW	Read/write
RO	Read-only; writes are ignored
WO	Write-only; reads return undefined data
R/W1C	Readable; writing 1 to a set bit clears it
R/TOW	Readable; writing 1 toggles the bit (see the GPIO `ISR`)
RSE	Read side effect: the read itself changes state (for example, a UART receive-buffer read pops the FIFO)

Reserved bits read as undefined values and must be written as 0. Reading a register with a read side effect purely for inspection (for example from a debugger memory view) consumes data or clears status.

Reset values apply after power-on or reset.

> **Note:** Loading a new program over JTAG does not reset the peripherals: registers keep whatever state the previous program left, including armed interrupts and half-finished bus transactions. Programs must not assume reset values at entry; initialize every peripheral before use.

The following two fragments are equivalent and turn the RGB LED green — the C form is the course template's idiom, the assembly form is the `asm_template` idiom. Both include `cmod_mcu.h`, which defines `RGB_BASE` (`0x4002_0000`), `GPIO_DATA` (offset `0x0`), and `RGB_GREEN` (bit 1):

```c
REG32(RGB_BASE + GPIO_DATA) = RGB_GREEN;  /* same as Xil_Out32(0x40020000, 0x2) */
```

```asm
li   t0, RGB_BASE          # RGB LED port base, 0x40020000
li   t1, RGB_GREEN         # bit 1 = green
sw   t1, GPIO_DATA(t0)     # DATA register: drive the pins
```

6.2 GPIO

The general-purpose GPIO ports carry polled digital I/O. Each port is memory-mapped with per-pin direction control: the `TRI` register sets each pin's direction and the `DATA` register reads or writes the pins. These ports have no interrupt hardware and cannot raise a processor interrupt; to watch a signal by interrupt, use the External Interrupt Inputs described in the next section.

6.2.1 DIP Connector GPIO (3 Groups × 8-bit)

Instance	Base Address	Width	DIP Pins	Description
`gpio_A_0_7`	`0x4003_0000`	8	Pin 1–8 (bit 0 = pin 1)	GPIO Group A, bidirectional I/O
`gpio_B_0_7`	`0x4004_0000`	8	Pin 26–33 (bit 0 = pin 33)	GPIO Group B, bidirectional I/O
`gpio_C_0_7`	`0x4005_0000`	8	Pin 41–48 (bit 0 = pin 48)	GPIO Group C, bidirectional I/O

Description: Each group is an 8-bit bidirectional port on eight adjacent pins, so a whole port is read or written as one byte; every bit can be an input or an output independently. Bit 0 of each group is the pin nearest the Pmod connector.

6.2.2 GPIO Register Map

Every general-purpose port has the same two registers. Offsets are relative to the port base address. Only the low WIDTH bits are implemented (8 for groups A–C; the on-board ports below are narrower); unimplemented bits read as 0.

Offset	Name	Access	Reset	Description
`0x000`	`DATA`	RW	`0x0`	Port data
`0x004`	`TRI`	RW	all 1	Per-pin direction (1 = input, 0 = output)

6.2.2.1 DATA — Port Data Register (Offset 0x000)

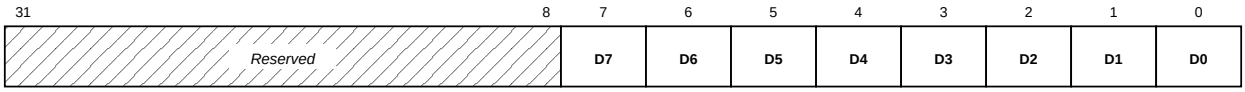


Figure 9: GPIO DATA layout: one bit per pin in the low bits; only the low W bits exist (W = 8 for groups A–C, fewer for the on-board ports)

Bits	Name	Access	Description
31:W	—	—	Reserved (W = port width). Read as 0.
W-1:0	`DATA`	RW	One bit per pin. A read returns the pin level for input pins and the last written value for output pins. A write drives pins configured as outputs; writes to input pins have no effect.

6.2.2.2 TRI — Port Direction Register (Offset 0x004)

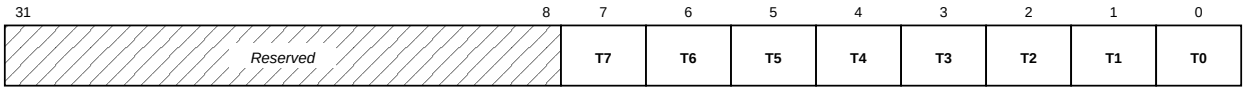


Figure 10: GPIO TRI layout: one direction bit per pin (1 = input, 0 = output), same low-bit layout as DATA

Bits	Name	Access	Description
31:W	—	—	Reserved. Read as 0.
W-1:0	`TRI`	RW	Per-pin direction: 1 = input, 0 = output. All pins are inputs after reset; software must clear the relevant bits before driving a pin. The fixed-direction ports (LEDs, RGB, button) do not implement this register (see On-Board GPIO).

> **Note:** Setting or clearing individual pins is a read-modify-write of `DATA` (read the port, change the bits, write it back); it is not atomic, so if an interrupt handler also writes the same port, keep all writes to that port in one context.

Usage recipe (board-verified). `GPIO` is the port base address (`0x4003_0000`, `0x4004_0000` or `0x4005_0000`); bit `n` of the registers is pin `n` of the group, whose DIP pin is listed in the pin maps:

```
Xil_Out32(GPIO + 0x4, 0x00);             /* TRI: all 8 pins outputs           */
Xil_Out32(GPIO + 0x0, 0xA5);             /* DATA: drive the byte 1010 0101    */

Xil_Out32(GPIO + 0x4, 0xFF);             /* TRI: all 8 pins inputs            */
u32 pins = Xil_In32(GPIO + 0x0) & 0xFF;  /* DATA: read the 8 pin levels       */

Xil_Out32(GPIO + 0x4, 0xF0);             /* TRI: bits 7:4 inputs, 3:0 outputs */
u32 v = Xil_In32(GPIO + 0x0);            /* read-modify-write one output:     */
Xil_Out32(GPIO + 0x0, v | (1u << 2));    /*   set bit 2, keep the others      */
```

6.2.3 On-Board GPIO

Instance	Base Address	Width	Direction	Connection	Description
`board_led_2bits`	`0x4000_0000`	2	Output	LD1, LD2	On-board LEDs × 2
`board_button`	`0x4001_0000`	1	Input	BTN1	On-board user button
`board_rgb`	`0x4002_0000`	3	Output	LD0	On-board RGB LED (bit 0 = Blue, bit 1 = Green, bit 2 = Red)

The on-board ports have a fixed direction: the LED and RGB ports are outputs and the button is an input. Only their `DATA` register is functional; `TRI` is not implemented on these ports: writes to it are ignored and it always reads `0xFFFF_FFFF`, whatever the port's direction. The `DATA` and `TRI` layout of each port follows.

On-board LEDs (`board_led_2bits`) — 2-bit output, base `0x4000_0000`.

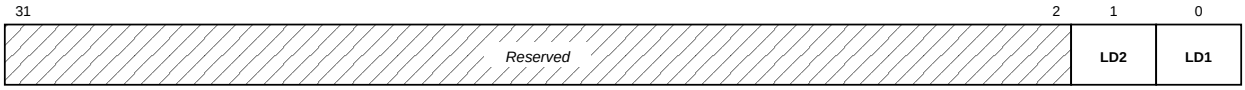


Figure 11: LED DATA register (offset 0x000): bit 0 drives LD1, bit 1 drives LD2

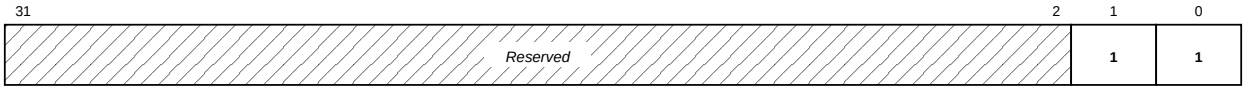


Figure 12: LED TRI register (offset 0x004): not implemented; writes are ignored and every bit reads 1

User button (`board_button`) — 1-bit input, base `0x4001_0000`.

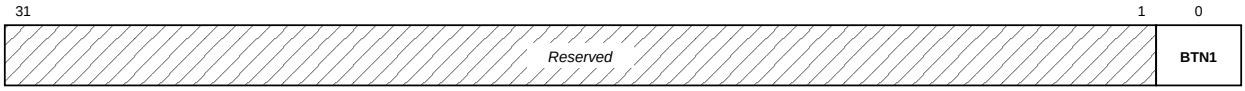


Figure 13: Button DATA register (offset 0x000): bit 0 reads BTN1

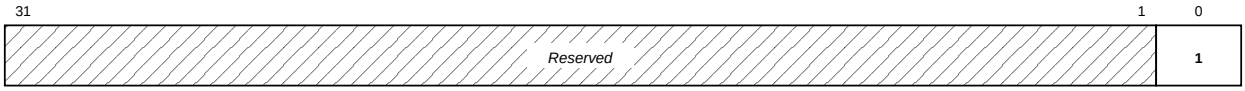


Figure 14: Button TRI register (offset 0x004): not implemented; writes are ignored and every bit reads 1

RGB LED (`board_rgb`, LD0) — 3-bit output, base `0x4002_0000`.

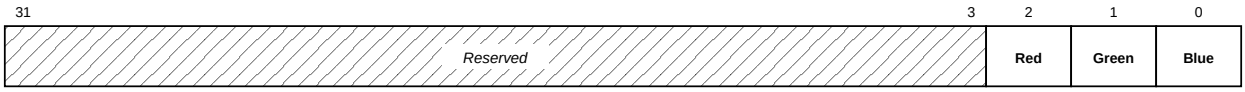


Figure 15: RGB DATA register (offset 0x000): bits 2:0 drive Red, Green, Blue

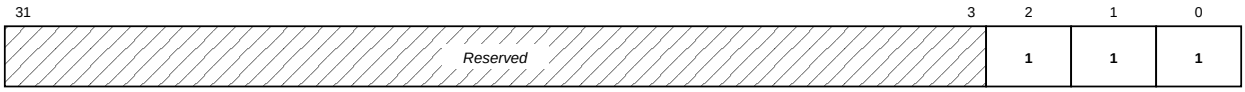


Figure 16: RGB TRI register (offset 0x004): not implemented; writes are ignored and every bit reads 1

> **Reference:** AMD AXI GPIO LogiCORE IP Product Guide (PG144).

6.3 External Interrupt Inputs

`INT_0_3` is a dedicated interrupt port: it is the only GPIO instance built with interrupt hardware. Its four pins are input-only, and a change on any of them (either edge) sets the interrupt status and raises interrupt controller input 5. The four pins share one interrupt line, so the handler reads `DATA` to see which pin changed. The pins can also be polled through `DATA` without using interrupts.

Item	Value
Base Address	`0x4007_0000`
Width	4-bit, input-only
DIP Pins	Pin 9 (INTR_0), Pin 17 (INTR_1), Pin 40 (INTR_2), Pin 34 (INTR_3)
Interrupt	Interrupt controller input 5

6.3.1 Interrupt Register Map

Offsets are relative to the base address. `DATA` reads the four pins; the three interrupt registers gate and report the shared interrupt.

Offset	Name	Access	Reset	Description
`0x000`	`DATA`	RO	`0x0`	Pin levels (bits 3:0)
`0x004`	`TRI`	RO	all 1	Direction, fixed to input
`0x11C`	`GIER`	RW	`0x0`	Global interrupt enable
`0x120`	`ISR`	R/TOW	`0x0`	Interrupt status
`0x128`	`IER`	RW	`0x0`	Interrupt enable

6.3.1.1 DATA — Pin Data Register (Offset 0x000)

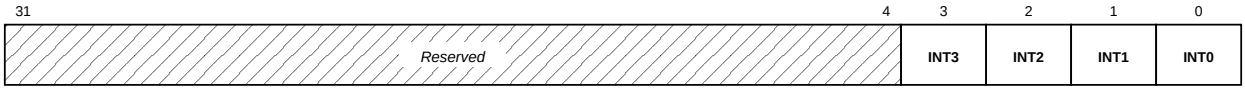


Figure 17: INT_0_3 DATA layout: the four interrupt input pins occupy bits 3:0

Bits	Name	Access	Description
31:4	—	—	Reserved.
3:0	`INT3–INT0`	RO	Live level of the four interrupt pins. The handler reads this to see which pin changed.

6.3.1.2 GIER — Global Interrupt Enable Register (Offset 0x11C)

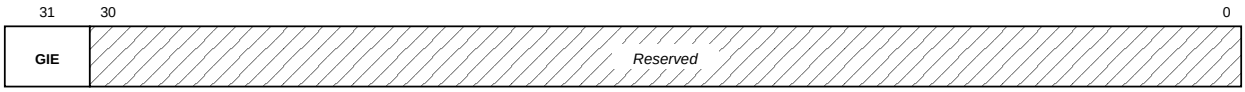


Figure 18: INT_0_3 GIER layout: the global interrupt enable is bit 31

Bits	Name	Access	Description
31	`GIE`	RW	Master gate for the port's interrupt output. Write `0x8000_0000` to enable.
30:0	—	—	Reserved.

6.3.1.3 ISR and IER — Interrupt Status and Enable (Offsets 0x120 / 0x128)

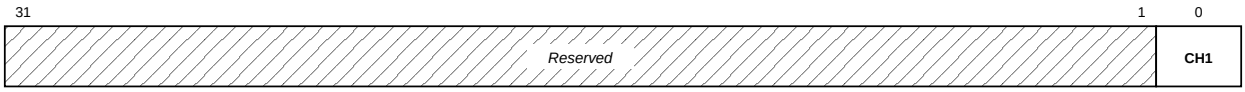


Figure 19: INT_0_3 ISR/IER layout: the single channel interrupt bit is bit 0

Register	Bit 0 — `CH1`
`ISR` (status, `0x120`, R/TOW)	Set by hardware when any of the four pins changes. Toggle-on-write: to clear, read the register and write the read value back.
`IER` (enable, `0x128`, RW)	Enables the port interrupt. Any of the four pins raises the same interrupt; the handler reads `DATA` to determine which line changed.

The port has a single interrupt channel (`CH1`, bit 0) shared by all four pins; `ISR` reports it and `IER` enables it. Bits 31:1 are reserved in both.

> **Note:** `ISR` toggles on write: writing a hard-coded 1 while the bit is clear sets a phantom pending interrupt, so always write back exactly the value just read. The interrupt reaches the processor through the interrupt controller as input 5.

Usage recipe (board-verified). Polled use needs no setup:

```
u32 pins = Xil_In32(EXTI + 0x000) & 0xF;   /* DATA: INTR_3..INTR_0 levels */
```

For interrupt use, the handler acknowledges the port and reads the pins, and the port's two enables are opened after the handler is installed on input 5 of the interrupt controller (the installation itself is shown in the Interrupt Controller section):

```
void exti_isr(void *ref)
{
    u32 st = Xil_In32(EXTI + 0x120);      /* ISR: read the status             */
    Xil_Out32(EXTI + 0x120, st);          /* write it back unchanged: clears  */
    u32 pins = Xil_In32(EXTI + 0x000);    /* DATA: compare with the previous  */
    /* ... */                             /* levels to find the pin(s) that   */
}                                         /* changed                          */

Xil_Out32(EXTI + 0x120, Xil_In32(EXTI + 0x120));  /* ISR: drop stale status   */
Xil_Out32(EXTI + 0x128, 0x1);                     /* IER: channel 1           */
Xil_Out32(EXTI + 0x11C, 0x80000000);              /* GIER: global enable      */
```

> **Note:** The four pins share the interrupt, so an unconnected interrupt pin that floats can raise it continuously and starve the program. Give every interrupt pin a defined level (drive it, or add a pull-up or pull-down resistor) before enabling the interrupt. The toggle-on-write behavior also has a use: writing `1` to `ISR` while it is clear raises the interrupt from software, which tests the complete handler chain without any wiring.

> **Reference:** The external interrupt inputs are the AMD AXI GPIO core in interrupt mode; see its LogiCORE IP Product Guide (PG144).

6.4 Timers and PWM

The device provides six 32-bit timer instances: three serve as general-purpose timers with interrupts, and three have their outputs routed to pins as PWM channels.

6.4.1 System Timers

Instance	Base Address	Interrupt	Description
timer_0	0x4010_0000	INTC In0	General-purpose system timer
timer_1	0x4011_0000	INTC In1	General-purpose timer
timer_2	0x4012_0000	INTC In2	General-purpose timer

6.4.2 PWM Outputs

Instance	Base Address	Output Pin	DIP Pin	Description
PWM_0	0x4020_0000	J3	Pin 10	PWM Channel 0
PWM_1	0x4021_0000	N3	Pin 18	PWM Channel 1
PWM_2	0x4022_0000	P3	Pin 19	PWM Channel 2

Description: These timer instances are configured in PWM mode to generate square-wave outputs, suitable for applications such as LED dimming, motor speed control, and buzzer tone generation. Frequency and duty cycle are configured through the Timer Load Registers and the PWM enable bit.

6.4.3 Timer/PWM Register Map

Each of the six instances (`timer_0`–`timer_2`, `PWM_0`–`PWM_2`) is one dual 32-bit counter block: counter 0 occupies offsets `0x00`–`0x08` and counter 1 the same layout at `+0x10`. The general-purpose timers normally use counter 0 alone; PWM mode uses both counters together (period and high time). All counters run at the 100 MHz bus clock, so one count equals 10 ns.

Offset	Name	Access	Reset	Description
0x00	TCSR0	RW	0x0	Counter 0 control and status
0x04	TLR0	RW	0x0	Counter 0 load value
0x08	TCR0	RO	0x0	Counter 0 current count
0x10	TCSR1	RW	0x0	Counter 1 control and status
0x14	TLR1	RW	0x0	Counter 1 load value
0x18	TCR1	RO	0x0	Counter 1 current count

6.4.3.1 TCSR — Control and Status Register (Offsets 0x00 and 0x10)

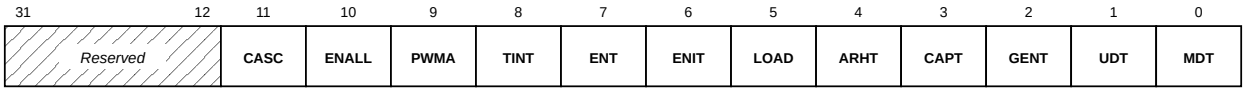


Figure 20: Timer TCSR bit layout: the twelve control and status fields occupy bits 11:0; bits 31:12 are reserved

Bits	Name	Access	Description
31:12	—	—	Reserved. Write 0.
11	CASC	RW	Cascade both counters into one 64-bit counter.
10	ENALL	RW	Writing 1 enables both counters simultaneously.
9	PWMA	RW	PWM mode (set in both TCSRs, together with GENT).
8	TINT	R/W1C	Interrupt status: reads 1 after the counter reaches its terminal value. Write the register back with this bit set to clear it.
7	ENT	RW	Enable the counter (starts counting).
6	ENIT	RW	Enable the interrupt output (general-purpose timers only).
5	LOAD	RW	While 1, forces TCR = TLR. Write a pulse (set, then clear) to load; the counter does not run while set.
4	ARHT	RW	Auto-reload: reload TLR and continue at the terminal value (1) or stop and hold (0).
3	CAPT	RW	External capture mode. Not wired in this device; write 0.
2	GENT	RW	Enable the generate output (required for PWM).
1	UDT	RW	Count direction: 0 = up, 1 = down.
0	MDT	RW	Mode: 0 = generate, 1 = capture. Write 0.

6.4.3.2 TLR — Load Register (Offsets 0x04 and 0x14)

The 32-bit value loaded into the counter by `LOAD` or by auto-reload. In PWM mode `TLR0` sets the period and `TLR1` the high time; the hardware adds a fixed two-cycle overhead to each interval.

6.4.3.3 TCR — Counter Register (Offsets 0x08 and 0x18)

The live 32-bit count, readable at any time without disturbing the counter.

Configuration recipes (board-verified, from the course examples):

Free-running cycle counter — count up from zero and read elapsed 10 ns cycles:

```
Xil_Out32(TIMER + 0x00, 0x0);          /* TCSR0: stop, clear mode  */
Xil_Out32(TIMER + 0x04, 0x0);          /* TLR0: start value 0      */
Xil_Out32(TIMER + 0x00, 0x20);         /* LOAD pulse: TCR0 <- TLR0 */
Xil_Out32(TIMER + 0x00, 0x80);         /* ENT: run                 */
u32 cycles = Xil_In32(TIMER + 0x08);   /* TCR0: elapsed cycles     */
```

PWM output — `TLR0` = period, `TLR1` = high time, both counters in PWM-generate mode (`0x296` = `PWMA|T-run|ARHT|GENT|UDT`):

```
Xil_Out32(PWM + 0x04, PERIOD_CYC);     /* TLR0: period             */
Xil_Out32(PWM + 0x00, 0x20);           /* LOAD counter 0           */
Xil_Out32(PWM + 0x14, HIGH_CYC);       /* TLR1: high time          */
Xil_Out32(PWM + 0x10, 0x20);           /* LOAD counter 1           */
Xil_Out32(PWM + 0x00, 0x296);          /* TCSR0: PWMA|ENT|ARHT|GENT|UDT */
Xil_Out32(PWM + 0x10, 0x296);          /* TCSR1: same              */
```

Periodic interrupt — counter 0 counts down from `TLR0`, raises `TINT`, and reloads (`0xD2` = `ENT|ENIT|ARHT|UDT`). The handler is installed on the timer's interrupt-controller input (In0–In2) as shown in the Interrupt Controller section, and must clear `TINT` on every call:

```
Xil_Out32(TIMER + 0x04, PERIOD_CYC - 2);  /* TLR0: period (+2-cycle overhead)  */
Xil_Out32(TIMER + 0x00, 0x20);            /* LOAD pulse: TCR0 <- TLR0          */
Xil_Out32(TIMER + 0x00, 0xD2);            /* ENT|ENIT|ARHT|UDT: run, interrupt */

void timer_isr(void *ref)
{
    Xil_Out32(TIMER + 0x00, Xil_In32(TIMER + 0x00) | 0x100);  /* clear TINT */
    /* ... periodic work ... */
}
```

> **Note:** The PWM instances are ordinary timer blocks whose outputs are routed to pins — there are no separate PWM registers. In interrupt use, the handler must clear `TINT` by writing `TCSR` back with bit 8 set; an unacknowledged timer interrupt re-enters the handler forever. A program loaded over JTAG on top of a running program can inherit an already-armed timer interrupt, so interrupt-driven programs should disable and acknowledge the timers during initialization before enabling their own handlers.

> **Reference:** AMD AXI Timer LogiCORE IP Product Guide (PG079).

6.5 UART

6.5.1 USB UART (`uart_USB`)

Item	Value
Base Address	`0x4030_0000`
TX / RX Pins	J18 / J17 — routed to the on-board Micro-USB connector
Interrupt	INTC In4

Description: A 16550-compatible UART provides host PC communication through the on-board Micro-USB connector. The baud rate is software-configured via the divisor latch: at a 100 MHz system clock, divisor 54 (0x36) gives 115200 baud. This UART is typically mapped as STDIN/STDOUT.

6.5.2 External UART (`uart_1`)

Item	Value
Base Address	`0x4031_0000`
TX / RX Pins	J1 (DIP 11) / K2 (DIP 12)
Interrupt	INTC In3

Description: A second 16550 UART is exposed on the DIP connector for communication with external devices (e.g., sensor modules, Bluetooth modules).

6.5.3 UART Register Map

Both UARTs are 16550-compatible and share the same registers. The 16550 register bank begins at offset `0x1000` from the instance base address; the Offset column below is measured from that base. Each register is one byte wide in the 16550 and occupies the low 8 bits of a 32-bit bus word.

The registers at `0x1000` and `0x1004` are shared between two functions. When the divisor-latch access bit `LCR.DLAB` is set they address the baud-rate divisor (`DLL`, `DLM`); when it is clear they address the data and interrupt-enable registers.

Offset	Name	Access	Description
`0x1000`	`RBR` / `THR` / `DLL`	RO / WO / RW	Receive buffer (read) · transmit holding (write) · divisor LSB when `DLAB` = 1
`0x1004`	`IER` / `DLM`	RW	Interrupt enable · divisor MSB when `DLAB` = 1
`0x1008`	`IIR` / `FCR`	RO / WO	Interrupt identification (read) · FIFO control (write)
`0x100C`	`LCR`	RW	Line control: framing and `DLAB`
`0x1010`	`MCR`	RW	Modem control (no modem lines are wired on this device)
`0x1014`	`LSR`	RO	Line status
`0x1018`	`MSR`	RO	Modem status
`0x101C`	`SCR`	RW	Scratch register (general-purpose storage)

6.5.3.1 LCR — Line Control Register (Offset 0x100C)

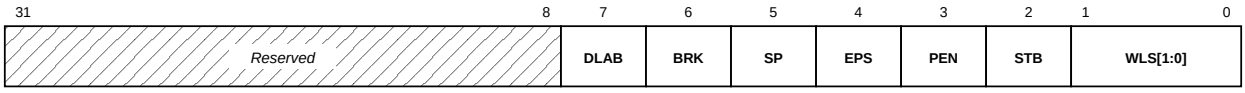


Figure 21: UART LCR bit layout: framing fields in bits 7:0, with the two-bit word-length select at bits 1:0

Bits	Name	Access	Description
31:8	—	—	Reserved.
7	`DLAB`	RW	Divisor latch access. Set to 1 to program `DLL`/`DLM`, then return to 0 to resume data access.
6	`BRK`	RW	Set break: hold the transmit line low.
5	`SP`	RW	Stick parity.
4	`EPS`	RW	Even parity select (1 = even, 0 = odd) when parity is enabled.
3	`PEN`	RW	Parity enable.
2	`STB`	RW	Stop bits: 0 = one stop bit, 1 = two stop bits.
1:0	`WLS`	RW	Word length: 00 = 5, 01 = 6, 10 = 7, 11 = 8 data bits.

8N1 framing is `WLS` = 11, `STB` = 0, `PEN` = 0, giving `LCR` = `0x03`.

6.5.3.2 LSR — Line Status Register (Offset 0x1014)

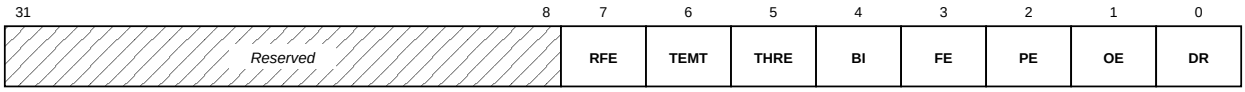


Figure 22: UART LSR bit layout: eight status flags in bits 7:0; DR at bit 0 and THRE at bit 5 are the poll bits

Bits	Name	Access	Description
31:8	—	—	Reserved.
7	RFE	RO	An errored byte is present in the receive FIFO.
6	TEMT	RO	Transmitter empty: both the holding register and the shift register are idle.
5	THRE	RO	Transmit holding register empty: ready to accept the next byte.
4	BI	RO	Break interrupt detected.
3	FE	RO	Framing error on the received byte.
2	PE	RO	Parity error on the received byte.
1	OE	RO	Overrun error: a received byte was lost.
0	DR	RO	Data ready: at least one received byte is available.

6.5.3.3 IER — Interrupt Enable Register (Offset 0x1004, DLAB = 0)

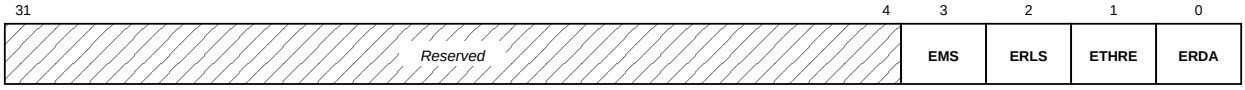


Figure 23: UART IER bit layout: four interrupt-enable bits in bits 3:0

Bits	Name	Access	Description
31:4	—	—	Reserved.
3	EMS	RW	Enable the modem-status interrupt.
2	ERLS	RW	Enable the receiver line-status interrupt.
1	ETHRE	RW	Enable the transmitter-holding-empty interrupt.
0	ERDA	RW	Enable the received-data-available interrupt.

6.5.3.4 FCR — FIFO Control Register (Offset 0x1008, write-only)



Figure 24: UART FCR bit layout: the trigger level in bits 7:6 and the FIFO enable and reset controls in bits 2:0

Bits	Name	Access	Description
31:8	—	—	Reserved.
7:6	RXTRIG	WO	Receive FIFO interrupt trigger level.
5:3	—	—	Reserved.
2	TXRST	WO	Reset (clear) the transmit FIFO.
1	RXRST	WO	Reset (clear) the receive FIFO.
0	FEN	WO	Enable the FIFOs.

Configuration recipe (board-verified). Set 115200 8N1. At the 100 MHz system clock the divisor is 100 MHz ÷ (16 × 115200) = 54:

```
Xil_Out32(UART + 0x100C, 0x80);   /* LCR: DLAB = 1 (open divisor latch)     */
Xil_Out32(UART + 0x1000, 54);     /* DLL: divisor low  (0x36)               */
Xil_Out32(UART + 0x1004, 0);      /* DLM: divisor high                      */
Xil_Out32(UART + 0x100C, 0x03);   /* LCR: DLAB=0, 8 data, 1 stop, no parity */
Xil_Out32(UART + 0x1008, 0x07);   /* FCR: enable and reset both FIFOs       */
```

Polled transmit and receive:

```
while (!(Xil_In32(UART + 0x1014) & 0x20)) ;  /* wait for THRE (LSR bit 5)  */
Xil_Out32(UART + 0x1000, c);                 /* THR: transmit one byte     */

if (Xil_In32(UART + 0x1014) & 0x01)          /* DR (LSR bit 0): byte ready */
    c = Xil_In32(UART + 0x1000);             /* RBR: the read pops the FIFO */
```

> **Note:** `DLAB` must be returned to 0 after the divisor is loaded, or accesses at `0x1000` and `0x1004` continue to address the divisor latch instead of the data and interrupt-enable registers. The divisor value 54 applies at the 100 MHz system clock; a different clock changes it (divisor = clock ÷ (16 × baud)).

> **Reference:** AMD AXI UART 16550 LogiCORE IP Product Guide (PG143).

6.6 I2C Controller

Item	Value
Base Address	`0x4070_0000`
SCL Frequency	100 kHz (standard mode)
Input Glitch Filter	50 ns on SCL and SDA — per the I2C tSP spike-suppression spec
SCL / SDA Pins	DIP Pin 13 (L1) / Pin 14 (L2)
Pull-ups	Weak FPGA internal pull-ups enabled; external 4.7 kΩ to 3.3 V recommended for real devices
Interrupt	INTC In6

Description: An I2C master controls external I2C devices (sensors, EEPROMs, OLED displays). The bus uses open-drain signaling: any device may only pull the line low, and the pull-up resistor returns it high. Devices are addressed by their 7-bit I2C address. Interrupt routing is listed in Section 4.4.

6.6.1 I2C Register Map

The registers below cover control, status, the FIFOs, the slave-mode address, and the software reset. Offsets are from the instance base (`0x4070_0000`).

Offset	Name	Access	Description
`0x1C`	`GIE`	RW	Global interrupt enable
`0x20`	`ISR`	R/TOW	Interrupt status
`0x28`	`IER`	RW	Interrupt enable
`0x40`	`SOFTR`	WO	Software reset. Write `0x0000000A`.
`0x100`	`CR`	RW	Control
`0x104`	`SR`	RO	Status
`0x108`	`TX_FIFO`	WO	Transmit FIFO
`0x10C`	`RX_FIFO`	RO	Receive FIFO (a read pops one entry)
`0x110`	`ADR`	RW	The controller's own address, used only when it is addressed as a slave. A master transfer sends the target's address as its first `TX_FIFO` byte instead.

6.6.1.1 CR — Control Register (Offset 0x100)

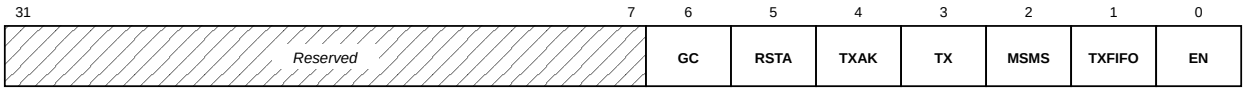


Figure 25: I2C CR bit layout: the master-transfer control fields occupy bits 6:0

Bits	Name	Access	Description
31:7	—	—	Reserved.
6	`GC`	RW	General-call enable (respond to the I2C general-call address).
5	`RSTA`	RW	Repeated start.
4	`TXAK`	RW	Transmit acknowledge: the level driven on ACK while receiving (1 = NACK).
3	`TX`	RW	Direction: 1 = transmit, 0 = receive.
2	`MSMS`	RW	Master mode: setting it issues a START and begins a master transfer; clearing it issues STOP.
1	`TXFIFO`	RW	Reset (clear) the transmit FIFO.
0	`EN`	RW	Enable the controller.

6.6.1.2 SR — Status Register (Offset 0x104)

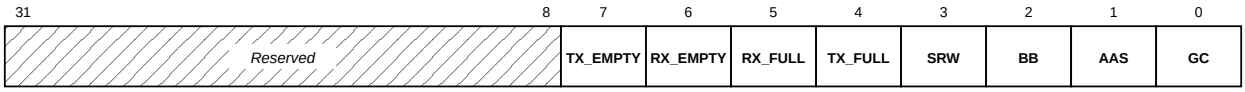


Figure 26: I2C SR bit layout: FIFO, bus-busy, and addressing status flags in bits 7:0

Bits	Name	Access	Description
31:8	—	—	Reserved.
7	TX_EMPTY	RO	Transmit FIFO empty.
6	RX_EMPTY	RO	Receive FIFO empty.
5	RX_FULL	RO	Receive FIFO full.
4	TX_FULL	RO	Transmit FIFO full.
3	SRW	RO	Slave read/write direction (when addressed as a slave).
2	BB	RO	Bus busy. Check this is clear before starting a transfer.
1	AAS	RO	Addressed as slave.
0	GC	RO	General call received.

> **Note:** A slave that never acknowledges (unpowered or mis-wired) can wedge the bus. Before starting a transfer, reset the controller by writing `0x0000000A` to `SOFTR` and confirm `SR.BB` (bit 2) is clear.

Transfer recipe (board-verified). Master transfers use the driver's low-level calls: each takes the controller base address and the 7-bit device address, runs the whole transaction (START, address, data, STOP), and returns the number of bytes transferred, which is 0 when no device acknowledges:

```
#include "xiic_l.h"

if (Xil_In32(I2C + 0x104) & 0x04)          /* SR.BB: bus wedged?            */
    Xil_Out32(I2C + 0x40, 0x0A);           /* SOFTR: reset the controller   */

u8 cmd[2] = { REG, VALUE };
unsigned n = XIic_Send(I2C, ADDR, cmd, 2, XIIC_STOP);  /* write two bytes    */

u8 val;
n = XIic_Recv(I2C, ADDR, &val, 1, XIIC_STOP);          /* read one byte      */
if (n == 0) {
    /* no acknowledge: device absent, unpowered, or wrong address */
}
```

Probing an address with a one-byte read is also how a bus scan finds the attached devices. A device that requires a repeated start between writing its register pointer and reading the data is read by passing `XIIC_REPEATED_START` instead of `XIIC_STOP` to the `XIic_Send` call.

> **Reference:** AMD AXI IIC Bus Interface LogiCORE IP Product Guide (PG090).

6.7 SPI Master

Item	Value
Base Address	`0x4080_0000`
SCLK	Software-programmable, ≈1.6 – 25 MHz (6.25 MHz at reset)
Clock Control	`0x4090_0000` — runtime serial-clock setting (see the Serial Clock Control Unit below)
Slave Selects	2 — two devices can share the bus
Pins	DIP 35 SCLK (V3) · 36 MOSI (W5) · 37 MISO (V4) · 38 SS0 (U4) · 39 SS1 (V5)
Interrupt	INTC In7

Description: An SPI master controls external devices (displays, ADCs, flash modules). The interface uses full-duplex push-pull signaling and requires no pull-up resistors; the active device is selected by driving its SS line low. The serial clock is set at runtime through the clock-control block: select a lower rate for breadboard wiring or long cables, and a higher rate for short-wired display modules.

> **Note:** This is a second, independent SPI controller. Do not confuse it with the QSPI flash controller at `0x4050_0000`, which is dedicated to the on-board boot flash; the two are addressed by their distinct base addresses.

6.7.1 SPI Register Map

Offsets are from the instance base (`0x4080_0000`). The registers below are the subset used for byte-level transfers.

Offset	Name	Access	Description
`0x1C`	`DGIER`	RW	Global interrupt enable
`0x20`	`ISR`	R/TOW	Interrupt status
`0x28`	`IER`	RW	Interrupt enable
`0x40`	`SRR`	WO	Software reset. Write `0x0000000A` to reset the core.
`0x60`	`CR`	RW	Control
`0x64`	`SR`	RO	Status
`0x68`	`DTR`	WO	Transmit data FIFO
`0x6C`	`DRR`	RO	Receive data FIFO (a read pops one entry)
`0x70`	`SSR`	RW	Slave select, active-low (one bit per slave)
`0x74`	`TFO`	RO	Transmit FIFO occupancy
`0x78`	`RFO`	RO	Receive FIFO occupancy

6.7.1.1 CR — Control Register (Offset 0x60)

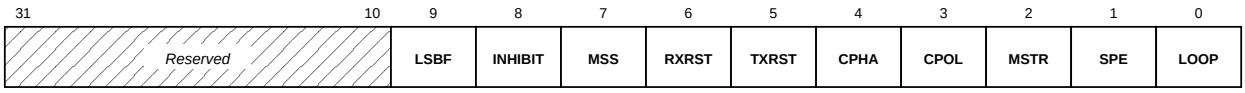


Figure 27: SPI CR bit layout: the transfer-control fields occupy bits 9:0

Bits	Name	Access	Description
31:10	—	—	Reserved.
9	`LSBF`	RW	Bit order: 1 = least-significant bit first, 0 = most-significant bit first (the SPI default).
8	`INHIBIT`	RW	Master transaction inhibit: set to hold off the transfer while the FIFO is loaded, clear to start.
7	`MSS`	RW	Manual slave-select mode: software drives `SSR`. Required for framed transactions.
6	`RXRST`	RW	Reset (clear) the receive FIFO.
5	`TXRST`	RW	Reset (clear) the transmit FIFO.
4	`CPHA`	RW	Clock phase.
3	`CPOL`	RW	Clock polarity.
2	`MSTR`	RW	Master mode (always set on this device).
1	`SPE`	RW	SPI system enable.
0	`LOOP`	RW	Internal loopback (transmit tied to receive) for self-test.

6.7.1.2 SR — Status Register (Offset 0x64)

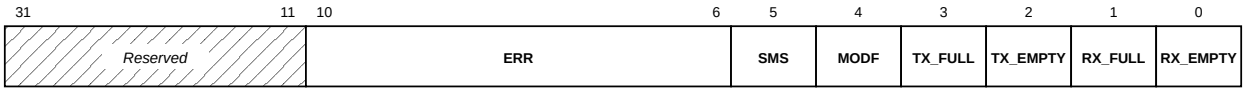


Figure 28: SPI SR bit layout: FIFO and fault flags in bits 4:0, the slave-mode select in bit 5, and the mode-error flags in bits 10:6

Bits	Name	Access	Description
31:11	—	—	Reserved.
10:6	`ERR`	RO	Mode-error flags of the dual and quad SPI modes; always 0 on this device.
5	`SMS`	RO	Slave-mode select: reads 1 while the core is not selected as a slave, which is always the case here (`SR` reads `0x25` after reset).
4	`MODF`	RO	Mode fault.
3	`TX_FULL`	RO	Transmit FIFO full.
2	`TX_EMPTY`	RO	Transmit FIFO empty.
1	`RX_FULL`	RO	Receive FIFO full.
0	`RX_EMPTY`	RO	Receive FIFO empty. Poll this to detect a received byte.

Byte transfer (board-verified). The control register is written in two steps — a setup value that resets the FIFOs, then a run value that does not — and the slave is framed by `SSR`:

```
Xil_Out32(SPI + 0x40, 0x0A);         /* SRR: reset the core                    */
Xil_Out32(SPI + 0x60, 0x1E6);        /* CR setup: enable, master, manual SS,   */
                                     /*           reset both FIFOs             */
/* settle ~10 us (about a dozen serial-clock periods) before the next write   */
Xil_Out32(SPI + 0x60, 0x086);        /* CR run: external transfer, SS via SSR  */
Xil_Out32(SPI + 0x70, ~(1u << n));   /* SSR: assert slave n (active-low)       */
Xil_Out32(SPI + 0x68, tx_byte);      /* DTR: queue a byte (<= 15 in flight)    */
while (Xil_In32(SPI + 0x64) & 0x01) ; /* SR.RX_EMPTY: wait for the reply       */
rx = Xil_In32(SPI + 0x6C);           /* DRR: the read pops the received byte   */
Xil_Out32(SPI + 0x70, 0xFFFFFFFF);   /* SSR: deassert all slaves               */
```

6.7.2 Serial Clock Control Unit

The serial clock is produced by a separate clock unit at `0x4090_0000`, which feeds the SPI core its bit clock. The rate is `SCK = 200 MHz ÷ N` with `N` from 8 to 128, giving roughly 1.6 to 25 MHz (6.25 MHz at reset, `N` = 32). Program it directly, then wait for the lock bit:

Offset	Name	Access	Description
`0x200`	`CFG0`	RW	Multiplier setting. Write `0x0801` (fixed for this device).
`0x208`	`CFG1`	RW	Divider `N`: `SCK = 200 MHz ÷ N`.
`0x25C`	`APPLY`	WO	Write `3` to apply the new setting.
`0x004`	`STATUS`	RO	Bit 0 = clock locked.

```
Xil_Out32(SPICLK + 0x200, 0x0801);        /* multiplier (fixed)             */
Xil_Out32(SPICLK + 0x208, N);             /* SCK = 200 MHz / N, N = 8..128  */
Xil_Out32(SPICLK + 0x25C, 3);             /* apply                          */
while (!(Xil_In32(SPICLK + 0x004) & 1)) ; /* wait for lock                  */
```

> **Note:** This SPI core clocks the bus asynchronously to the CPU. Reliable register-level transfers follow three rules: after a software reset or a FIFO-reset write, wait about a dozen serial-clock periods before the next register write; keep no more than 15 bytes in flight, because the FIFO holds 16 and `TX_FULL` lags; and detect received bytes by the `SR.RX_EMPTY` level, never by the occupancy counters, which lag in both directions.

> **Reference:** AMD AXI Quad SPI LogiCORE IP Product Guide (PG153).

6.8 Analog-to-Digital Converter (XADC)

Item	Value
Base Address	`0x4060_0000`
Resolution	12-bit
Conversion Rate	≈481 kSPS aggregate, ≈96 kSPS per channel
Sequencer	Continuous scan over the 5 enabled channels

Enabled Channels:

Channel	Source	Description
VAUX4	DIP Pin 15 (G3/G2)	External analog input 0 (on-board divider: 0–3.3 V → 0–1 V)
VAUX12	DIP Pin 16 (H2/J2)	External analog input 1 (on-board divider: 0–3.3 V → 0–1 V)
Temperature	Internal	FPGA die temperature monitor
VCCINT	Internal	Core voltage monitor (1.0 V)
VCCAUX	Internal	Auxiliary voltage monitor (1.8 V)

Description: The Artix-7 built-in 12-bit ADC measures external analog signals and monitors internal FPGA temperature and supply voltages. External input pins pass through an on-board resistive voltage divider (2.32 kΩ / 1 kΩ, ratio ≈ 0.301), accepting up to 3.3 V at the DIP pin.

Effective Per-Channel Sampling Rate: The sequencer continuously cycles through all 5 enabled channels (VAUX4, VAUX12, Temperature, VCCINT, VCCAUX). The converter clock is 100 MHz ÷ 8 = 12.5 MHz and each conversion takes 26 converter clocks, so the aggregate rate is ≈481 kSPS and each channel is refreshed at ≈481 k ÷ 5 ≈ 96 kSPS.

6.8.1 XADC Register Map

The registers below are the status register and the per-channel result registers. Offsets are from the instance base (`0x4060_0000`).

Offset	Name	Access	Description
`0x000`	`SRR`	WO	Software reset. Write `0x0000000A`.
`0x004`	`SR`	RO	Status: conversion busy, end-of-conversion, end-of-sequence.
`0x200`	`TEMP`	RO	Die-temperature result
`0x204`	`VCCINT`	RO	Core-supply result
`0x208`	`VCCAUX`	RO	Auxiliary-supply result
`0x250`	`VAUX4`	RO	External analog input 0 (DIP pin 15)
`0x270`	`VAUX12`	RO	External analog input 1 (DIP pin 16)

6.8.1.1 SR — Status Register (Offset 0x004)

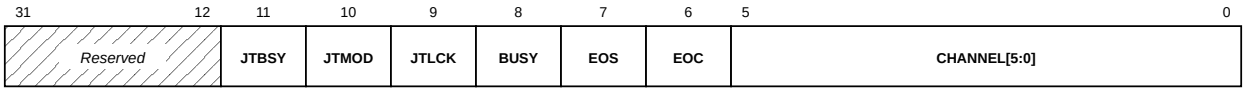


Figure 29: XADC SR bit layout: conversion status in bits 8:6 and the current channel number in bits 5:0

Bits	Name	Access	Description
31:12	—	—	Reserved.
11	`JTBSY`	RO	Configuration-port access busy.
10	`JTMOD`	RO	A configuration-port write has occurred.
9	`JTLCK`	RO	Configuration-port access is locked.
8	`BUSY`	RO	A conversion is in progress.
7	`EOS`	RO	End of sequence: the sequencer finished one pass over the channels.
6	`EOC`	RO	End of conversion: a new sample is ready.
5:0	`CHANNEL`	RO	Number of the channel just converted.

6.8.1.2 Conversion Data Format

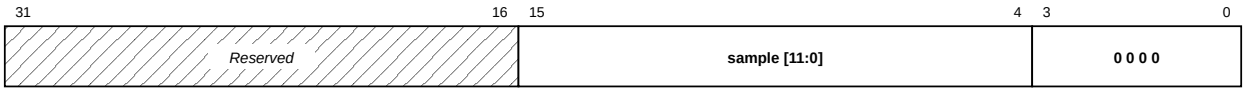


Figure 30: XADC result format: the 12-bit sample is left-justified in bits 15:4; bits 3:0 read 0

Each result register returns a 12-bit sample left-justified in bits 15:4 of the low half-word; bits 3:0 read 0. Shift the register right by 4 to obtain the 12-bit value. An external input passes through the on-board divider (ratio ≈ 0.301), so the voltage at the DIP pin is approximately `(sample ÷ 4096) × 3.32 V` (full scale ≈ 3.3 V).

> **Note:** The sequencer scans the five enabled channels continuously, so each result register always holds the most recent sample and no start-conversion step is required. A result read returns the left-justified 16-bit word; shift right by 4 for the 12-bit value.

Reading recipe (board-verified). One load per channel, then the conversion to physical units in integer arithmetic:

```
u32 t   = Xil_In32(XADC + 0x200) >> 4;   /* TEMP: 12-bit code            */
u32 vcc = Xil_In32(XADC + 0x204) >> 4;   /* VCCINT (0x208: VCCAUX)       */
u32 a0  = Xil_In32(XADC + 0x250) >> 4;   /* VAUX4 = DIP pin 15 (0x270: 16) */

int mdegC  = (int)(t * 503975u / 4096) - 273150;  /* die temperature, m°C   */
u32 vcc_mV = vcc * 3000u / 4096;                  /* supplies: 3 V scale    */
u32 pin_mV = a0 * 3322u / 4096;                   /* DIP pin, via divider   */
```

At room temperature the die reads roughly 45–60 °C, `VCCINT` ≈ 1.0 V and `VCCAUX` ≈ 1.8 V; these internal channels are a quick self-test of the ADC without any wiring.

> **Reference:** AMD XADC Wizard LogiCORE IP Product Guide (PG091).

6.8.2 Analog Input Circuit

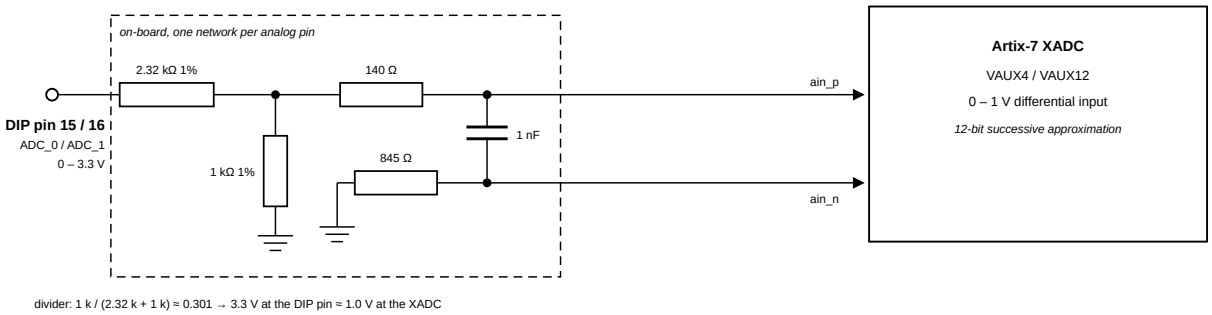


Figure 31: On-board voltage divider circuit for XADC analog inputs

The XADC expects an input range of 0–1 V. The board includes a resistive voltage divider (2.32 kΩ / 1 kΩ, all 1% precision) that scales the DIP pin voltage down to the FPGA's acceptable range. A 140 Ω series resistor and 1 nF capacitor are placed on the differential pair for filtering. The ADx_N path has an 845 Ω series resistor.

Parameter	Value
Max input voltage on DIP pin 15/16	**3.3 V** (relative to GND on pin 25)
Voltage at FPGA after divider	0–1 V
Divider ratio	1 kΩ / (2.32 kΩ + 1 kΩ) ≈ 0.301
Resistor tolerance	1%

> **Note:** Pins 15 and 16 are routed through on-board voltage dividers and are dedicated analog inputs; they are not available as digital I/O. Do not exceed 3.3 V on these pins.

6.9 Interrupt Controller

The interrupt controller and its source-to-input map are introduced in the System Architecture chapter, with the block diagram. This section documents its registers. Offsets are from the instance base (`0x4040_0000`). The status, enable, pending, and acknowledge registers all use the same layout: bit `n` corresponds to interrupt input `n`.

6.9.1 Interrupt Controller Register Map

Offset	Name	Access	Description
`0x00`	`ISR`	RW	Interrupt status (raw, one bit per input)
`0x04`	`IPR`	RO	Pending = status AND enable
`0x08`	`IER`	RW	Interrupt enable (one bit per input)
`0x0C`	`IAR`	WO	Acknowledge: write 1 to a bit to clear that handled interrupt
`0x10`	`SIE`	WO	Set individual enable bits
`0x14`	`CIE`	WO	Clear individual enable bits
`0x18`	`IVR`	RO	Number of the lowest pending input
`0x1C`	`MER`	RW	Master enable

6.9.1.1 IER — Interrupt Enable Register (Offset 0x08)

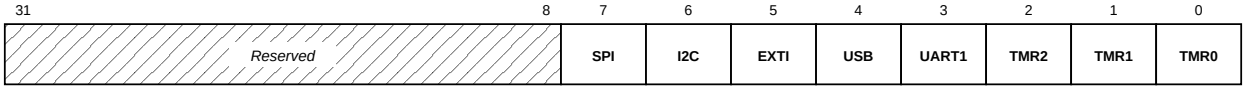


Figure 32: Interrupt enable register: bit 0 enables Timer 0 through bit 7 enabling the SPI master, matching the interrupt map

Each bit enables one source; the bit position is the input number (bit 0 = Timer 0 … bit 7 = SPI). `ISR`, `IPR`, and `IAR` share this bit-per-input layout. Bits 31:8 are reserved.

6.9.1.2 MER — Master Enable Register (Offset 0x1C)

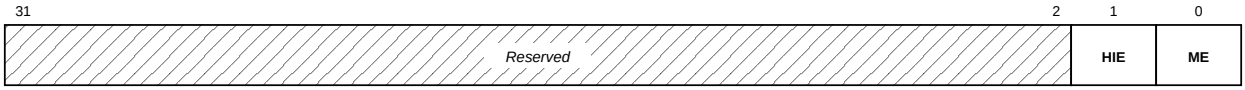


Figure 33: Master enable register: the master enable and hardware-interrupt-enable bits in bits 1:0

Bits	Name	Access	Description
31:2	—	—	Reserved.
1	`HIE`	RW	Hardware interrupt enable (write-once).
0	`ME`	RW	Master enable: gates all interrupt output to the processor.

> **Note:** Enabling an interrupt takes two gates: the per-source bit in `IER` and the master enable `MER.ME`. When loading a new program over JTAG, clear `MER` and `IER` first — an interrupt left pending by the previous program is taken as soon as the master enable is set, before the new handlers are installed.

Installing a handler (board-verified). The processor enters every external interrupt through the board-support package's trap entry, and the controller's driver calls the C handler registered for the active input. The complete sequence for one source, input `N` of the interrupt map:

```
#include "xintc.h"
#include "xil_exception.h"
#include "cmod_mcu.h"                /* ITCM_FUNC, DTCM_DATA */

DTCM_DATA static XIntc intc;         /* read on every interrupt */

XIntc_Initialize(&intc, XPAR_XINTC_0_BASEADDR);
XIntc_Connect(&intc, N, (XInterruptHandler)my_isr, NULL); /* handler for input N */
XIntc_Start(&intc, XIN_REAL_MODE);                        /* MER: ME + HIE        */
Xil_ExceptionInit();
Xil_ExceptionRegisterHandler(XIL_EXCEPTION_ID_INT,
                             (Xil_ExceptionHandler)XIntc_InterruptHandler, &intc);
/* ... enable the interrupt inside the peripheral itself ... */
XIntc_Enable(&intc, N);                                   /* IER: bit N           */
Xil_ExceptionEnable();                                    /* CPU: take interrupts */
```

`my_isr` is a C function `void my_isr(void *ref)`, declared `ITCM_FUNC` so that it runs from the ITCM. The driver acknowledges the controller (`IAR`) itself; the handler must clear the flag inside the peripheral (`TINT` for a timer, `ISR` for the external interrupt port), or the interrupt is taken again immediately. Further sources are added with one more `XIntc_Connect` / `XIntc_Enable` pair each. The trap entry and the dispatch code always run from the ITCM, and the tables they read live in the DTCM (`mcu/cmod_mcu.ld` places them). With the handler in the ITCM and the `XIntc` instance in the DTCM, as above, an interrupt reaches the handler in 187 cycles (1.87 µs) whatever the cache state (see Interrupt Latency in the Memory Specification).

> **Reference:** AMD AXI Interrupt Controller LogiCORE IP Product Guide (PG099).

7 JTAG Debug Mode

This chapter describes how to load and debug a MicroBlaze RISC-V application over JTAG using the AMD Vitis Unified IDE. JTAG loads the program directly into RAM and provides full debug control. Nothing is written to flash.

7.1 Prerequisites

- The hardware design as an `.xsa` file: use the prebuilt `release/top_wrapper.xsa`
- Vitis Unified IDE 2025.2 installed
- Cmod A7-35T board connected via its micro-USB cable (the cable carries power, UART, and JTAG)
- A serial terminal program for the board's console output

7.2 Open the Workspace

Launch the Vitis Unified IDE and click **Set Workspace** on the Welcome page.

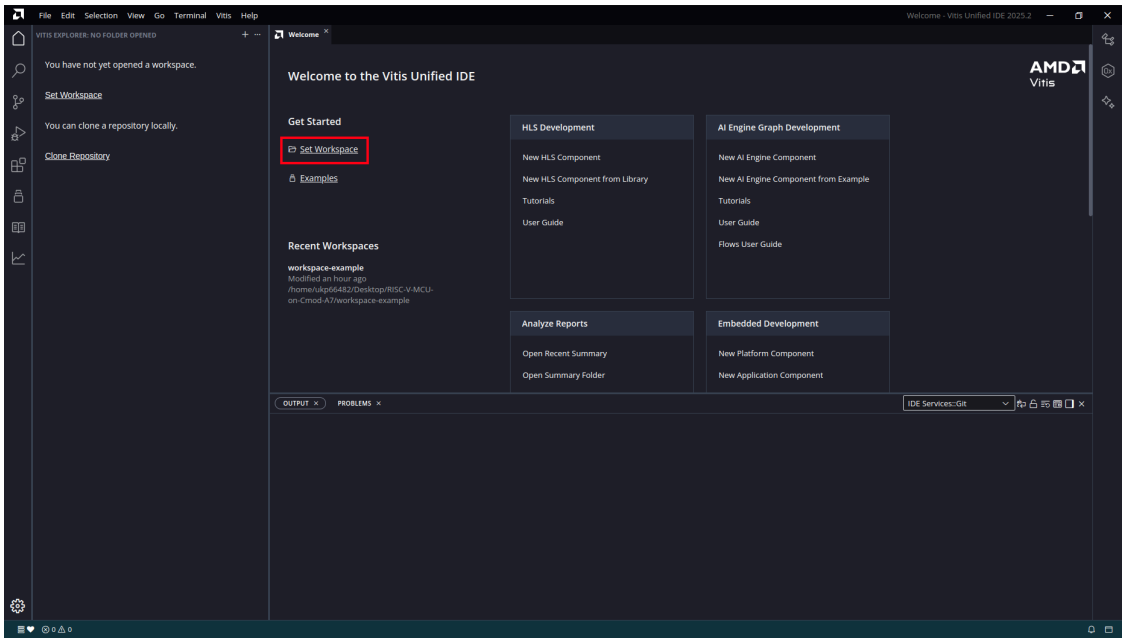


Figure 34: Set Workspace

Select the `workspace-example/` folder in the repository. The template paths in this chapter assume this location; any empty folder can also be used if the paths are adjusted accordingly.

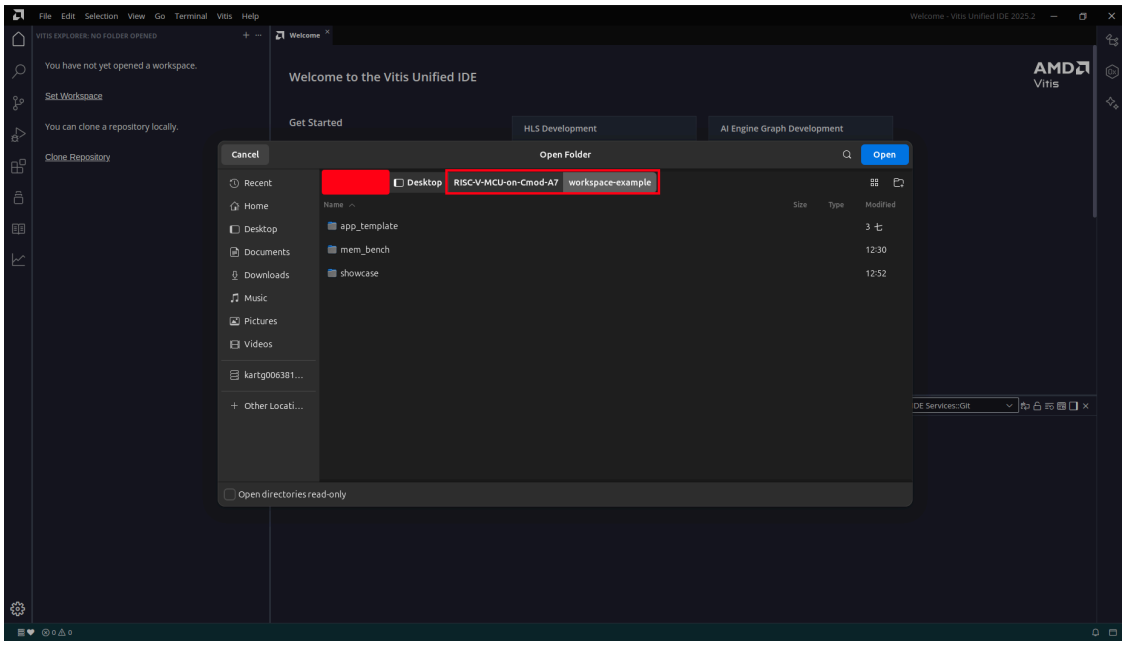


Figure 35: Open Folder

The first time a Vitis installation opens this workspace it may show an **Update Workspace** dialog. Click **Update**. This action refreshes only the workspace metadata and does not modify any component.

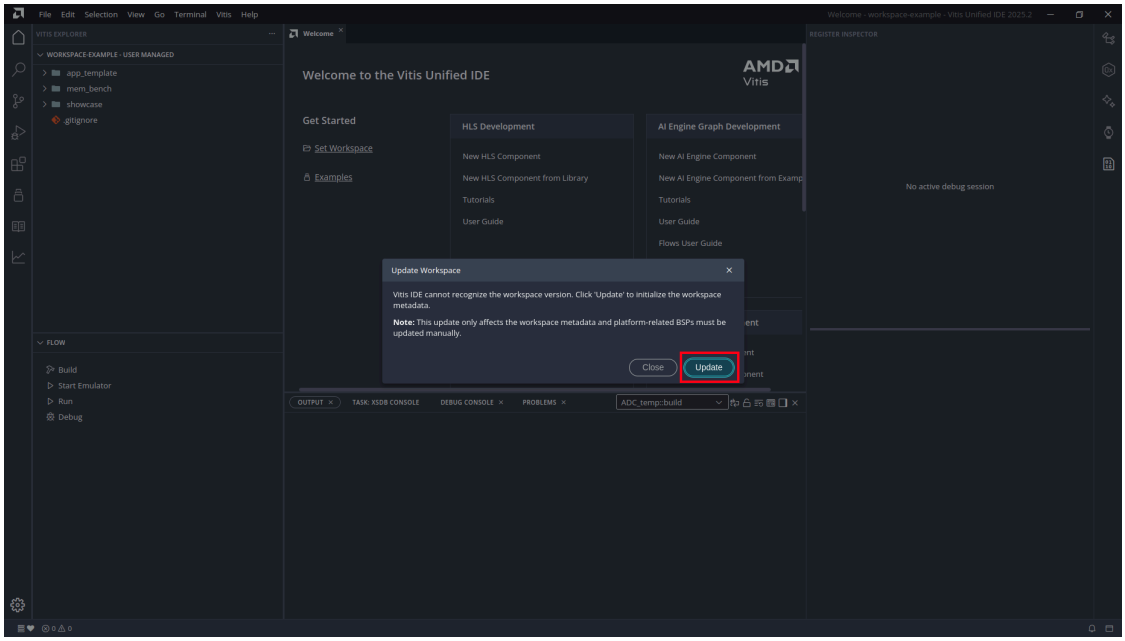


Figure 36: Update Workspace

7.3 Create a Platform

The platform wraps the hardware design and provides the driver layer (BSP). It is created once per workspace and reused by every application. If the workspace already contains a `platform` component, skip to Section 7.4.

7.3.1 New Platform Component

From the menu bar, select **File > New Component > Platform**.

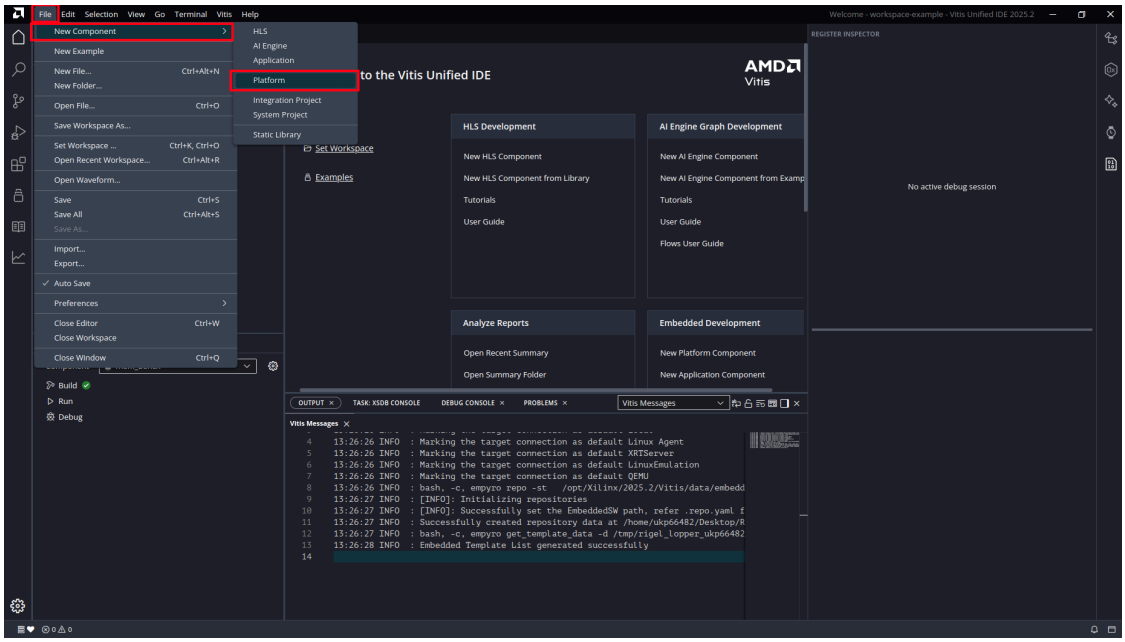


Figure 37: File > New Component > Platform

7.3.2 Platform Name and Location

Enter `platform` as the component name and keep the workspace directory as the location. Click **Next**.

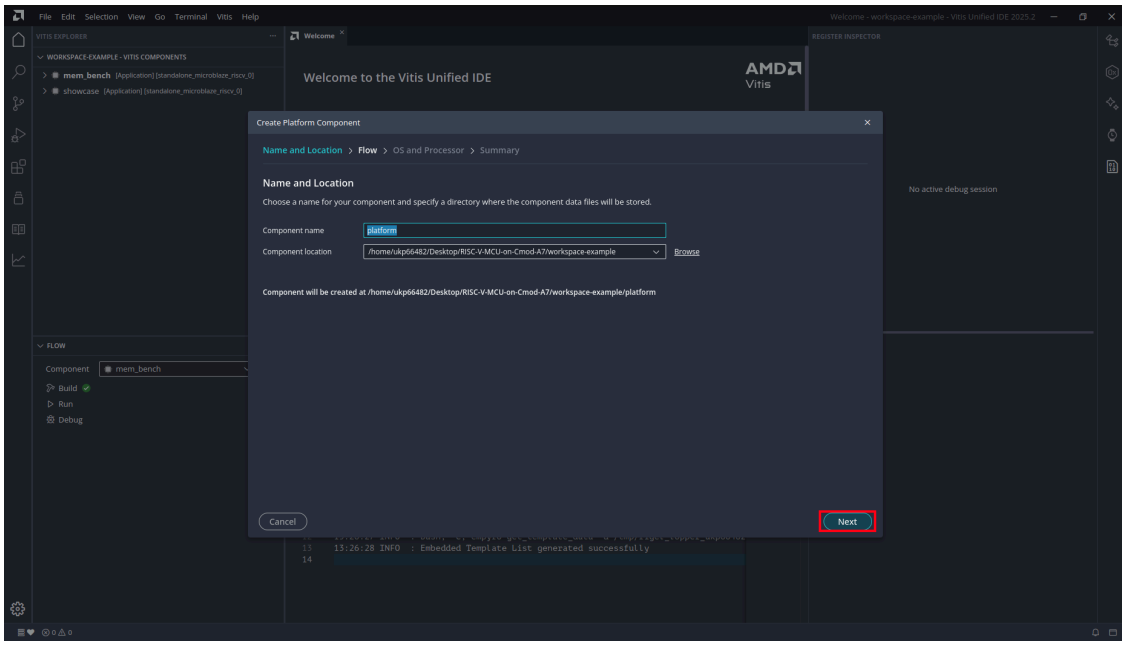


Figure 38: Platform Name and Location

7.3.3 Select the Hardware Design (XSA)

On the **Flow** page, select **Hardware Design** and click **Browse** next to the *Hardware Design (XSA) For Implementation* field.

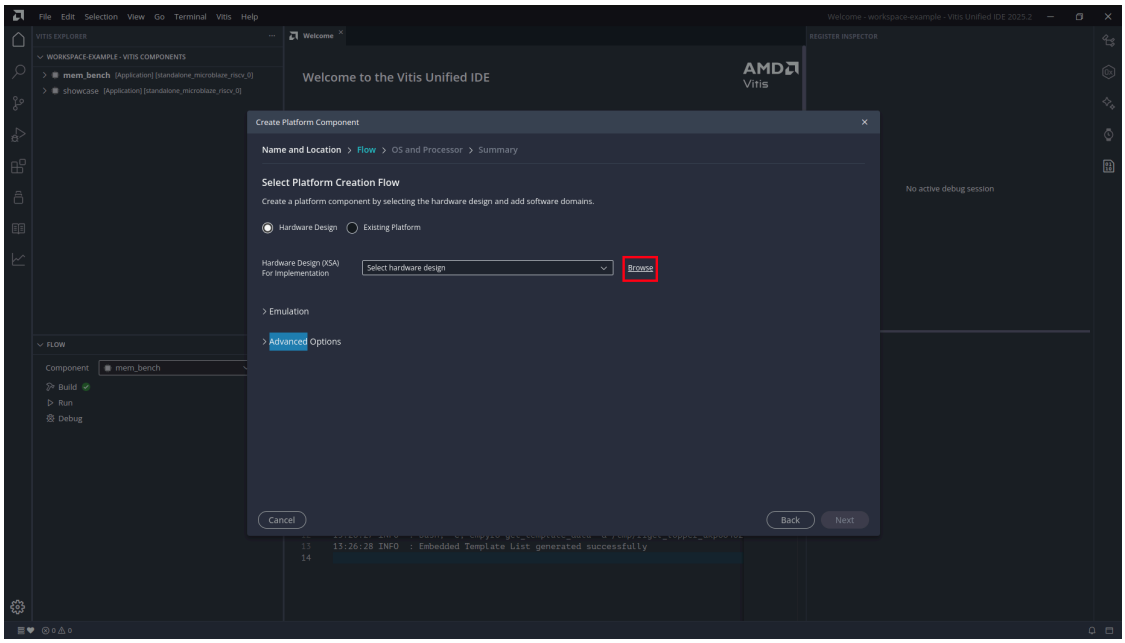


Figure 39: Select Platform Creation Flow

Select the prebuilt `release/top_wrapper.xsa` from the repository.

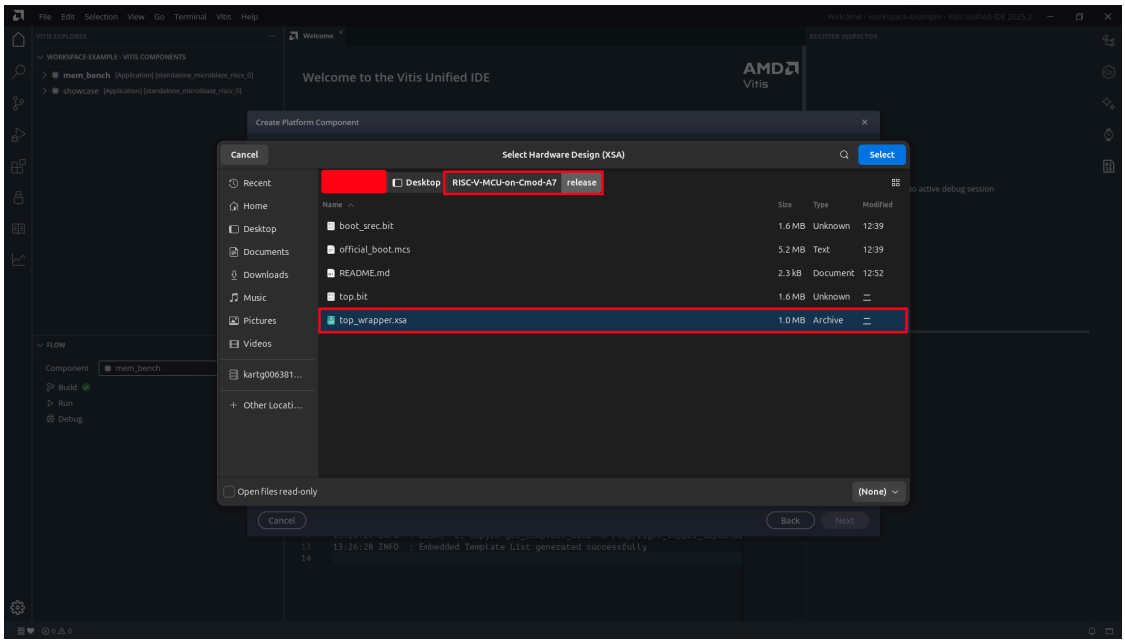


Figure 40: Select top_wrapper.xsa

7.3.4 Select Operating System and Processor

- **Operating system**: `standalone`
- **Processor**: `microblaze_riscv_0`

Click **Next**.

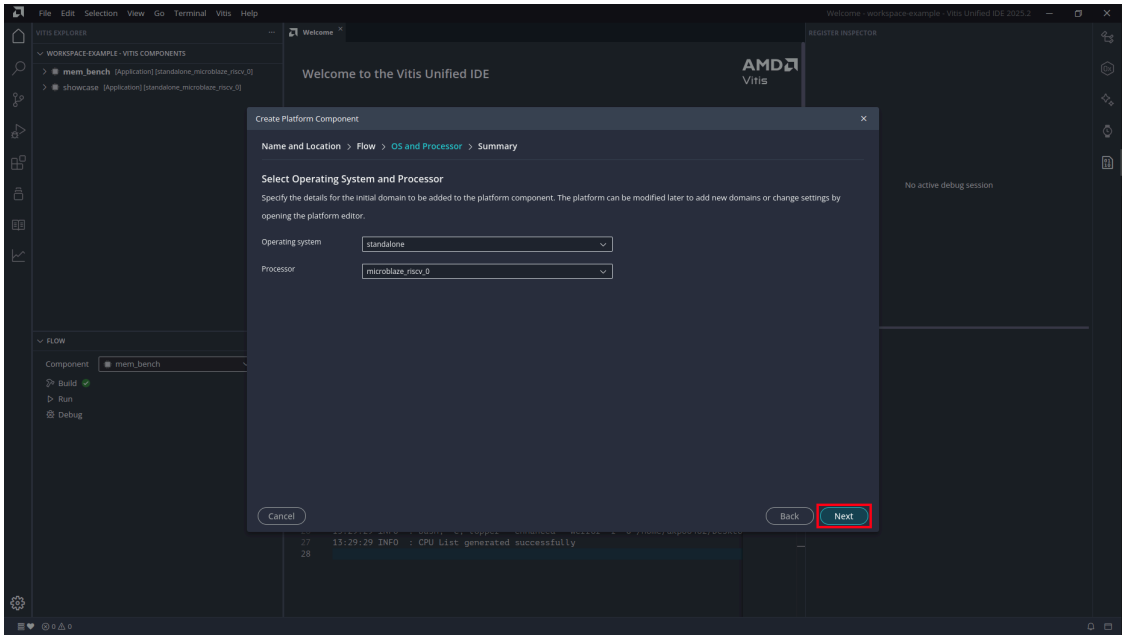


Figure 41: OS and Processor

7.3.5 Platform Summary

Review the settings and click **Finish**.

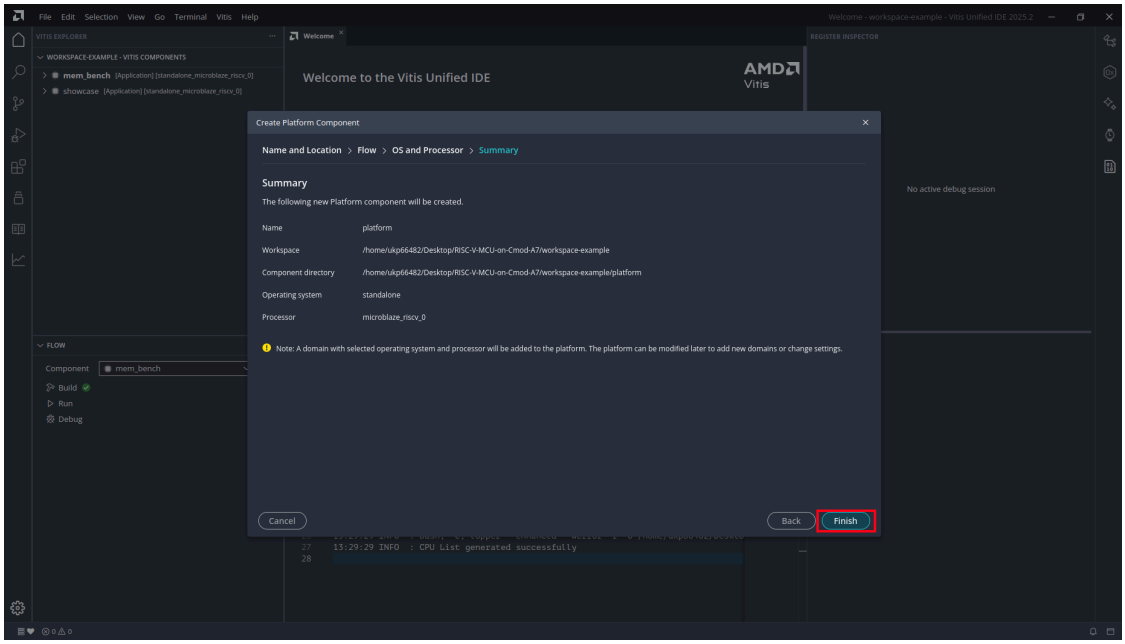


Figure 42: Platform Summary

Vitis generates the platform component. When it finishes, the Explorer shows the `platform` component with its `Sources` and `Output` trees, and the notification *“Created platform: platform”* appears.

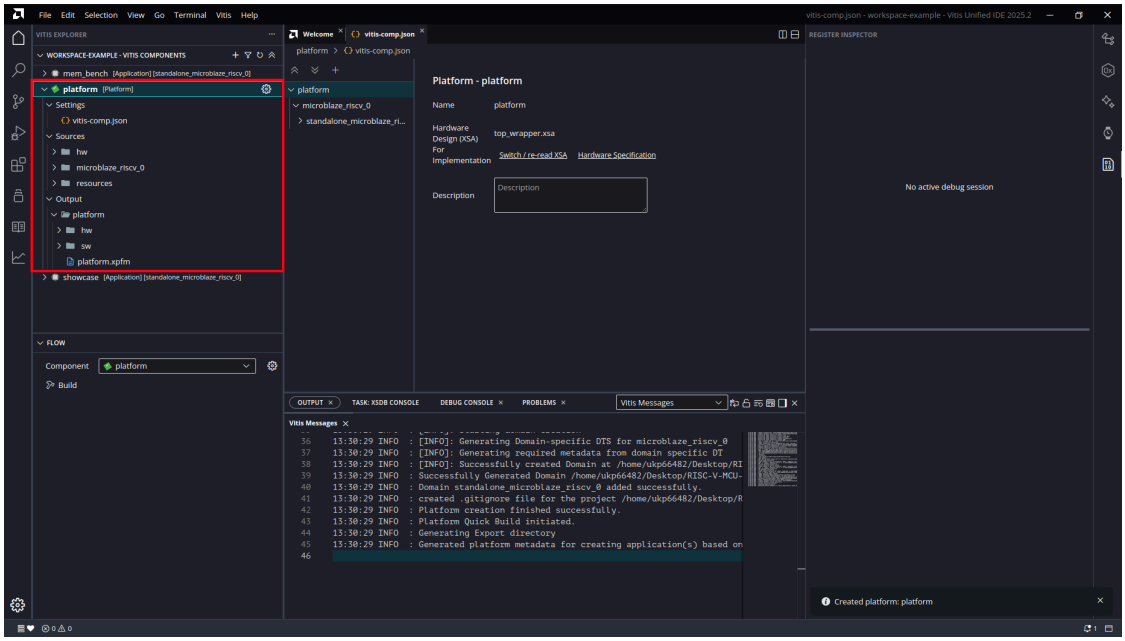


Figure 43: Platform Created

7.4 Create an Application

7.4.1 Create from Template

Open the **Examples** view from the left activity bar. Under *Embedded Software Examples*, select **Hello World** and click the **+** (Create Application Component from Template) button.

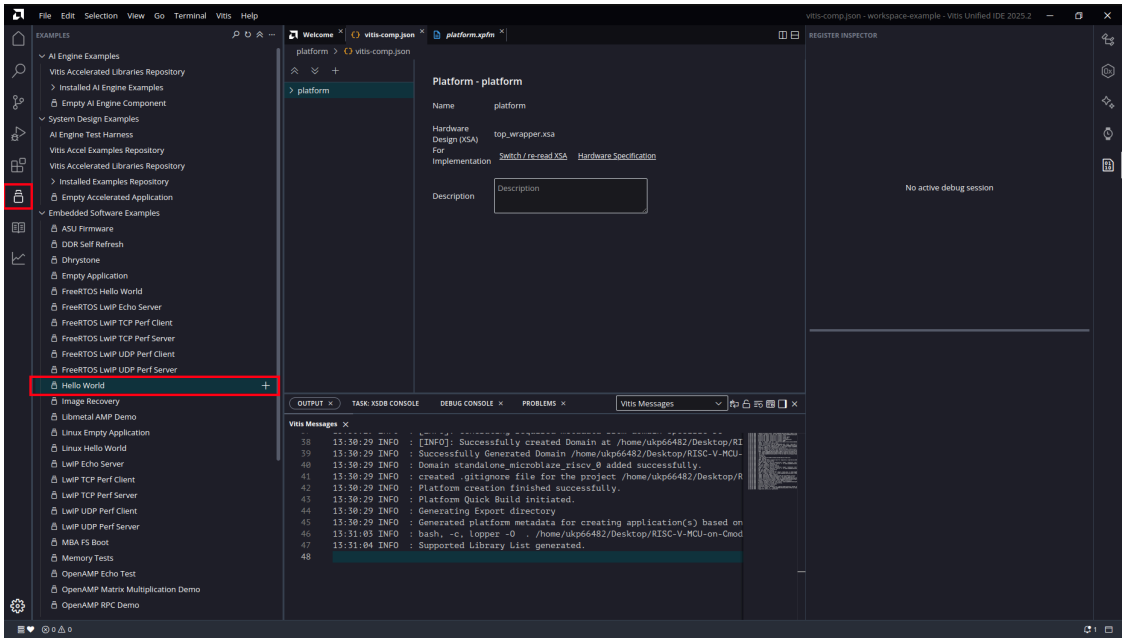


Figure 44: Hello World Template

7.4.2 Application Name and Location

Enter your application name (this chapter uses `test`). Click **Next**.

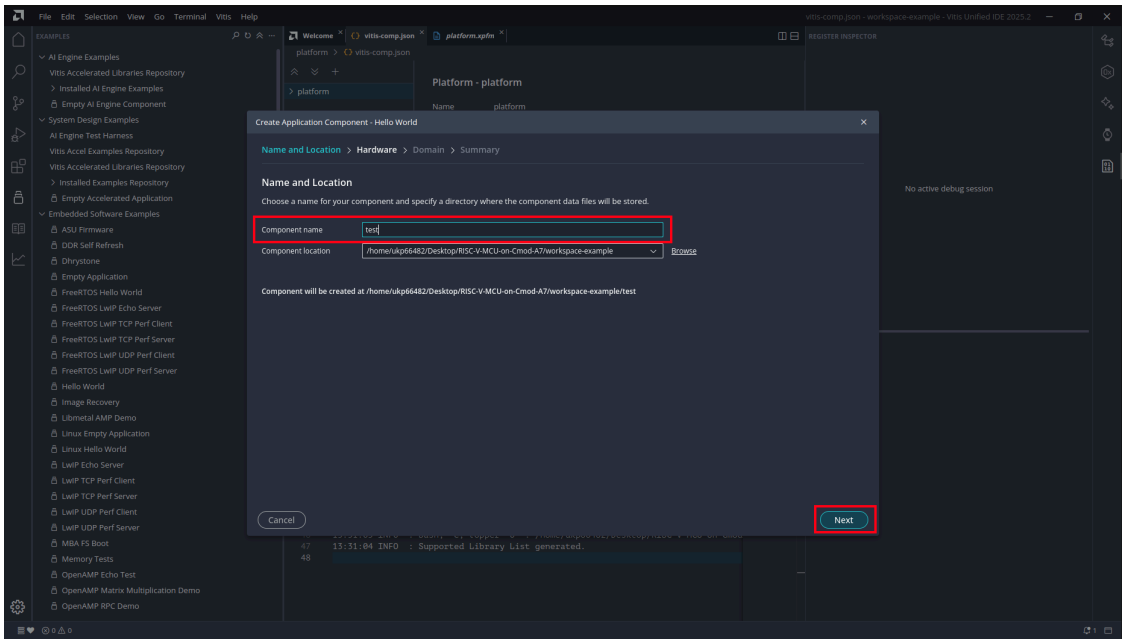


Figure 45: Application Name

7.4.3 Select the Platform

Select the `platform` component created in Section 7.3 (Board: `cmod_a7-35t`). Click **Next**.

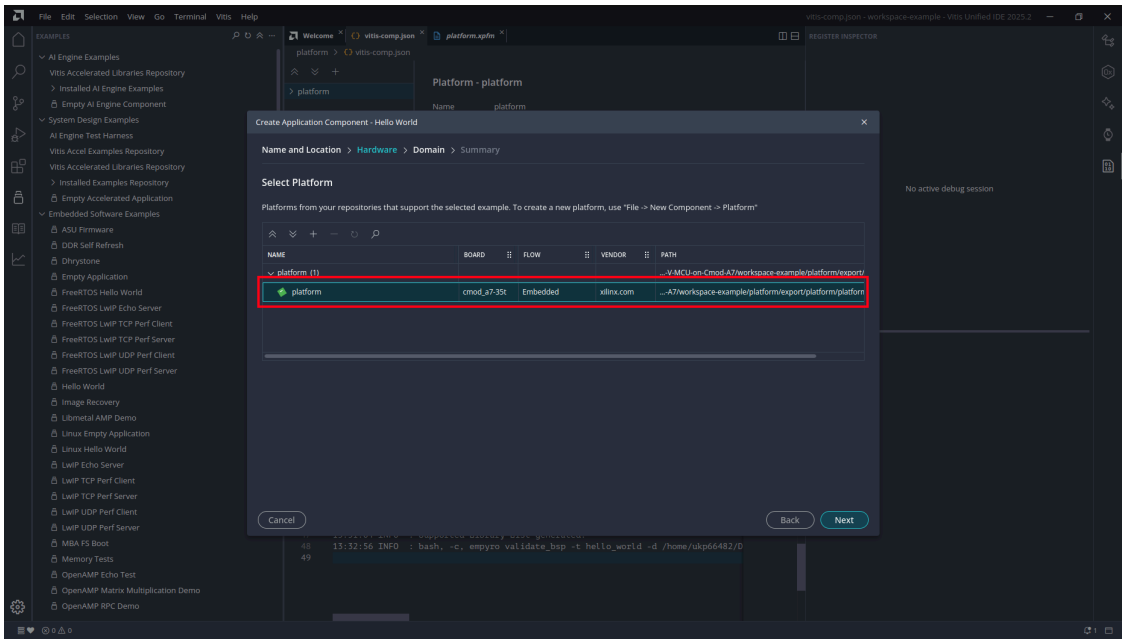


Figure 46: Select Platform

7.4.4 Select the Domain

Select `standalone_microblaze_riscv_0`. Click **Next**.

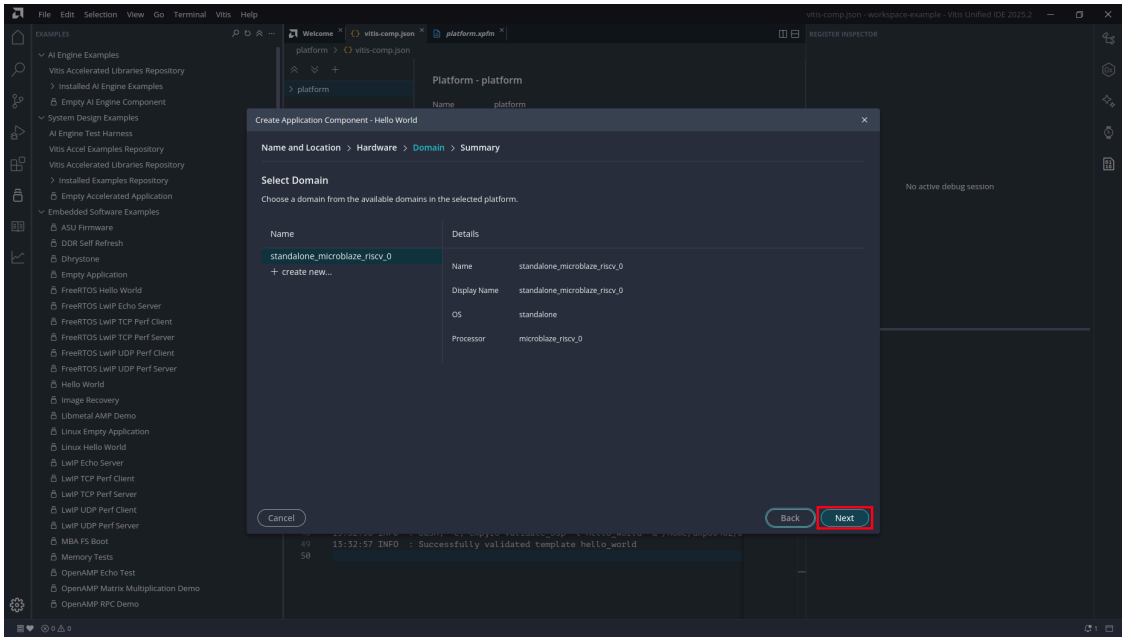


Figure 47: Select Domain

7.4.5 Application Summary

Review the settings and click **Finish**.

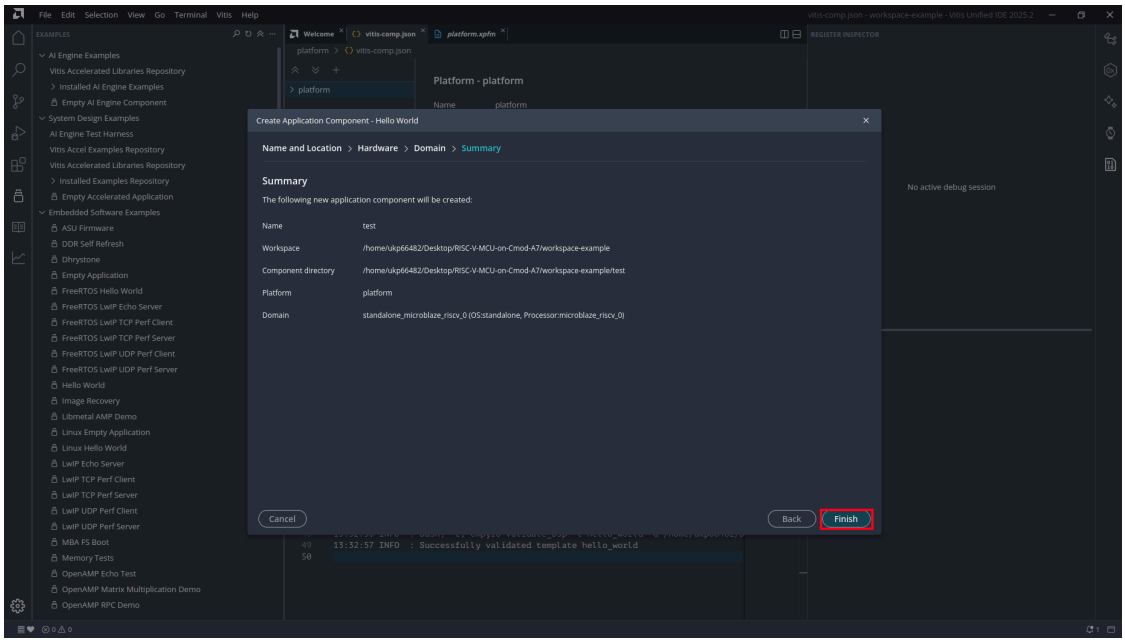


Figure 48: Application Summary

The new component appears in the Explorer. Its `src/` folder holds the template's generated files: `helloworld.c`, `lscript.ld`, `platform.c`, and `platform.h`.

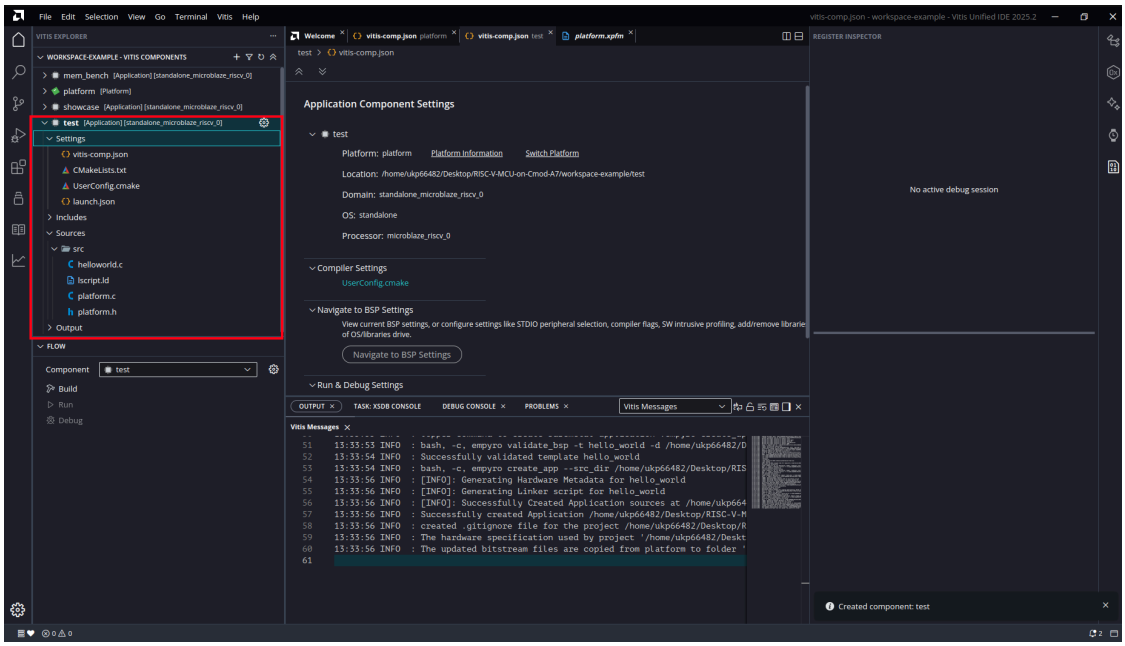


Figure 49: Application Created

7.5 Replace the Sources with the Course Template

The generated Hello World sources are replaced by the course template's `main.c`, and the component is pointed at the workspace's `mcu/` folder, which holds the two files every application builds against:

File	Content
`mcu/cmod_mcu.h`	Device header: base address, register offsets, and bit masks of every peripheral (C and assembly), plus `REG32()`, the `ITCM_FUNC`/`DTCM_DATA` placement attributes, and `mcu_init()` for C
`mcu/cmod_mcu.ld`	Linker script: places code and data in ITCM, DTCM, and SRAM

7.5.1 Delete the Generated Files

In the Explorer, select all four files under `src/` (`helloworld.c`, `lscript.ld`, `platform.c`, `platform.h`), right-click, and choose **Delete**.

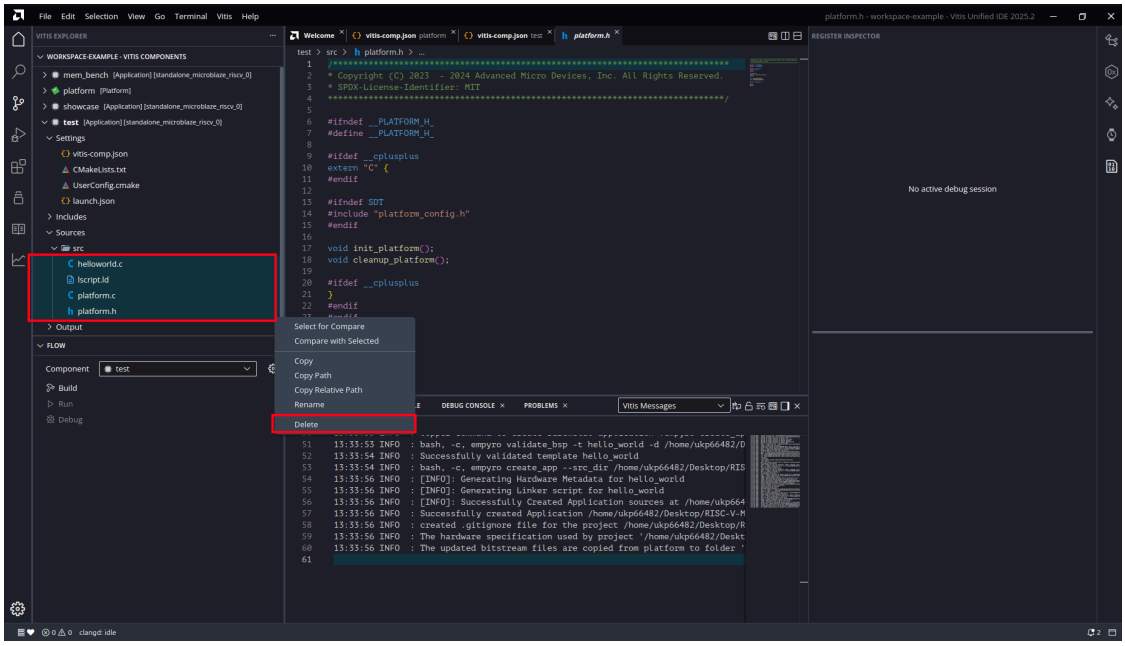


Figure 50: Delete Generated Sources

7.5.2 Import the Template File

Right-click the `src/` folder and choose **Import > Files…**

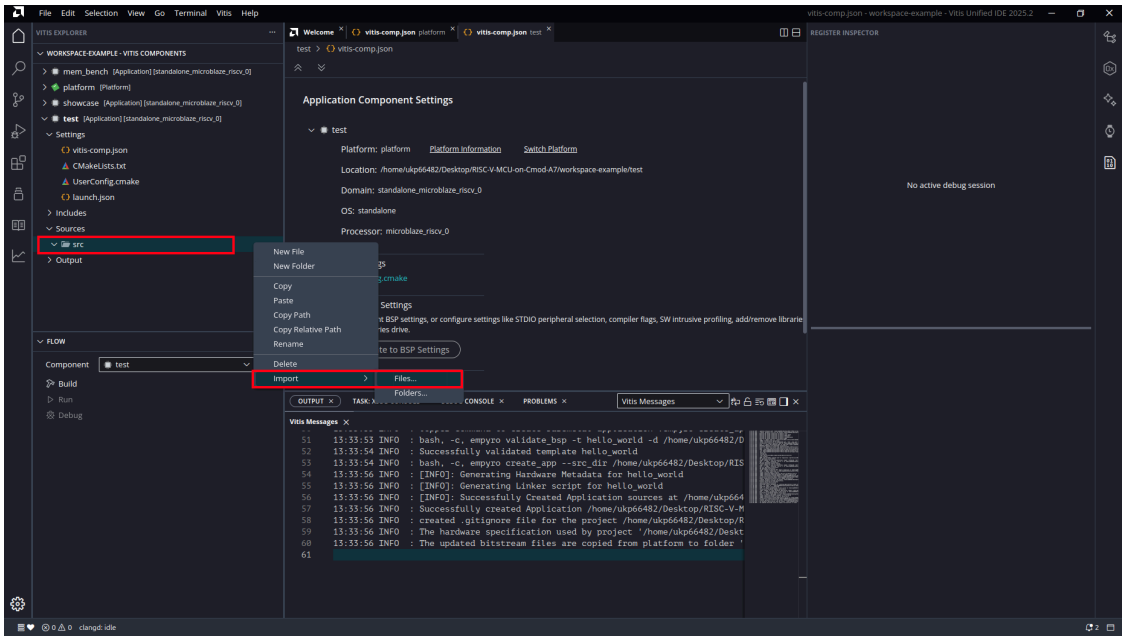


Figure 51: Import Files

Navigate to `workspace-example/app_template/src/` and select `main.c`. Click **Open**.

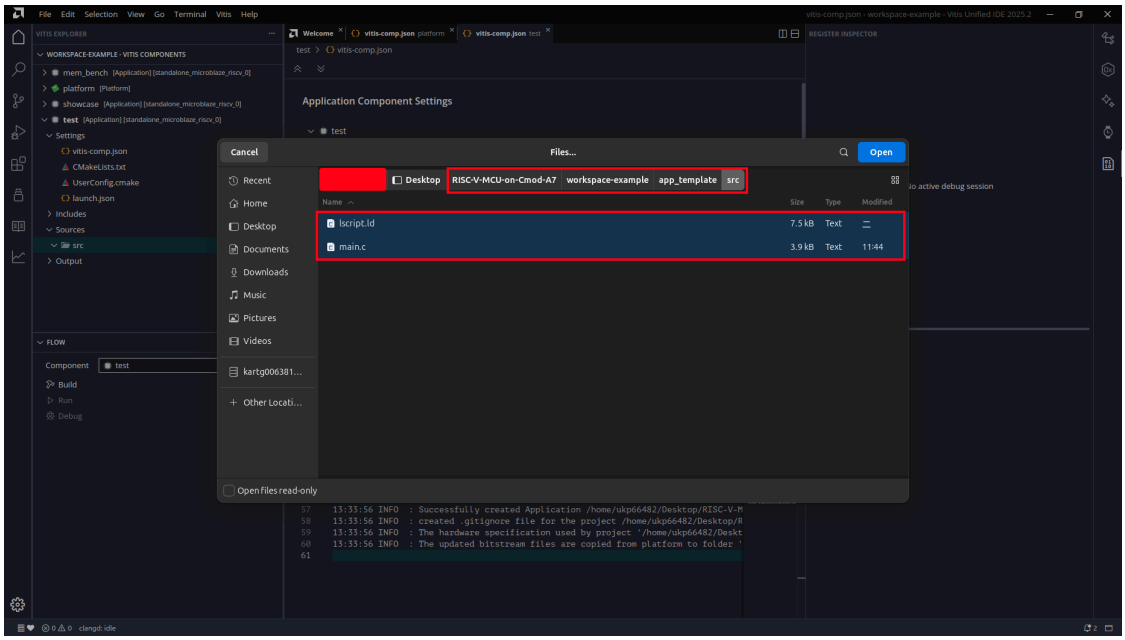


Figure 52: Select Template Files

Afterwards `src/` contains only `main.c`. C files inside `src/` are compiled automatically. Assembly files (`.S`) are not collected automatically: list them under **Compiler Settings > Sources > Compile sources** (`USER_COMPILE_SOURCES`) in the component's settings.

7.5.3 Point the Component at the MCU Folder

Open the component's **Settings > UserConfig.cmake** and set two values:

Setting	Value
Compiler Settings > Includes > Include paths (-I)	`${CMAKE_SOURCE_DIR}/../../mcu`
Linker Settings > Linker Script > Linker script file	`${CMAKE_SOURCE_DIR}/../../mcu/cmod_mcu.ld`

`${CMAKE_SOURCE_DIR}` is the component's `src/` folder, so both values refer to `workspace-example/mcu/`. In the file `src/UserConfig.cmake` the two settings read:

```
set(USER_INCLUDE_DIRECTORIES
"${CMAKE_SOURCE_DIR}/../../mcu"
)
set(USER_LINKER_SCRIPT "${CMAKE_SOURCE_DIR}/../../mcu/cmod_mcu.ld")
```

> **Note:** Keep `mcu_init();` as the first line of `main()`. It switches interrupts off until the application enables them (a program loaded over a running one must not inherit its interrupts), and it sets the USB UART to 115200; the bootloader leaves it at 9600.

The demo loop cycles the on-board RGB LED and prints a status line about once a second.

7.5.4 About the Linker Script

`cmod_mcu.ld` defines three memory regions: `ITCM` (32 KB), `DTCM` (64 KB), and `SRAM` (512 KB). Everything goes to SRAM except `ITCM_FUNC` code and the interrupt entry path (ITCM), and `DTCM_DATA` variables, the interrupt tables, and the stack (DTCM). The loader writes ITCM and DTCM directly, so `ITCM_FUNC` code and `DTCM_DATA` variables, initial values included, are in place before `main()` runs.

The stack receives all DTCM not used by `DTCM_DATA`, growing down from `0x0001_FFF0`; the heap receives all SRAM after the program. `_STACK_SIZE` and `_HEAP_SIZE` at the top of `cmod_mcu.ld` are minimums: the link fails if less remains. A different minimum for one application goes under **Linker Settings > Miscellaneous > Linker flags**, for example `-Wl,--defsym=_STACK_SIZE=0x4000`. The linker-script generator on the settings page writes a new `lscript.ld`, which this project does not use.

7.6 Set the Console UART (BSP)

The BSP determines which UART carries `xil_printf` output. The default is `uart_1` (the DIP-header pins); with the default setting, no output appears on the USB console.

Open the platform's **Settings > vitis-comp.json**, navigate to **standalone** (Board Support Package > microblaze_riscv_0 > standalone). The configuration table shows `standalone_stdin` and `standalone_stdout` set to `uart_1`:

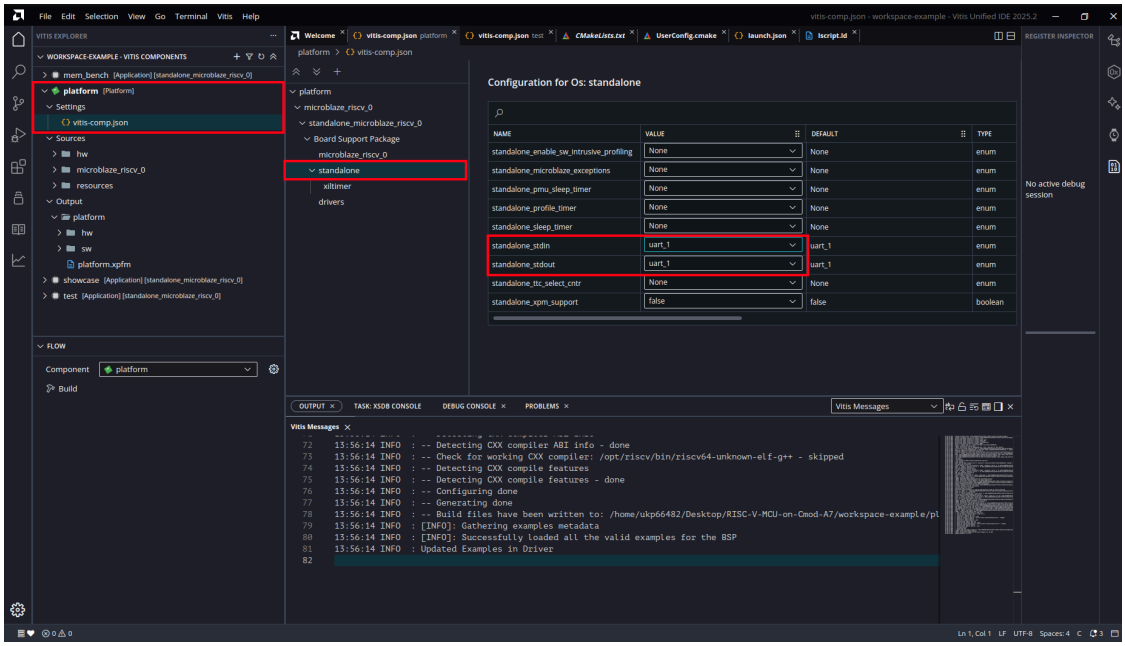


Figure 53: BSP Defaults to uart_1

Change both to `uart_USB`. The output log reports the domain being reconfigured.

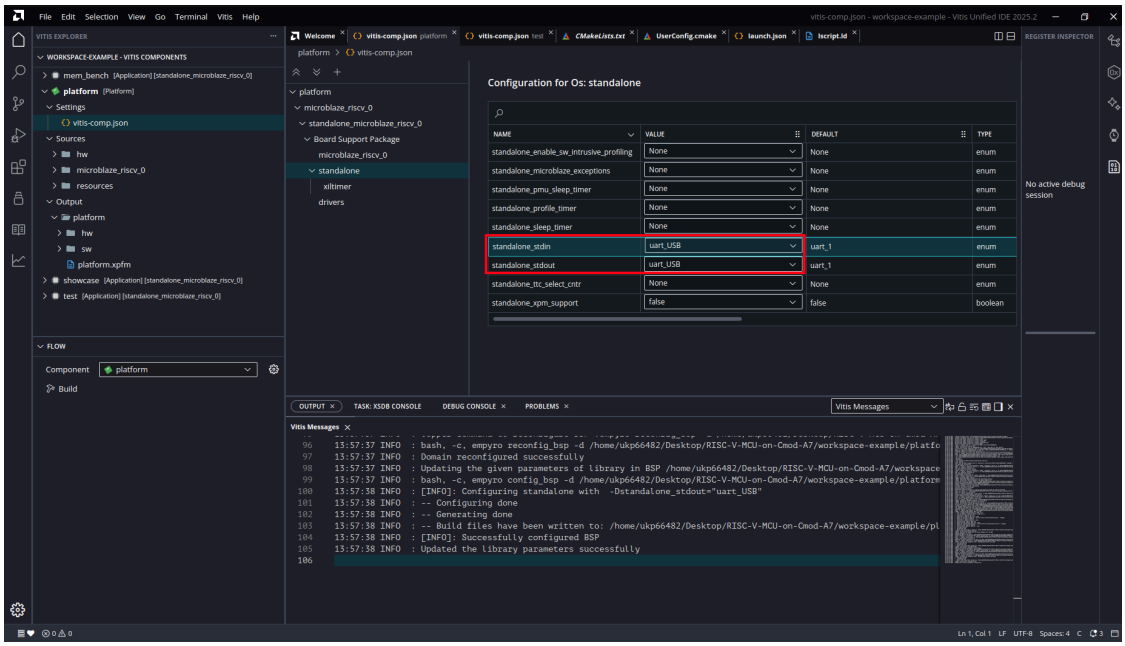


Figure 54: stdin/stdout Set to uart_USB

> **Note:** If the platform BSP is ever regenerated (for example after changing another BSP option), re-check this page; regeneration can reset stdin/stdout to `uart_1`. After changing it, rebuild both the platform and the application; the setting is compiled into the application's BSP library.

7.7 Build

7.7.1 Build the Platform

In the **FLOW** panel, select the `platform` component and click **Build**. Wait for *"Platform Build Finished successfully"*.

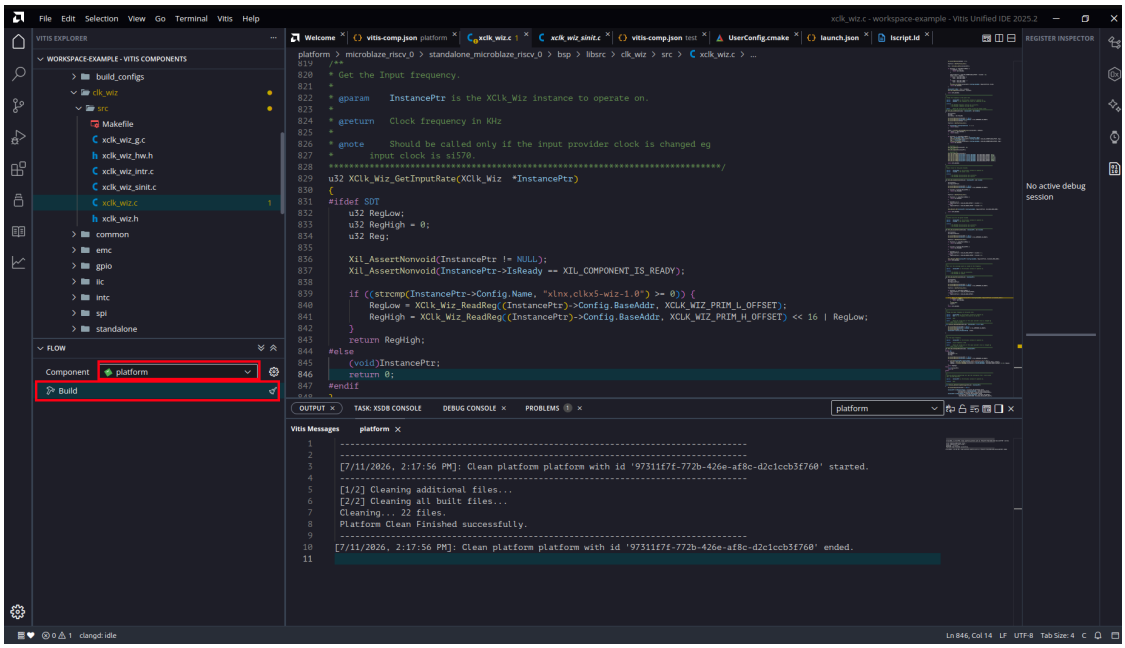


Figure 55: Build Platform

7.7.2 Build the Application

Switch the FLOW component to your application and click **Build**.

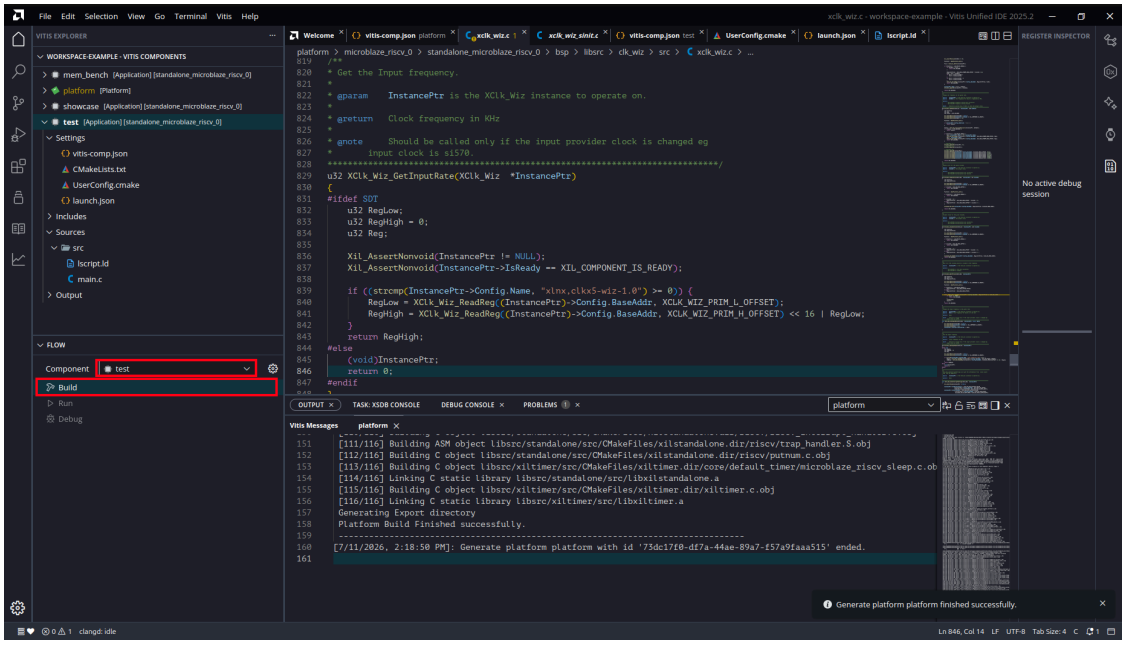


Figure 56: Build Application

The first application build asks how platform builds should be handled. Select **Always build platform with application** and click **Save in Workspace Preference**. BSP changes (such as Section 7.6) then propagate automatically on every build.

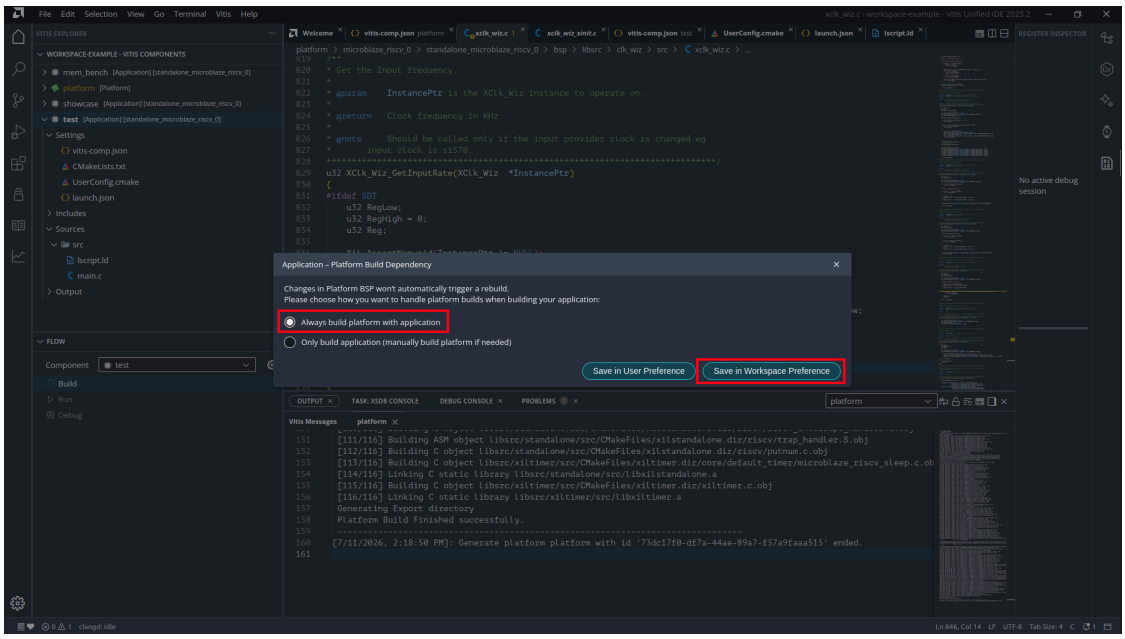


Figure 57: Platform Build Dependency

The build produces `<workspace>/test/build/test.elf`.

> **Note:** During the build, Vitis automatically links in the BSP's `boot.S` and the toolchain C library's startup object `crt0.o`: `boot.S` runs first after reset (initializes registers, stack pointer, and trap vector), and `crt0.o` clears `.bss` before calling `main()`.

7.8 Debug over JTAG

7.8.1 Start a Debug Session

In the **FLOW** panel, click **Debug**. Vitis programs the FPGA over JTAG, loads the ELF into memory, and pauses execution at `main()`.

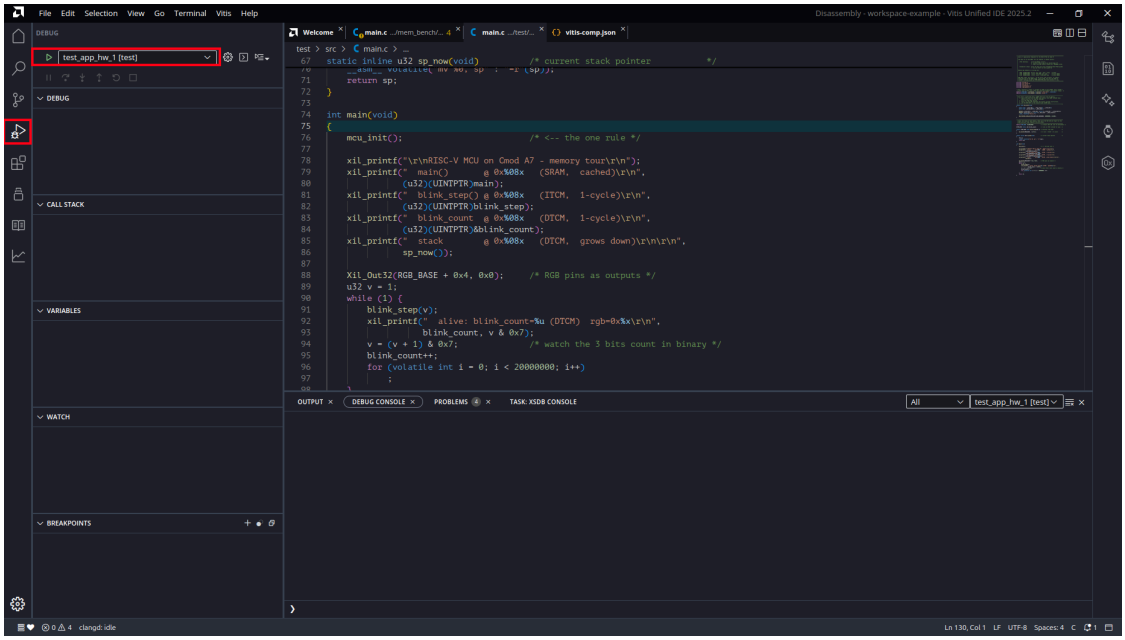


Figure 58: Debug Launched

The Debug view shows the target (*RISC-V at USER2*, *Hart #0 — PAUSED ON BREAKPOINT*), the call stack, local variables, and breakpoints. Note the call stack address: `main()` is located at `0x600009EC`, in SRAM, exactly where the linker script placed it.

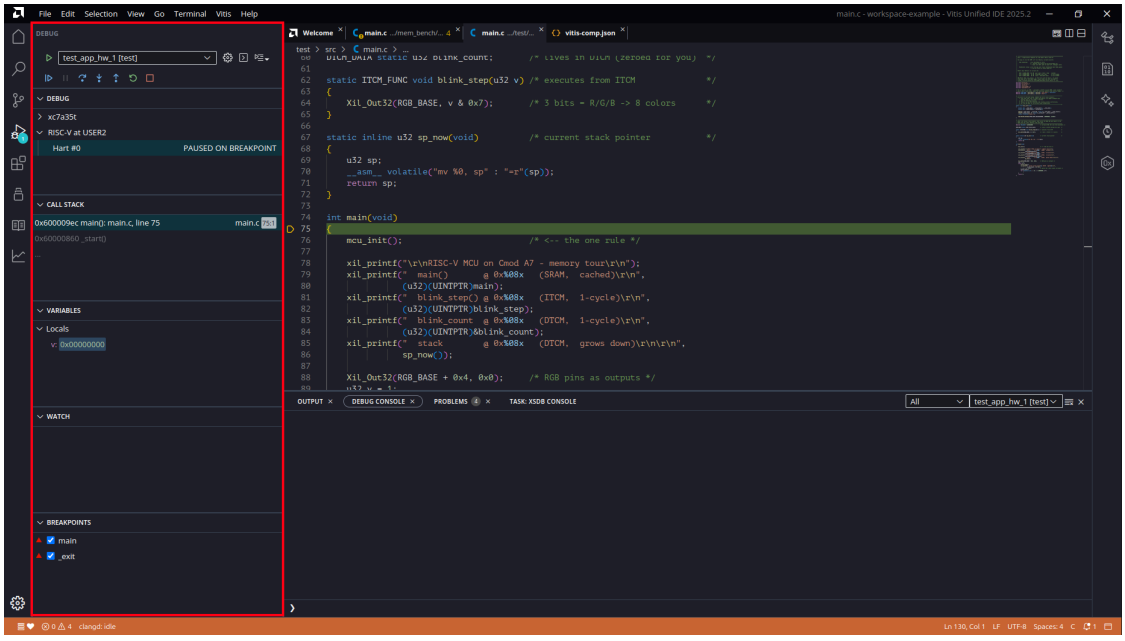


Figure 59: Paused at main()

7.8.2 Open a Serial Terminal, Then Continue

Open a serial terminal at 115200 before resuming execution.

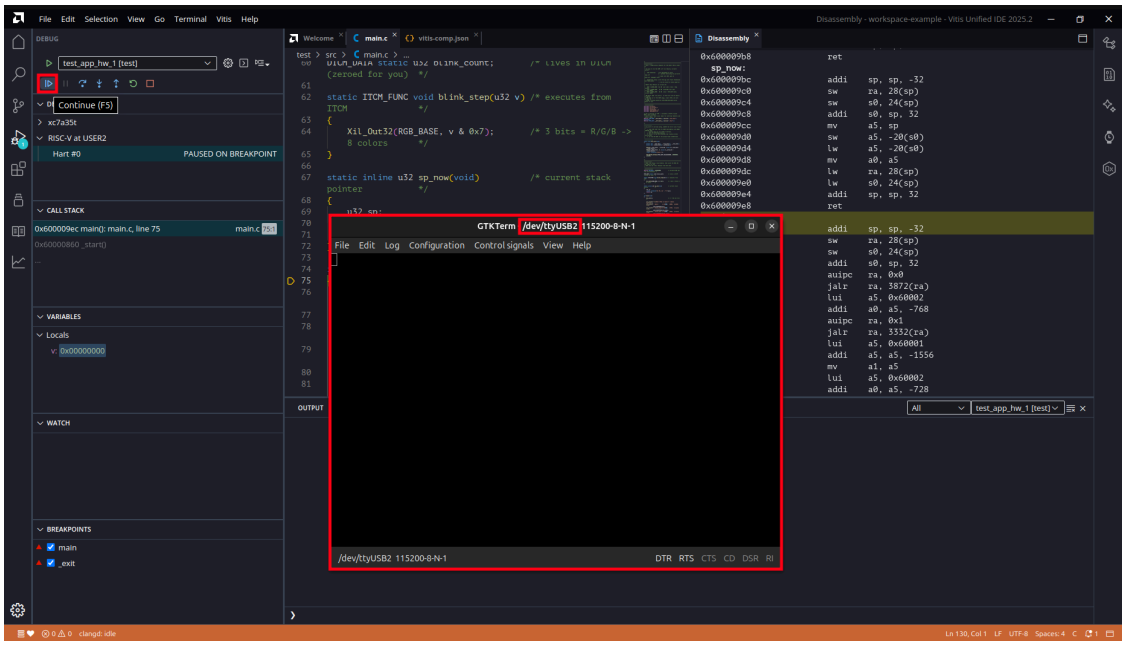


Figure 60: GTKTerm on the Board UART

Click **Continue (F5)**. The application starts: the memory tour prints once, followed by a status line about once a second, and the on-board RGB LED cycles through its colors.

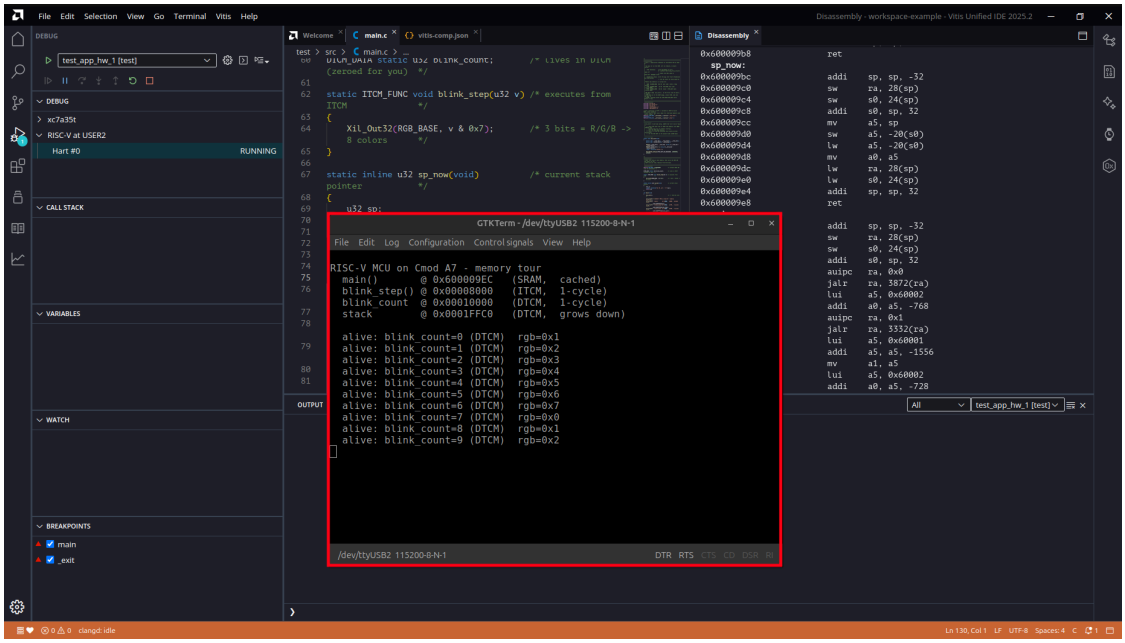


Figure 61: Memory Tour and Status Output

7.8.3 Breakpoints and Stepping

Click in the gutter next to a line number to set a breakpoint. The toolbar provides **Continue**, **Step Over**, **Step Into**, and **Step Out**. The left panel shows the call stack, variables, and watch expressions while the target is paused.

To run the application without the debugger, use **Run** in the FLOW panel instead. The loading sequence is the same, but execution does not pause at `main()`.

7.9 Inspection Tools

While a debug session is active, the **View** menu provides three inspectors:

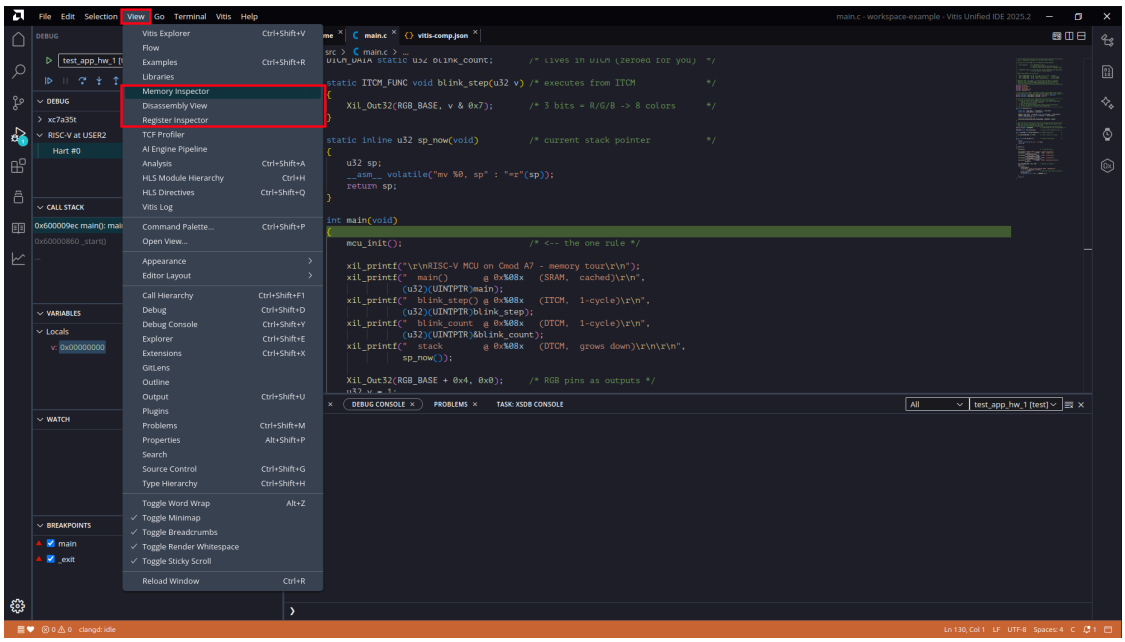


Figure 62: View Menu

7.9.1 Memory Inspector

Enter an address to inspect memory directly, for example `0x60000000` (the application image in SRAM), `0x8000` (ITCM), or `0x10000` (DTCM).

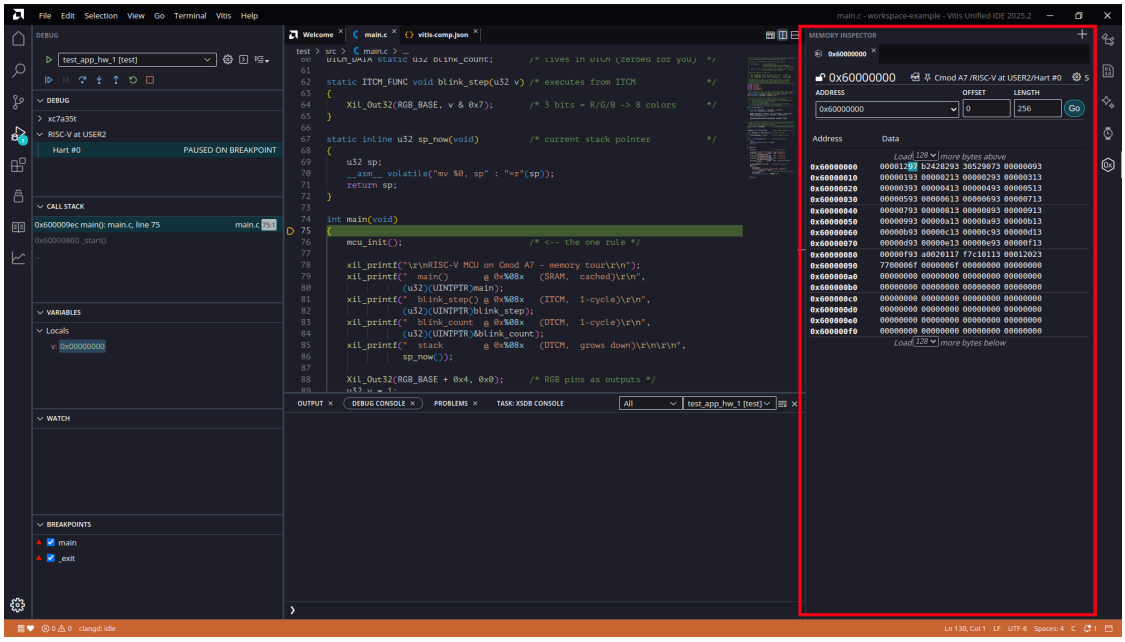


Figure 63: Memory Inspector

7.9.2 Register Inspector

The Register Inspector shows all processor registers with live values. With the target paused at `main()`, `sp` reads `0x0001FFF0`, just below the top of the DTCM, matching the linker script's stack placement (the top 16 bytes hold the startup code's argument slot).

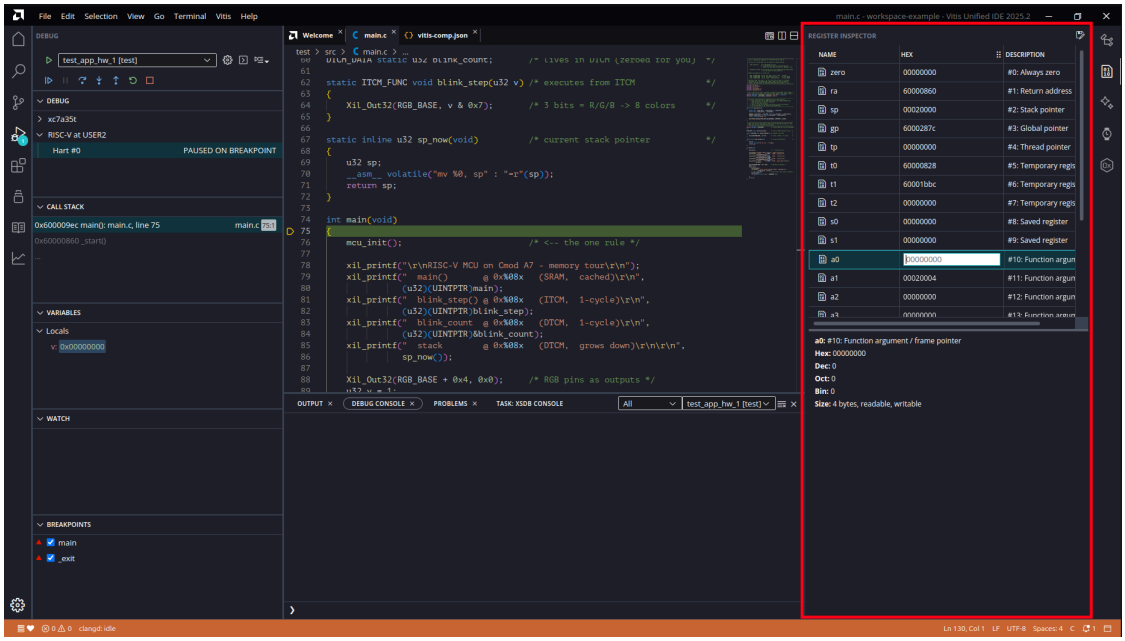


Figure 64: Register Inspector

7.9.3 Disassembly View

The Disassembly view shows the executing machine code with source interleaving. The instruction addresses around `main()` all fall in `0x6000_xxxx`; the program runs from SRAM.

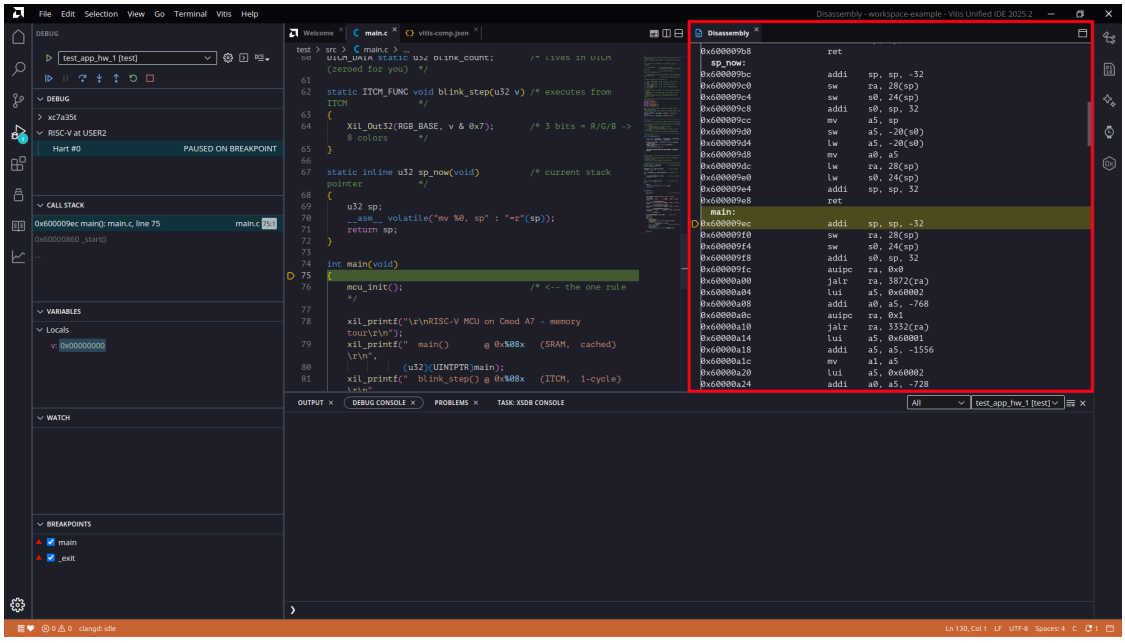


Figure 65: Disassembly

7.10 Troubleshooting

Platform build fails with `error: passing argument 2 of 'strcmp' ... [-Wint-conversion]` in `xclk_wiz.c` (reported as *"Error in generating platform"*). Two conditions combine to cause this error:

- The clock-management driver source that ships with Vitis 2025.2 (`clk_wiz` v1_10) contains a misplaced parenthesis in `xclk_wiz.c` line 839.
- If another RISC-V toolchain with GCC 14 or newer is on your PATH (for example one installed at `/opt/riscv` for assembly coursework), the BSP build may select it instead of the Vitis-bundled compiler. GCC 14+ treats this construct as an error; the bundled compiler treats it as a warning.

The offending line as displayed in the IDE:

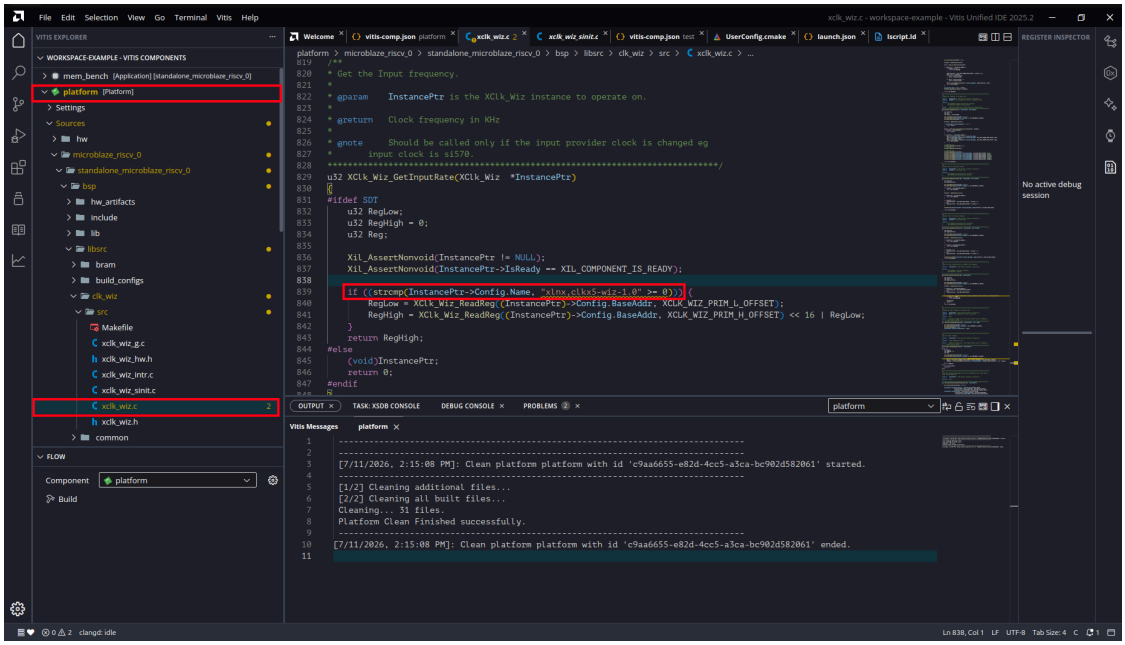


Figure 66: clk_wiz Driver Bug

Fix the source (this fix works with either compiler): change line 839 from

```
strcmp(InstancePtr->Config.Name, "xlnx,clkx5-wiz-1.0" >= 0)
```

to

```
strcmp(InstancePtr->Config.Name, "xlnx,clkx5-wiz-1.0") >= 0
```

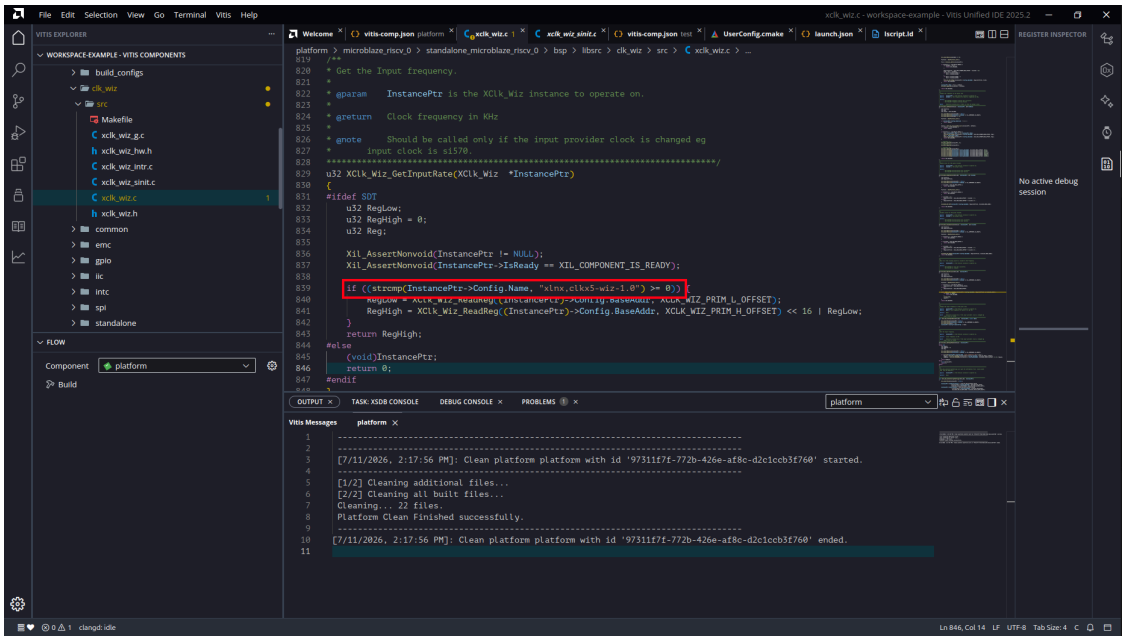


Figure 67: After the Fix

Then rebuild the platform. Apply the same one-line fix to the Vitis installation copy at `<install>/2025.2/data/embeddedsw/XilinxProcessorIPLib/drivers/clk_wiz_v1_10/src/xclk_wiz.c`, where `<install>` is the Xilinx installation directory (`/opt/Xilinx` by default on Linux). The platform re-imports driver sources whenever the BSP is regenerated, so an unpatched installation reintroduces the error.

To check which compiler configured the BSP, read `platform/.../bsp/libsrc/build_configs/gen_bsp/compile_commands.json`. To force reselection, delete `platform/.../bsp/libsrc/build_configs` and rebuild from a session whose PATH does not contain the other toolchain.

The application runs (RGB LED cycles) but the USB console is silent. `xil_printf` output is directed to the DIP-header UART. The BSP stdout has reverted to `uart_1`: redo Section 7.6, then rebuild both the platform and the application. The compiled setting is recorded in `platform/export/platform/sw/standalone_microblaze_riscv_0/include/bspconfig.h`: `STDOUT_BASEADDRESS` must read `0x40300000` (uart_USB), not `0x40310000` (uart_1).

8 Standalone Boot

This chapter describes how an application is stored in the QSPI flash and started automatically at power-on. A small bootloader (AMD's standard SREC SPI Bootloader, carried inside the hardware image) copies the application from flash into RAM and starts it, with no PC attached, in under a second after power is applied.

The walkthrough covers the complete path: building the bootloader, merging it into the hardware image, converting an application to a flash image, and programming both into the flash. Everything after the hardware image itself is done in the Vitis Unified IDE.

8.1 How Standalone Boot Works

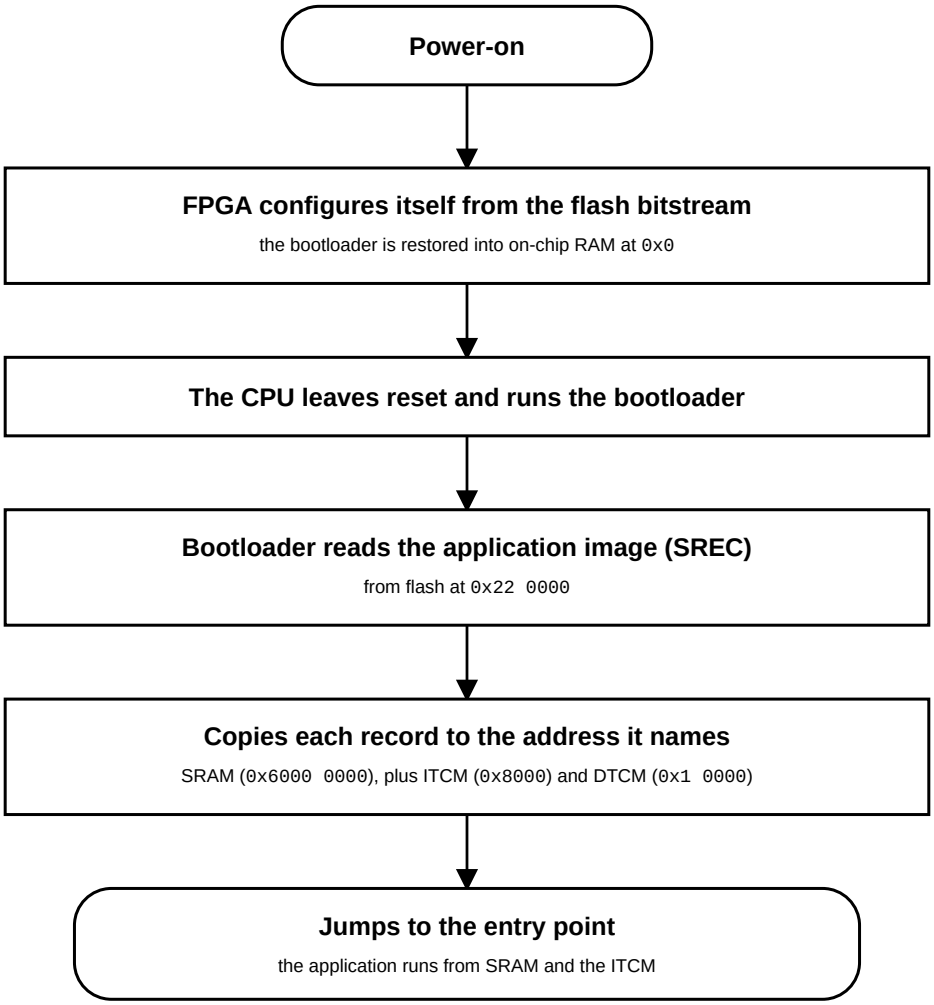


Figure 68: Standalone boot sequence: the FPGA restores the bootloader, which copies the application from flash into SRAM, ITCM, and DTCM and runs it

The flash holds two independent regions, read by two different readers:

Flash offset	Contents	Read by
`0x000000–0x21FFFF`	Hardware image (bitstream, with the bootloader in its Block RAM initialization)	The FPGA configuration engine, at power-on
`0x220000–0x3FFFFF`	Application slot (SREC image, 1.875 MB)	The bootloader

Block RAM is volatile, so the bootloader cannot live there across a power cycle. Its persistent copy travels inside the bitstream: a bitstream carries the initial contents of every block RAM, and the CPU's local memory is built from block RAM. At every power-on the configuration engine reloads the bitstream from flash, which restores the bootloader, so application updates never touch it. The CPU then leaves reset at address `0x0` and executes it.

The application is stored as an SREC image (Motorola S-record), a text format in which every line carries its own destination address. The bootloader copies each record to the address it names, so the linker script decides where the program runs. The course linker script (`mcu/cmod_mcu.ld`) places the program in SRAM at `0x6000_0000`, and `ITCM_FUNC` code and `DTCM_DATA` variables in the ITCM and DTCM, so the image is loaded into those three memories.

Memory	Size	Role
QSPI flash	4 MB	hardware image + bootloader · application slot @ `0x220000` (non-volatile)
SRAM @ `0x60000000`	512 KB	where the application executes (code/data/heap, I/D-cached)
BRAM @ `0x00000000`	128 KB	32 KB bootloader · 32 KB ITCM @ `0x8000` · 64 KB DTCM @ `0x10000`

> **Note:** The repository ships the results of this walkthrough prebuilt in `release/`: `boot_srec.bit` is the hardware image with the bootloader already merged in. To deploy an application with the stock bootloader, skip directly to Section 8.7.

8.2 Prerequisites

- A workspace with the platform and a built application, prepared as described in Section 7
- The hardware image and its memory map from the repository: `release/top.bit` and `release/top_wrapper.mmi`
- Cmod A7-35T board connected via its micro-USB cable
- A serial terminal program for the board's console output

8.3 Create the Bootloader Component

The bootloader is not custom code: it is the stock **SREC SPI Bootloader** example that ships with Vitis, built like any other application component.

8.3.1 New Application from Template

Open the **Examples** view from the left activity bar. Under *Embedded Software Examples*, select **SREC SPI Bootloader** and click **Create Application Component from Template**.

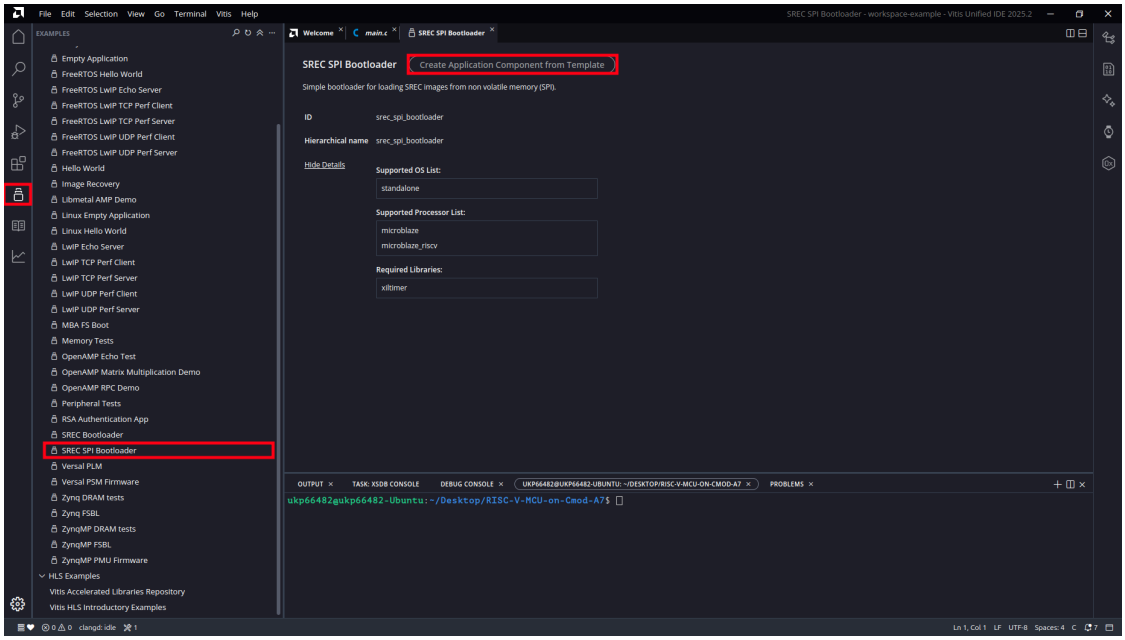


Figure 69: SREC SPI Bootloader Template

8.3.2 Name and Location

Keep the proposed name `srec_spi_bootloader` and the workspace directory as the location. Click **Next**.

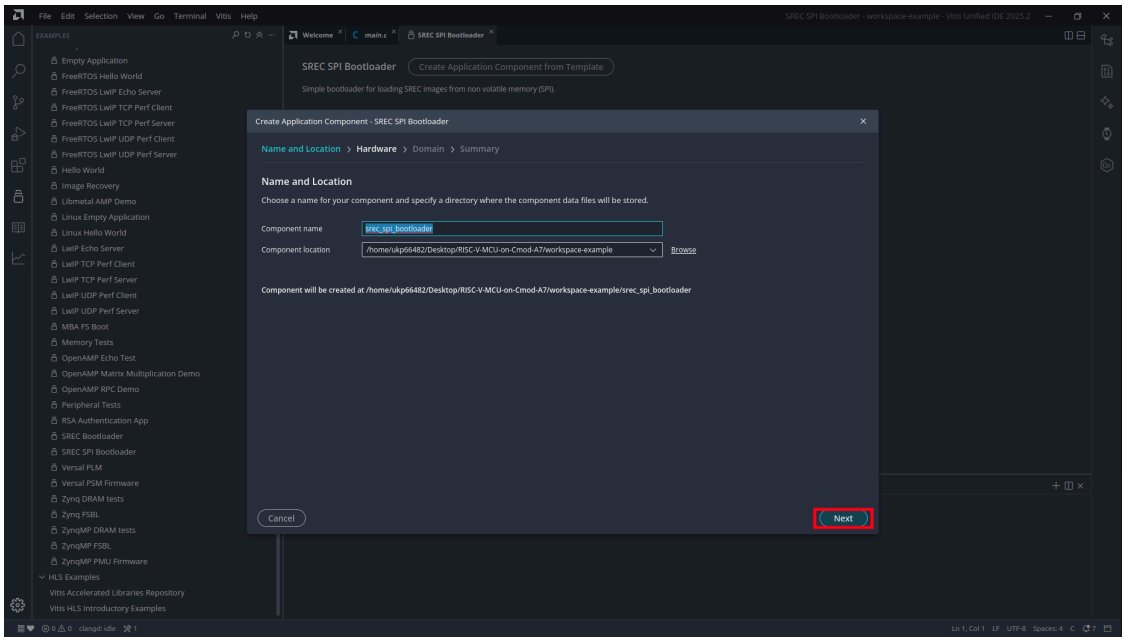


Figure 70: Bootloader Name and Location

8.3.3 Select the Platform

Select the `platform` component (Board: `cmod_a7-35t`). Click **Next**.

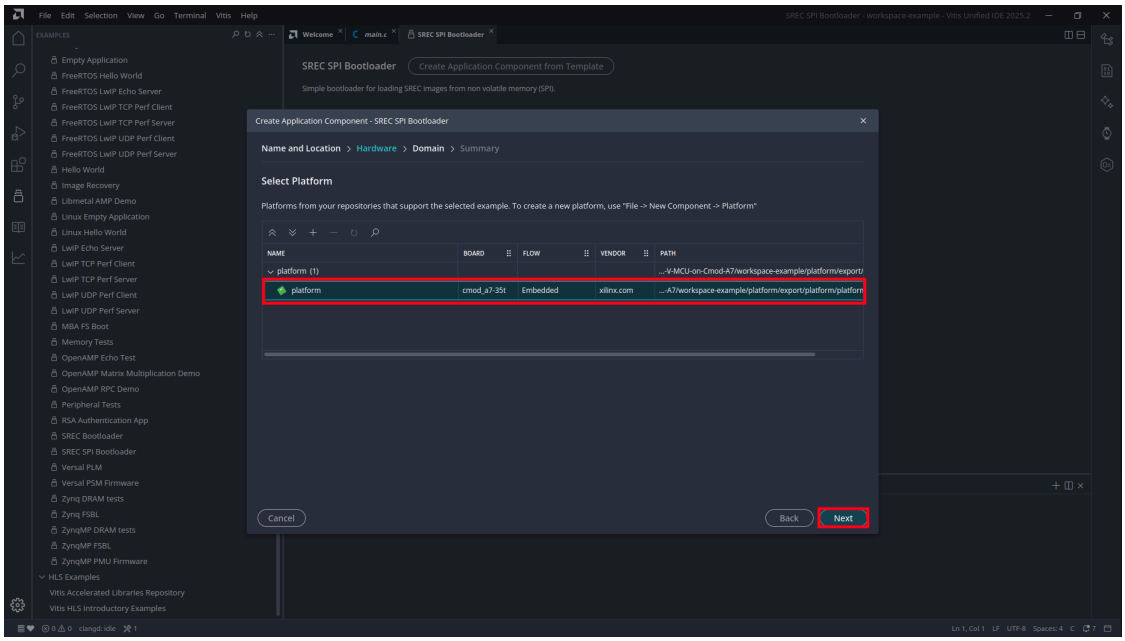


Figure 71: Select Platform

8.3.4 Summary

Review the settings (the domain is `standalone_microblaze_riscv_0`) and click **Finish**.

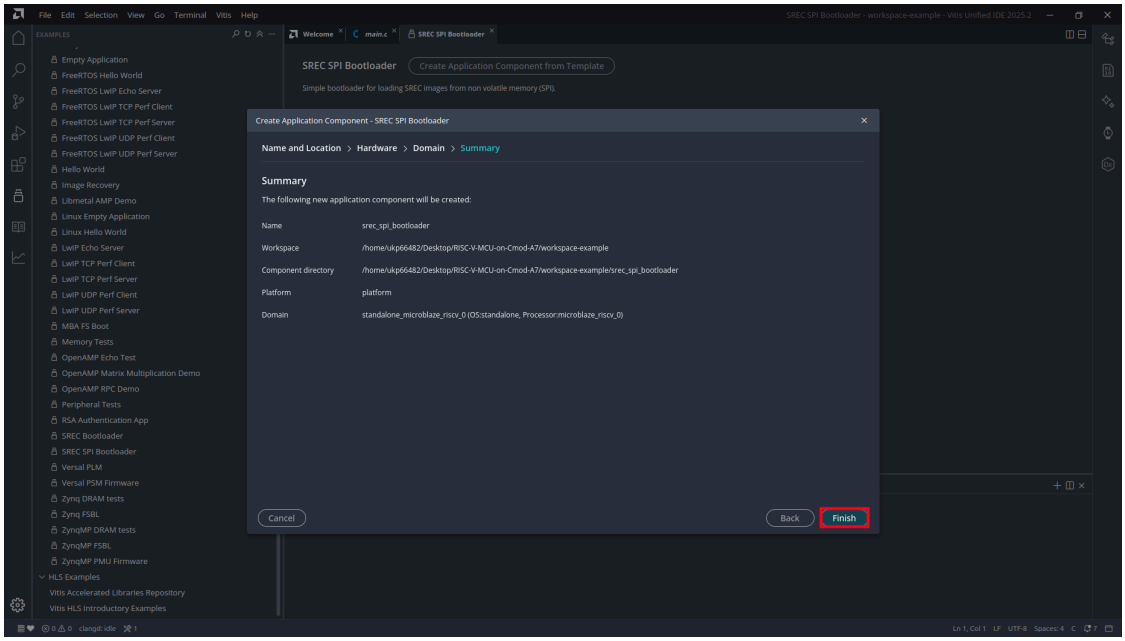


Figure 72: Bootloader Summary

> **Note:** The template automatically selects this board's QSPI flash controller, and it links the bootloader to Block RAM at `0x0`, below the application's ITCM and DTCM, so the bootloader can load an application without overwriting itself. Neither setting needs to be adjusted.

8.4 Configure the Bootloader

Two source edits configure the template for this board. Both files are under the component's `src/` folder.

8.4.1 Set the Application Slot Address

`blconfig.h` defines where in the flash the bootloader looks for the application. The template ships with a placeholder address and a `#warning` that reminds you to change it:

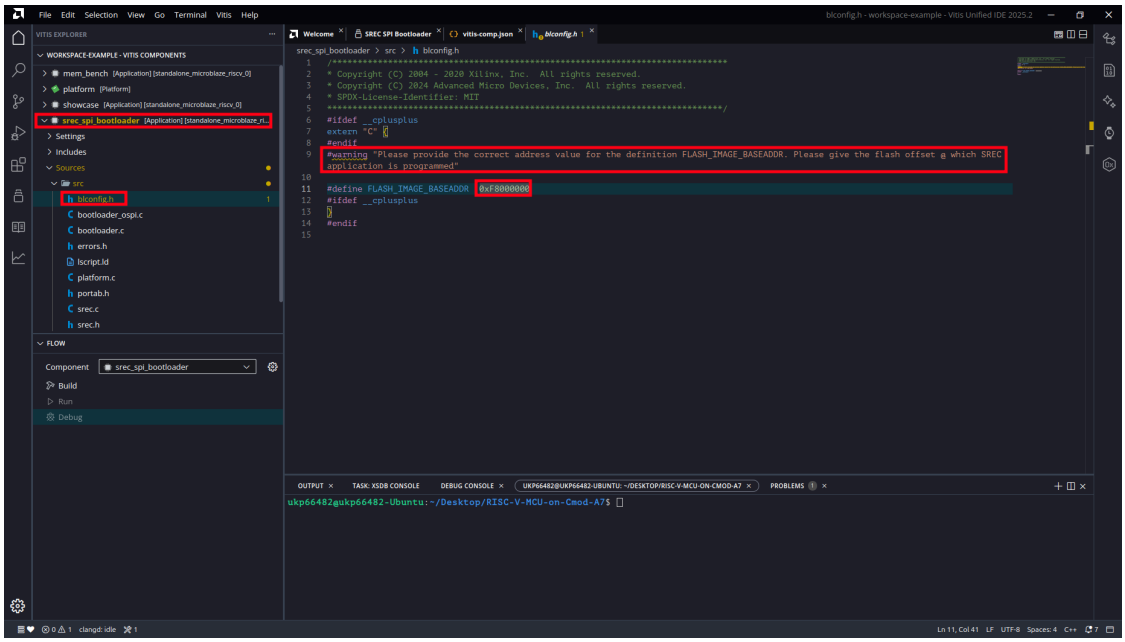


Figure 73: blconfig.h As Generated

Set `FLASH_IMAGE_BASEADDR` to `0x00220000` (the application slot, which starts immediately after the bitstream region) and comment out the `#warning` line:

```
#define FLASH_IMAGE_BASEADDR  0x00220000
```

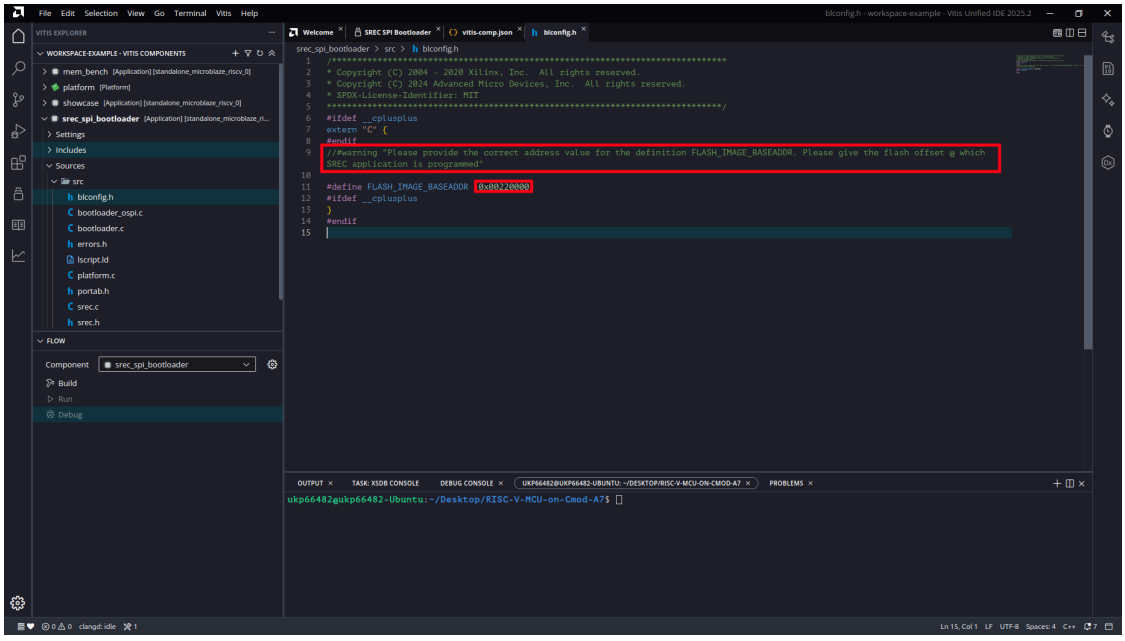


Figure 74: blconfig.h Configured

8.4.2 Disable Verbose Output

`bootloader.c` defines `VERBOSE` by default (line 45):

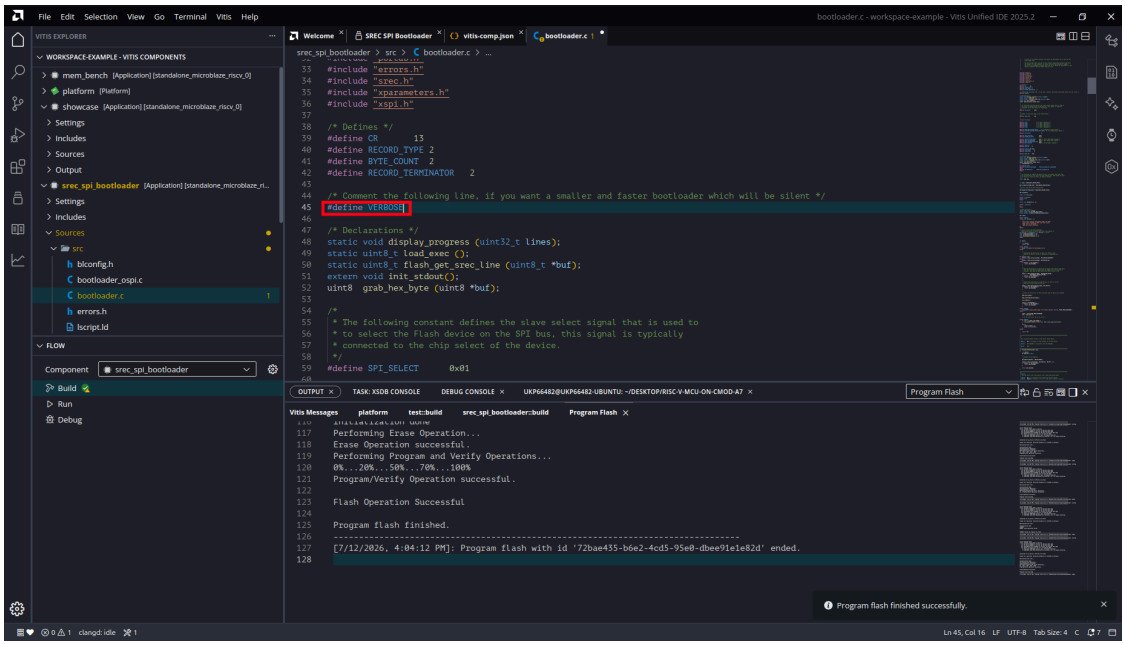


Figure 75: VERBOSE As Generated

Comment it out:

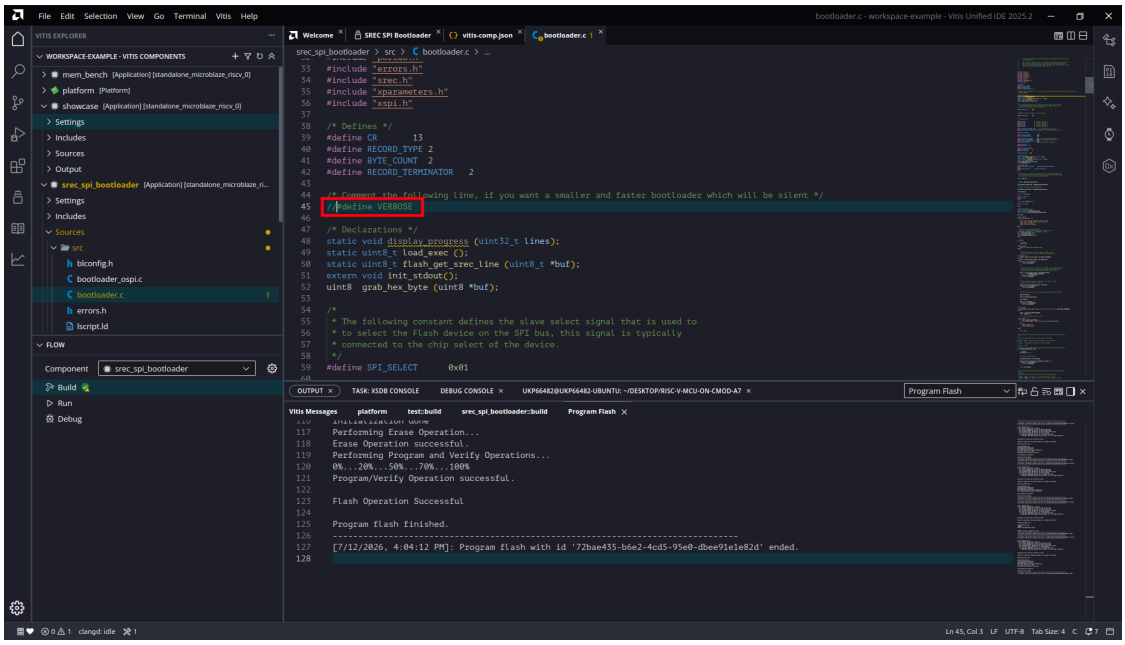


Figure 76: VERBOSE Disabled

> **Note:** With `VERBOSE` enabled the bootloader prints progress for every record it copies, through a UART whose baud divisor it never initializes. The application still boots, but loading takes many seconds instead of well under one second. Disabling `VERBOSE` gives a silent bootloader and restores the sub-second boot. The definition returns whenever the template is regenerated, so re-check this line when creating a new bootloader component.

8.5 Build the Bootloader

In the **FLOW** panel, set the component to `srec_spi_bootloader` and click **Build**.

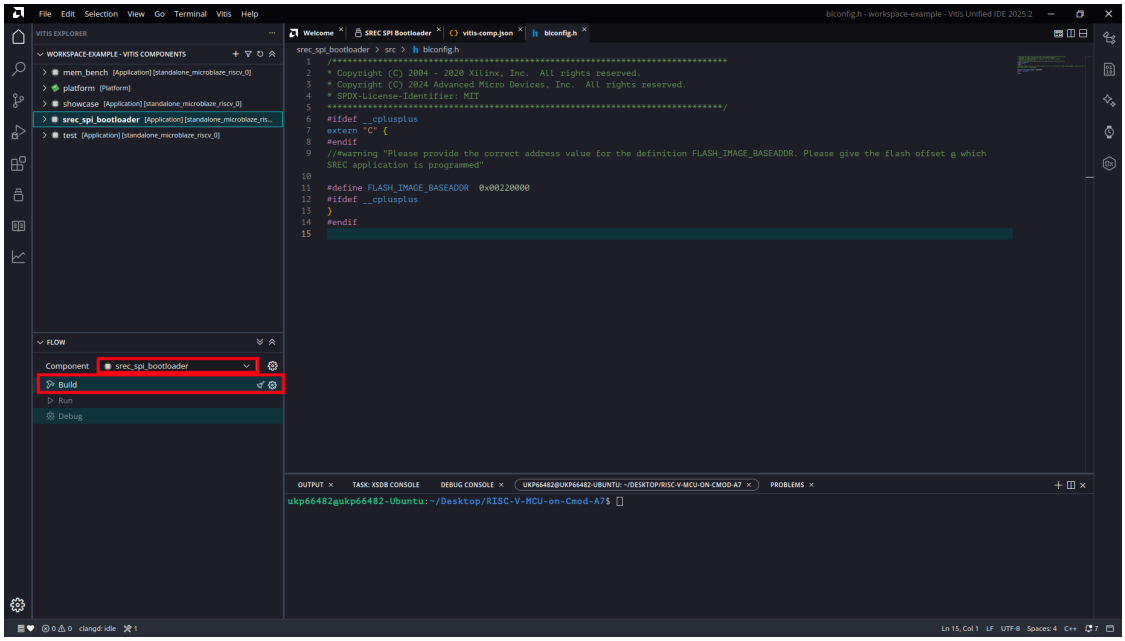


Figure 77: Build the Bootloader

The build produces `srec_spi_bootloader.elf` under the component's `Output` tree (about 16 KB of code, well inside its 32 KB Block RAM region).

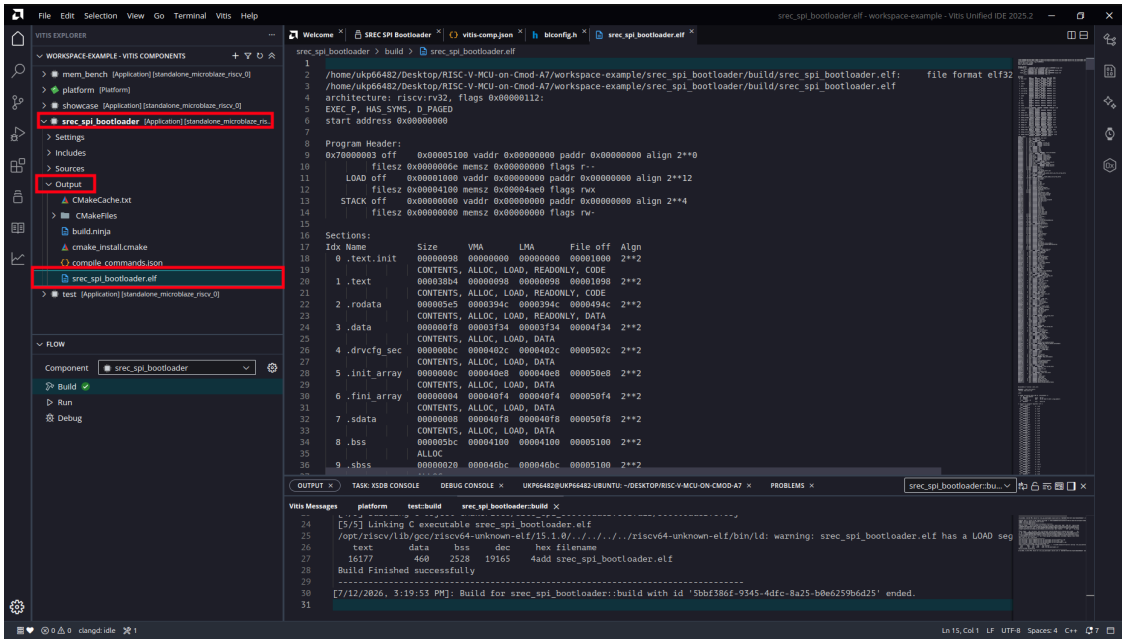


Figure 78: Bootloader ELF Built

8.6 Merge the Bootloader into the Hardware Image

The bootloader ELF is merged into the bitstream's Block RAM initialization with `updatemem`, a command-line tool that ships with Vitis. Only the RAM initialization words change; the logic, routing, and timing of the hardware image are untouched, so the merge takes seconds and needs no hardware rebuild.

Open a terminal in the repository root (**Terminal > New Terminal**) and run:

```
updatemem -force \
  -meminfo release/top_wrapper.mmi \
  -data    workspace-example/srec_spi_bootloader/build/srec_spi_bootloader.elf \
  -bit     release/top.bit \
  -proc    top_i/microblaze_riscv_0 \
  -out     release/boot_srec.bit
```

The inputs and what they mean:

Argument	File	Purpose
`-meminfo`	`top_wrapper.mmi`	Which physical Block RAM cell holds which CPU address (fixed at hardware placement)
`-data`	`srec_spi_bootloader.elf`	The program to place into Block RAM
`-bit`	`top.bit`	The hardware image, Block RAM blank
`-proc`	`top_i/microblaze_riscv_0`	Which processor's memory to initialize
`-out`	`boot_srec.bit`	The bootable hardware image

> **Note:** A successful run prints `updatemem completed successfully` and rewrites the output file. If the tool prints only its usage text, one of the paths is wrong and no output is produced; check the output file's timestamp before using it. The MMI file must come from the same hardware build as the bitstream.

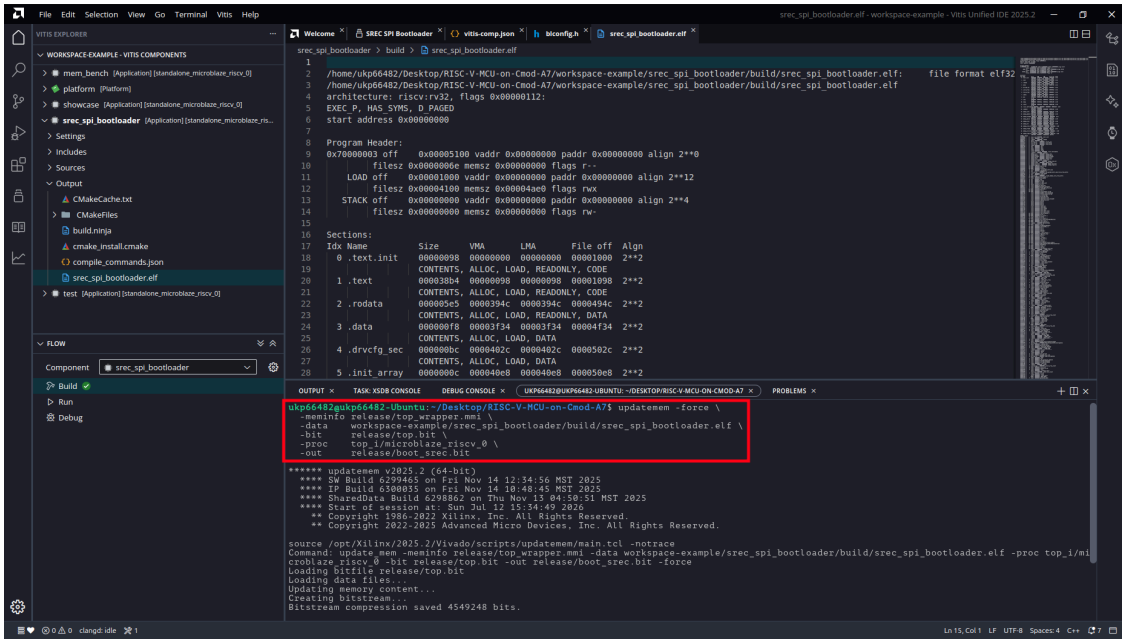


Figure 79: updatemem in the Integrated Terminal

8.7 Prepare the Application Image

The bootloader reads SREC text, so the application ELF is converted with the toolchain's `objcopy`. This walkthrough deploys the `test` application built in Section 7; the same ELF runs unchanged under JTAG and from flash.

```
riscv32-amd-linux-gnu-objcopy -O srec \
  workspace-example/test/build/test.elf \
  workspace-example/test/build/test.srec
```

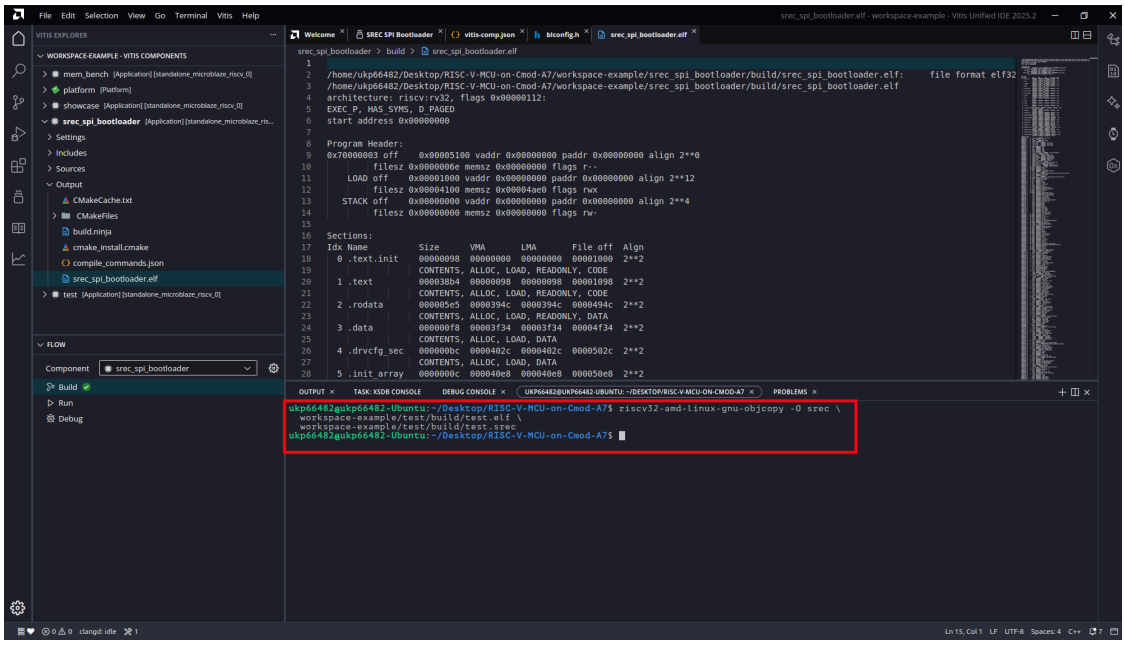


Figure 80: objcopy to SREC

Then give the image a `.bin` extension:

```
cp workspace-example/test/build/test.srec \
   workspace-example/test/build/test_srec.bin
```

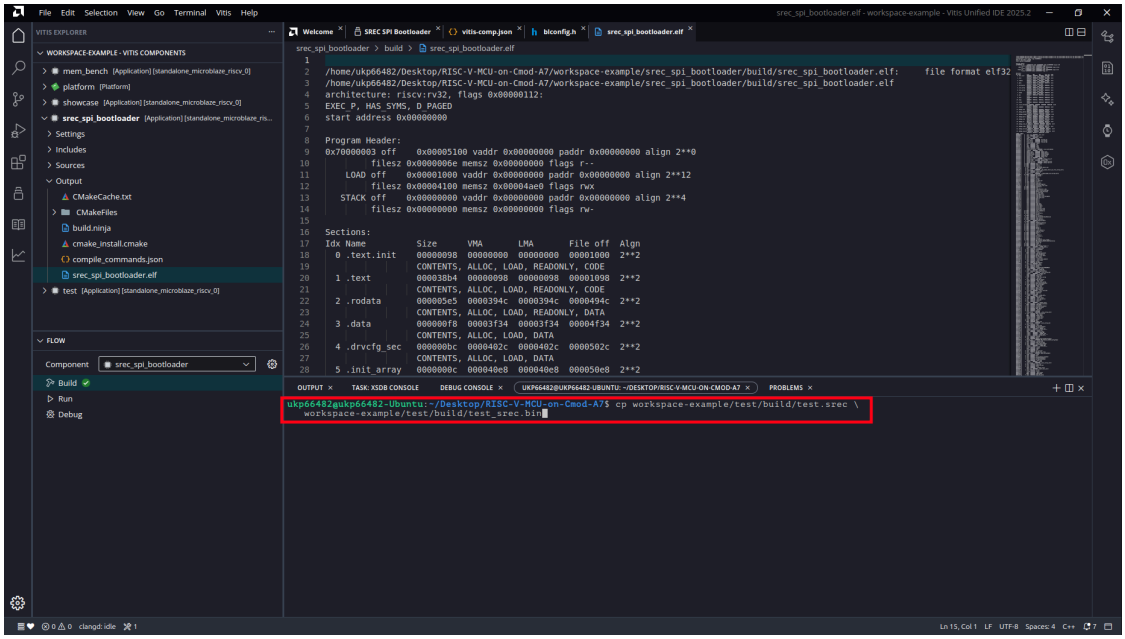


Figure 81: Rename to .bin

> **Note:** The extension matters. The flash programmer treats a `.srec` file as an image format and decodes it to the memory addresses inside it; a `.bin` file is written to the flash byte for byte, which is what the bootloader expects to find in the slot. The file name itself is free; only the `.bin` extension is significant.

Each S3 line of the SREC text embeds its destination address; with the course linker script every record falls in SRAM (addresses beginning `6000`), the ITCM (`00008000–0000FFFF`), or the DTCM (`00010000–0001FFFF`), and none lies below `0x0000_8000`, where the bootloader lives. The image is about three times the size of the binary it encodes; the 1.875 MB slot therefore never limits an application before the 512 KB SRAM does.

8.8 Program the Flash

Both flash regions are written from **Vitis > Program Flash…** in the menu bar. Each write erases and programs only the address range of its own image, so the two regions never disturb each other.

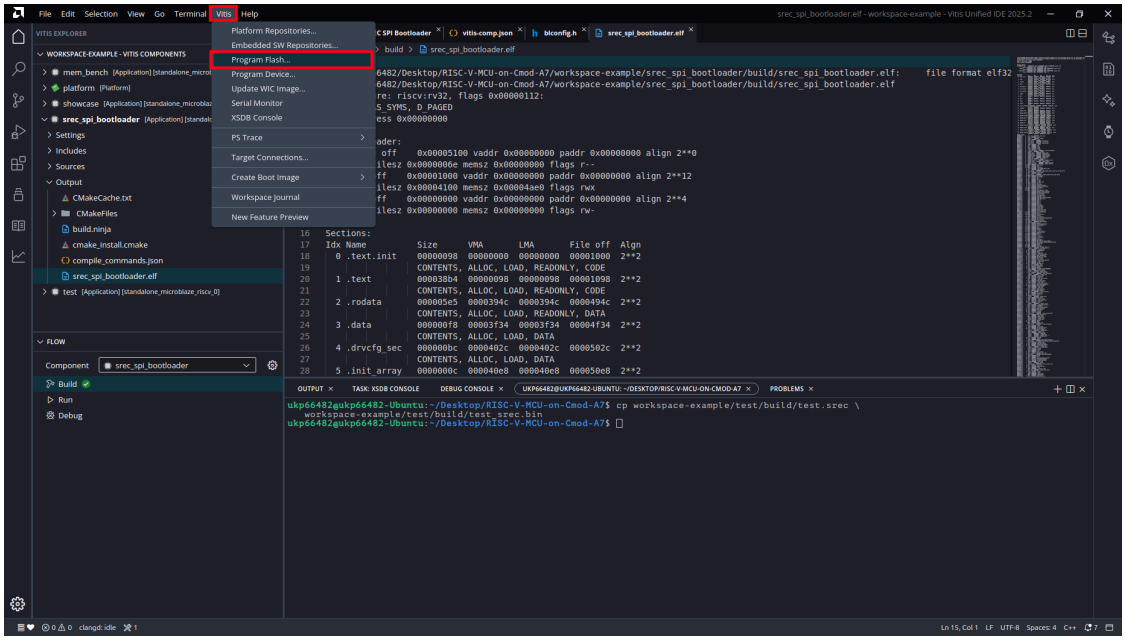


Figure 82: Vitis Program Flash Menu

8.8.1 Program the Hardware Image

This step is needed once, and again only after the bootloader or the hardware design changes. In the *Program Flash Memory* dialog set:

Field	Value
Image File	`release/boot_srec.bit`
Offset	`0x0`
Flash Type	`mx25l3273f-spi-x1_x2_x4`
Verify after flash	checked

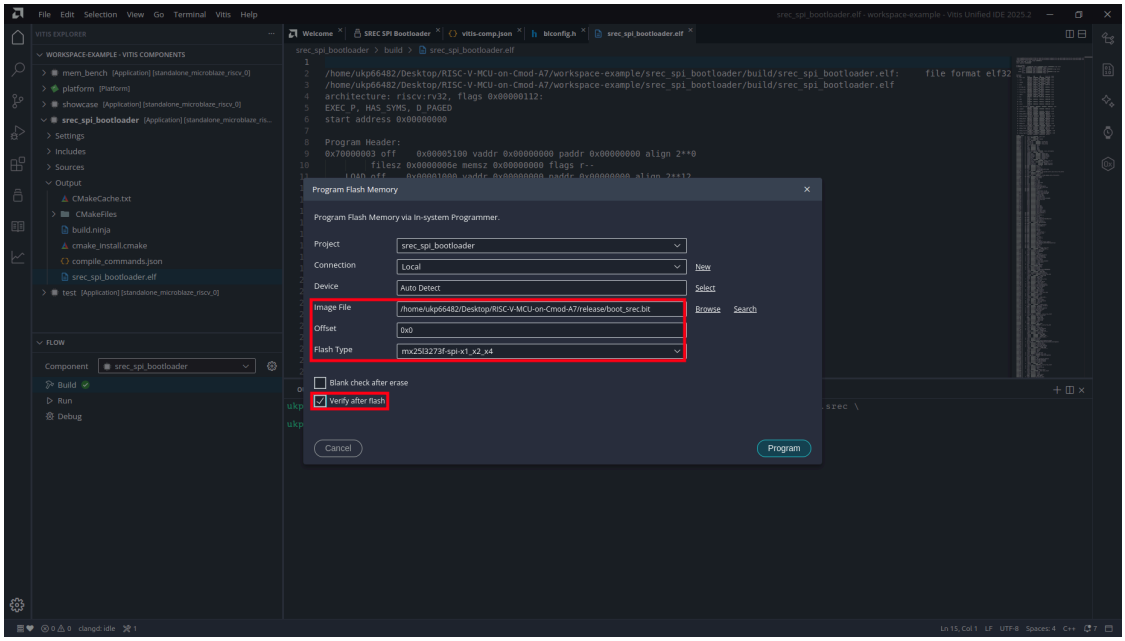


Figure 83: Program the Hardware Image

Click **Program** and wait for *Program flash finished.*

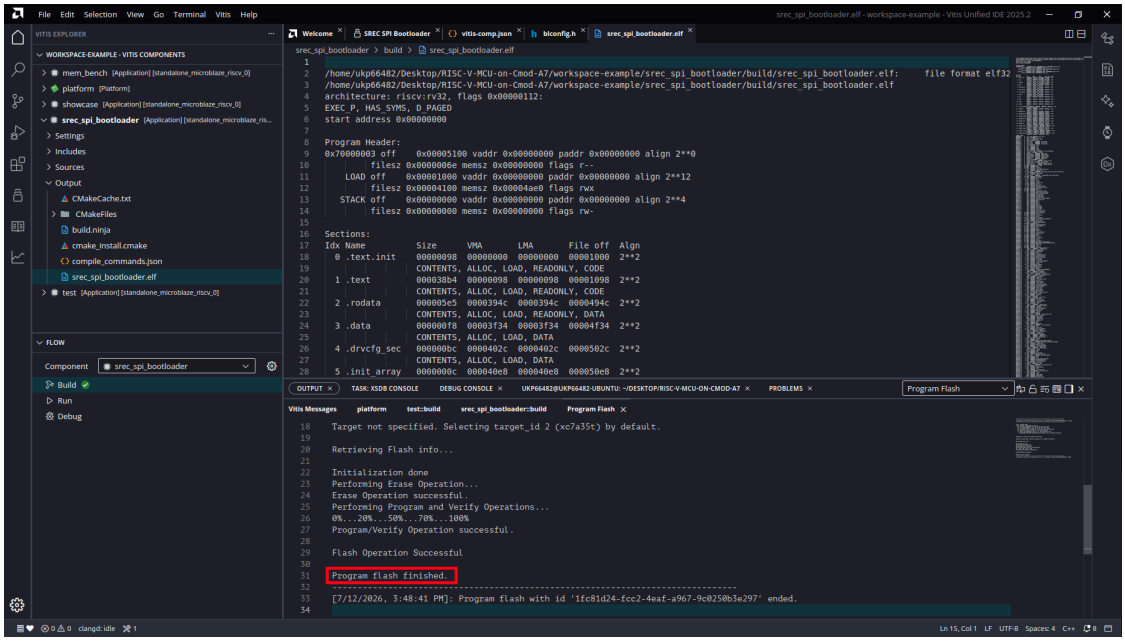


Figure 84: Hardware Image Programmed

> **Note:** The flash type entry covers the board's Macronix MX25L3273F device. Older boards carry a Micron N25Q032A instead; if the IC3 marking says N25Q032, select `n25q32-3.3v-spi-x1_x2_x4`.

8.8.2 Program the Application

Open **Vitis > Program Flash…** again and set:

Field	Value
Image File	the `.bin` application image
Offset	`0x220000`
Flash Type	`mx25l3273f-spi-x1_x2_x4`
Verify after flash	checked

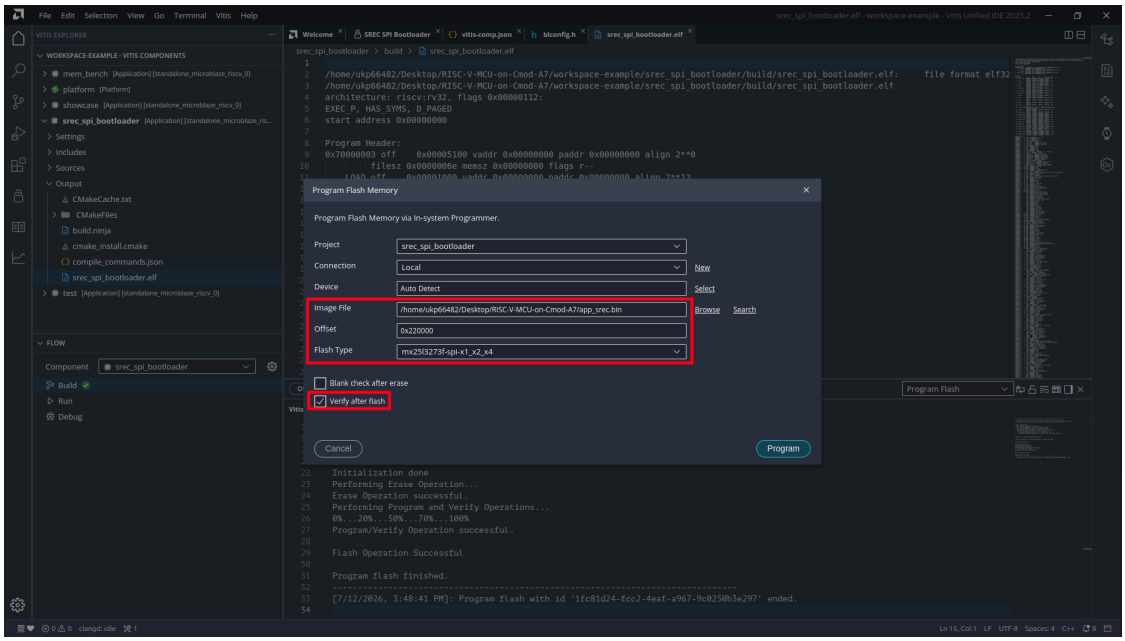


Figure 85: Program the Application Slot

Click **Program** and wait for *Program flash finished.*

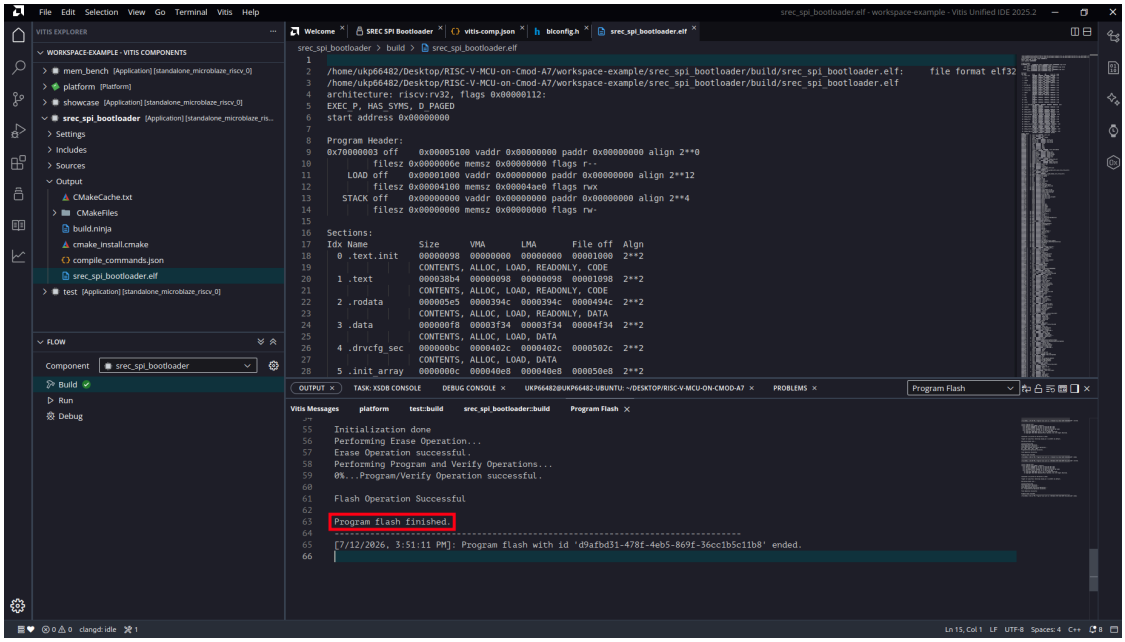


Figure 86: Application Slot Programmed

8.9 Boot and Verify

Disconnect the USB cable and plug it back in. This is a true cold boot: the FPGA configures itself from the flash, the bootloader loads the application, and the application runs, with no debugger and no IDE involved.

Open a serial terminal at 115200. The application output appears as soon as the board finishes booting.

> **Note:** The reset button (BTN0) restarts the CPU into the bootloader, which re-copies the application from flash, so a reset behaves like a power-cycle. Modified `.data` globals are restored because the image is re-read from flash at every boot.

8.10 Updating the Application

After the one-time setup above, the day-to-day deployment loop has three steps, none of which touch the bootloader or the hardware image:

1. Build the application in Vitis.
2. Convert and rename the image (the two commands in the application image step).
3. **Vitis > Program Flash…** to offset `0x220000`.

Power-cycle, and the new application boots.

8.11 Working with JTAG Debug Mode

Both modes coexist on the same board with no switching:

	JTAG (Vitis Run/Debug)	Standalone (flash)
Code goes to	RAM directly (volatile)	Flash (persistent)
After power-cycle	the flashed application boots	the deployed application boots
Breakpoints / stepping	yes	no
Typical use	daily development loop	shipping a finished program

The typical development cycle is edit, build, and JTAG Run/Debug. When the program works, deploy it to flash once. JTAG always takes priority over the bootloader: it halts the CPU before the bootloader runs, so the flash contents cannot interfere with a debug session.

8.12 Troubleshooting

1. **Boot works but takes many seconds:** `VERBOSE` is still defined in `bootloader.c`. Comment it out, rebuild the bootloader, re-run `updatemem`, and reprogram the hardware image.
2. **Nothing at all after power-on (no LED, no output):** The flash holds `top.bit` (blank Block RAM, no bootloader) instead of `boot_srec.bit`, or the hardware image region is empty or corrupt. Reprogram the hardware image. JTAG remains available for recovery.
3. **`updatemem` printed only usage text:** One of the argument paths is wrong; no output file was written, and reprogramming would flash the previous image. Fix the path and confirm the `updatemem completed successfully` message.
4. **Program Flash fails with `Flash Operation Failed`:** Another process is holding the JTAG connection, typically a Vitis debug session or an XSDB console. Close it (or unplug and replug the board) and retry.
5. **Garbage on the terminal:** The baud rate must be 115200. The course template sets the UART divisor in `mcu_init()`; the plain Hello World template does not set it at all.
6. **Deployed, but silent after power-on (works under JTAG):** The ELF was not linked with the course linker script, `mcu/cmod_mcu.ld`. The bootloader honors the SREC record addresses, and a default all-BRAM layout collides with the bootloader itself. Rebuild against the `mcu/` folder and redeploy.
7. **Board boots an old application:** The flash write did not complete. Repeat the application programming step and confirm the verify pass succeeds.
8. **Power-on boots normally, but the reset button (BTN0) hangs the board:** The application was linked with an earlier course linker script, which wrote over the bootloader's first instructions at every boot. A power cycle restores the bootloader; a reset does not. Rebuild against the current `mcu/cmod_mcu.ld` and redeploy, then check the SREC text: in every `S3` line the address (the eight hex digits after `S3` and the two-digit byte count) must be `00008000` or higher.